

# Leant2: A Unified Architecture Proposal

Lean-native synthesis of programs and their contracts,  
consolidated from nine independent design proposals

Prepared for Vladimir Reshetnikov

September 17, 2026

## Abstract

Nine independently prepared design proposals for Leant2, a Lean-native successor to the Haskell synthesizers Leant and Djex, were written against the same repository snapshots and the same Lean 4.34.0 toolchain. They agree on more than they disagree. This document consolidates them into one proposal. Its architectural core is the consensus of all nine: exact Lean expressions as the only semantic authority; a transactional search whose unit of backtracking is a partial program together with *all* of its unfinished obligations; recursion admitted only through typed schemas with explicit recursive-call capabilities; behavioral information with graded authority, where tests refute, proofs certify, and only proof-producing analyses may prune; and one acceptance gate that kernel-checks the exact program and the exact contract before anything is published. Where the proposals differ, this document selects the best-developed treatment and records its origin.

Part I fixes the architecture. Part II gives the algorithms: the construction search and spine expansion, dependent elimination and transport, retrieval and carrier invention, recursion-schema and motive synthesis, contract propagation and certified counterexample-guided refinement, scheduling and memoization, and the proof-service and acceptance procedures, each as pseudocode merged from the best-specified proposals, with the worked traces that recur across them. Two requirements come from the project owner rather than from the proposals. First, Leant2 has a single adaptive engine: no engine switches and, for now, no user-facing settings. Second, the minimal acceptance baseline is the existing Leant `:synth` golden corpus, thirty transcripts with 278 queries, read semantically: Leant2 must match or exceed every recorded outcome, but may return better candidates in a different order. Nothing here adds new experimental evidence; the evidence sections cite the nine packages and the sibling project Forge, whose kernel-checked certificate checkers and orchestration frontend Leant2 should reuse.

**Status.** This is a design specification. The nine source proposals each contain a small remotely compiled Lean prototype; collectively they demonstrate the continuation-backtracking core, dependent witness selection, dictionary identity, finite counterexample-guided synthesis, and hand-supplied certified recursion. They do not demonstrate automatic recursion discovery, library indexing, scheduling, or the acceptance gate. Section 21 states exactly what has and has not been shown.

# Contents

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>I</b> | <b>Architecture</b>                                         | <b>6</b>  |
| <b>1</b> | <b>Objective, baseline, and sources</b>                     | <b>6</b>  |
| 1.1      | What Leant2 is . . . . .                                    | 6         |
| 1.2      | How this document was produced . . . . .                    | 6         |
| 1.3      | The baseline: Leant’s :synth corpus . . . . .               | 7         |
| 1.4      | What the predecessors established . . . . .                 | 8         |
| 1.5      | What to take from Forge . . . . .                           | 9         |
| <b>2</b> | <b>Principal decisions</b>                                  | <b>9</b>  |
| <b>3</b> | <b>The synthesis problem</b>                                | <b>11</b> |
| 3.1      | Queries and answers . . . . .                               | 11        |
| 3.2      | Kinds of constraint . . . . .                               | 11        |
| 3.3      | The result algebra . . . . .                                | 12        |
| 3.4      | Trust profiles . . . . .                                    | 12        |
| <b>4</b> | <b>Representations and search state</b>                     | <b>12</b> |
| 4.1      | Three representations, one type theory . . . . .            | 12        |
| 4.2      | Origin record and environment epochs . . . . .              | 13        |
| 4.3      | Branch state and the obligation graph . . . . .             | 13        |
| <b>5</b> | <b>Transactional search</b>                                 | <b>14</b> |
| 5.1      | The atomic attempt . . . . .                                | 14        |
| 5.2      | Rule outcomes and blocked states . . . . .                  | 15        |
| 5.3      | Joint mode and frozen mode . . . . .                        | 15        |
| <b>6</b> | <b>Adaptive scheduling</b>                                  | <b>15</b> |
| 6.1      | Lanes . . . . .                                             | 15        |
| 6.2      | Allocation . . . . .                                        | 16        |
| 6.3      | How the adaptive engine replaces Leant’s settings . . . . . | 16        |
| 6.4      | Ranking and multiple answers . . . . .                      | 17        |
| 6.5      | Resource limits and determinism . . . . .                   | 17        |
| <b>7</b> | <b>Native construction</b>                                  | <b>18</b> |
| 7.1      | Introductions . . . . .                                     | 18        |
| 7.2      | Application spines . . . . .                                | 18        |
| 7.3      | Known-major projections . . . . .                           | 18        |
| 7.4      | Polymorphic and higher-rank instantiation . . . . .         | 19        |
| 7.5      | Instances and dictionary identity . . . . .                 | 19        |

|           |                                                                        |           |
|-----------|------------------------------------------------------------------------|-----------|
| 7.6       | Normalization and equality . . . . .                                   | 19        |
| 7.7       | The constructive propositional lane and classical candidates . . . . . | 19        |
| <b>8</b>  | <b>Dependent elimination</b>                                           | <b>20</b> |
| <b>9</b>  | <b>Library composition and carrier invention</b>                       | <b>20</b> |
| 9.1       | The provider index . . . . .                                           | 20        |
| 9.2       | Component recipes and the join . . . . .                               | 21        |
| 9.3       | Providers with contracts . . . . .                                     | 21        |
| 9.4       | Carrier invention . . . . .                                            | 21        |
| <b>10</b> | <b>Recursion</b>                                                       | <b>22</b> |
| 10.1      | Routes, in order . . . . .                                             | 22        |
| 10.2      | Schemas and capabilities . . . . .                                     | 22        |
| 10.3      | Motive construction . . . . .                                          | 22        |
| 10.4      | Proof-carrying motives . . . . .                                       | 23        |
| 10.5      | Accumulator invariants . . . . .                                       | 23        |
| 10.6      | Termination . . . . .                                                  | 23        |
| 10.7      | The kernel/compiler boundary . . . . .                                 | 24        |
| 10.8      | Status . . . . .                                                       | 24        |
| <b>11</b> | <b>Behavioral constraints</b>                                          | <b>24</b> |
| 11.1      | Contract decomposition . . . . .                                       | 24        |
| 11.2      | Typed observations and residual evaluation . . . . .                   | 25        |
| 11.3      | Proof services . . . . .                                               | 25        |
| 11.4      | Certified counterexample-guided refinement . . . . .                   | 25        |
| 11.5      | Reversible observational buckets . . . . .                             | 26        |
| 11.6      | Certified pruning . . . . .                                            | 26        |
| <b>12</b> | <b>Caching and equivalence</b>                                         | <b>26</b> |
| <b>13</b> | <b>The acceptance gate</b>                                             | <b>27</b> |
| 13.1      | Pipeline . . . . .                                                     | 27        |
| 13.2      | Receipts . . . . .                                                     | 28        |
| 13.3      | Isolation . . . . .                                                    | 28        |
| <b>14</b> | <b>Frontends and deployment</b>                                        | <b>28</b> |
| 14.1      | Entry points . . . . .                                                 | 28        |
| 14.2      | Compatibility REPL for the baseline . . . . .                          | 28        |
| 14.3      | Worker and controller . . . . .                                        | 29        |
| 14.4      | Proposed module layout . . . . .                                       | 29        |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>II Algorithms</b>                                                    | <b>29</b> |
| <b>15 Algorithms: construction search</b>                               | <b>30</b> |
| 15.1 The main loop over the obligation graph . . . . .                  | 30        |
| 15.2 Introductions . . . . .                                            | 33        |
| 15.3 Spine expansion . . . . .                                          | 34        |
| 15.4 Known-major projections . . . . .                                  | 35        |
| 15.5 Type invention and higher-rank instantiation . . . . .             | 36        |
| 15.6 Instance and dictionary handling . . . . .                         | 37        |
| 15.7 Worked traces . . . . .                                            | 38        |
| <b>16 Algorithms: dependent elimination, retrieval, and carriers</b>    | <b>40</b> |
| 16.1 The case-analysis transaction . . . . .                            | 40        |
| 16.2 Equality repair and transport . . . . .                            | 42        |
| 16.3 Elimination restrictions . . . . .                                 | 43        |
| 16.4 Worked elimination traces . . . . .                                | 43        |
| 16.5 Provider index construction and retrieval . . . . .                | 44        |
| 16.6 Component recipes, saturation, and the join . . . . .              | 45        |
| 16.7 Carrier invention . . . . .                                        | 46        |
| 16.8 Helper discovery by typed anti-unification . . . . .               | 48        |
| 16.9 What ran and what did not . . . . .                                | 49        |
| <b>17 Algorithms: recursion synthesis</b>                               | <b>49</b> |
| 17.1 Structural schema generation from recursor metadata . . . . .      | 49        |
| 17.2 Proof-carrying motives and certificate algebras . . . . .          | 52        |
| 17.3 Recursive-call capabilities . . . . .                              | 53        |
| 17.4 Accumulator introduction and invariant discovery . . . . .         | 54        |
| 17.5 Well-founded recursion . . . . .                                   | 56        |
| 17.6 Executable lowering . . . . .                                      | 57        |
| 17.7 Worked traces . . . . .                                            | 58        |
| 17.8 What was demonstrated and what was not . . . . .                   | 59        |
| <b>18 Algorithms: contracts, counterexamples, and certified pruning</b> | <b>60</b> |
| 18.1 Contract decomposition and backward propagation . . . . .          | 60        |
| 18.2 Typed observations and residual evaluation . . . . .               | 61        |
| 18.3 The certified CEGIS loop . . . . .                                 | 63        |
| 18.4 Reversible observational buckets . . . . .                         | 65        |
| 18.5 Certified pruning . . . . .                                        | 66        |
| 18.6 Solver bridges . . . . .                                           | 69        |
| 18.7 Worked traces . . . . .                                            | 69        |
| <b>19 Algorithms: scheduling, caching, and completeness</b>             | <b>71</b> |

|            |                                                                         |            |
|------------|-------------------------------------------------------------------------|------------|
| 19.1       | Tasks, cursors, and the step interface . . . . .                        | 71         |
| 19.2       | The outer loop . . . . .                                                | 72         |
| 19.3       | Obligation selection, assignment versions, and wake-ups . . . . .       | 73         |
| 19.4       | Two layers: heuristic best-first and the fair reference layer . . . . . | 74         |
| 19.5       | The cost-indexed reference enumerator . . . . .                         | 75         |
| 19.6       | Cost vectors, profiles, and the Pareto frontier . . . . .               | 76         |
| 19.7       | Verification scheduling . . . . .                                       | 77         |
| 19.8       | Memoization and equivalence . . . . .                                   | 77         |
| 19.9       | Resource accounting . . . . .                                           | 79         |
| 19.10      | What the search guarantees . . . . .                                    | 80         |
| <b>20</b>  | <b>Algorithms: proof services and the acceptance gate</b>               | <b>81</b>  |
| 20.1       | The proof-service adapter . . . . .                                     | 82         |
| 20.2       | The constructive propositional lane . . . . .                           | 85         |
| 20.3       | The freeze step and the transaction adapter . . . . .                   | 87         |
| 20.4       | The acceptance gate as one procedure . . . . .                          | 88         |
| 20.5       | Negative results . . . . .                                              | 89         |
| 20.6       | Adversarial acceptance checklist . . . . .                              | 90         |
| 20.7       | Worked traces . . . . .                                                 | 91         |
| <b>III</b> | <b>Evidence, evaluation, and roadmap</b>                                | <b>92</b>  |
| <b>21</b>  | <b>Evidence available today</b>                                         | <b>92</b>  |
| <b>22</b>  | <b>Evaluation plan</b>                                                  | <b>93</b>  |
| 22.1       | Tiers . . . . .                                                         | 93         |
| 22.2       | Leakage and controls . . . . .                                          | 93         |
| 22.3       | Metrics and ablations . . . . .                                         | 94         |
| <b>23</b>  | <b>Roadmap</b>                                                          | <b>94</b>  |
| <b>24</b>  | <b>Risks</b>                                                            | <b>95</b>  |
| <b>A</b>   | <b>Provenance ledger</b>                                                | <b>95</b>  |
| <b>B</b>   | <b>The baseline corpus</b>                                              | <b>97</b>  |
| <b>C</b>   | <b>Interface sketches</b>                                               | <b>99</b>  |
|            | <b>References</b>                                                       | <b>101</b> |

## Part I

# Architecture

## 1 Objective, baseline, and sources

### 1.1 What Leant2 is

Leant and Djex generate implementations for a subset of Lean type signatures, optionally under behavioral constraints. Both are implemented in Haskell, both search over a Haskell-shaped approximation of Lean types, and Djex must remain compatible with its Haskell frontend. Leant2 is the successor that removes all three limitations: it is implemented in Lean, it has only a Lean frontend, and it is designed for Lean types together with behavioral constraints. It cannot decide inhabitation of arbitrary Lean types, and no proposal claims that it can. The target is a large fraction of the types that occur in practice, with every positive answer checked by Lean’s kernel. Three constraints on this document come from the project owner and override anything in the nine source proposals.

**Decision 1.1** (Single adaptive engine). Leant2 has one engine. There are no engine switches of the kind Leant exposes (`synth-engine djinn | exference | both`), and no user-facing settings are implemented at first. Budgets, lane weights, verification windows, and candidate counts are internal parameters of an adaptive scheduler (Section 6), derived from the query and from search progress, not from the user. Configuration may be added later behind the same engine, but no acceptance criterion in this document depends on it.

**Decision 1.2** (Semantic baseline). Leant2 must pass every `:synth` test in the Leant repository, read semantically: for each recorded query it must match or exceed the recorded outcome. It may return more candidates, better candidates, and candidates in a different order. Section 1.3 makes this precise.

**Decision 1.3** (Reuse of Forge). Forge is a sibling project with kernel-checked certificate checkers, an orchestration frontend with full rollback and proof-term auditing, and a written review of the Leant/Djex verification boundary. Leant2 may freely borrow Lean code and ideas from it, and this document does so explicitly (Section 1.5).

### 1.2 How this document was produced

The nine proposals are numbered P1 to P9 in the order of their directories under `docs/proposals/`. Each is a 32-to-45-page article with a small Lean 4.34.0 prototype checked remotely through the AXLE service, transcribed receipts, and helper scripts. All nine inspected the same Leant revision `6bf05ad` and one of two Djex revisions (`e8778f4`, the standalone head, or `e237e86`, the submodule pinned by Leant); three of them note that the two must not be conflated.

The consolidation rule is simple. An idea present in most proposals is adopted as core and stated once in its clearest form. Where proposals differ in the treatment of one subsystem, the best-developed version is adopted and the alternatives are recorded in Appendix A. Ideas are marked with their origin as <sup>[P<sub>n</sub>]</sup> on first use so that the more detailed discussion in the source can be consulted; a marker lists only the proposals that develop the idea most fully, not every proposal that mentions it. Nothing in this document is new evidence. Where the proposals’ prototypes have shown something, Section 21 says which one and how.

Names are unified as follows. What the proposals call the *obligation graph*, *goal forest*, *hole graph*, or *coupled continuation* is called the *obligation graph* here. What they call *recipes*, *search plans*,

*construction plans*, or the *control language* is called the *search plan*. What they call *acceptance*, *admission*, *publication*, *finalization*, or the *candidate gate* is called the *acceptance gate*. Their result taxonomies, which range from five to nine constructors, are merged into the single result algebra of Section 3.3.

### 1.3 The baseline: Leant’s :synth corpus

The Leant repository carries thirty golden-transcript fixtures `test/synth-*.txt`, each piped into `leant -plain` and diffed against `test/synth-*.golden` by `test/run-tests.sh`. Together they contain 278 :synth queries. Of these, 266 produce at least one displayed candidate, five produce the verdict “provably uninhabited”, and seven are deliberate negative controls: three “no term found within the search bounds” under restricted provider settings, two “none passed both Lean verification and the supplied assertion”, one assertion that fails type checking, and one assertion mentioning an unknown name. Twelve queries carry a behavioral assertion in the form `:synth f : T where P`. Appendix B tabulates the fixtures.

The transcripts also contain session declarations (`inductive`, `structure`, `class`, `instance`, `axiom`, `opaque`, `def`, `abbrev`, `universe`), `:reset`, and 170 `:set` lines that switch engines and adjust steps, windows, verification counts, classical mode, library use, and provider discovery. The recorded outputs include exact candidate spellings, ranking order, truncation notes, and engine-specific telemetry.

The goldens cannot be matched textually by a different engine, and Decision 1.1 removes the settings they exercise. The baseline is therefore defined on outcomes, not on text.

**Definition 1.4** (Baseline corpus). The baseline corpus is the set of :synth queries in `test/synth-*.txt` at Leant revision 6bf05ad, each taken together with the session declarations that precede it in its transcript, and each labeled with the outcome recorded in the corresponding golden file. Every `:set` line is ignored. The recorded outcome of a query is one of: *positive* (at least one displayed candidate), *uninhabited* (“provably uninhabited”), *bounded miss* (“no term found within the search bounds”), *rejected* (candidates proposed, none passed), or *preflight error* (assertion rejected before search).

**Definition 1.5** (Baseline acceptance). Leant2 passes the baseline when, for every query in the baseline corpus, run under Leant2’s default policy with no per-query configuration and within a fixed per-query wall-clock budget:

1. **Positive.** Leant2 returns at least one *accepted* candidate: kernel-checked at the exact requested type in the exact session context and, for a `where` query, with a kernel-checked proof of the assertion for that exact candidate. The candidate need not be the recorded one, and the order of candidates is unconstrained.
2. **Uninhabited.** Leant2 returns a *refutation* (a kernel-checked proof of  $T \rightarrow \text{False}$  in the session context) or a *certified fragment non-derivability* (Section 3.3). For four of the five recorded cases a refutation exists and is preferred; the fifth, `forall p : Prop, p or not p`, is classically provable and must not be reported as refuted.
3. **Bounded miss, rejected.** Leant2 returns either an accepted candidate (this exceeds the recorded outcome and is allowed) or a qualified negative. It must not return an unaccepted candidate as a success.
4. **Preflight error.** Leant2 rejects the query before search with a diagnostic of the same category (ill-typed assertion; unknown name).

The set of accepted candidates is a superset in strength of Leant’s: any candidate Leant2 displays has passed the acceptance gate of Section 13, which is stricter than Leant’s re-elaboration.

Three consequences follow. First, the five `synth-church-rankn`-style fixtures with 69 rank-N and Church-encoding queries are part of the baseline; they exercise nested polymorphic

callbacks, universe-polymorphic families, strict-implicit binders, and seeded Church compositions, and are the hardest part of the corpus for a native engine that does not inherit Djex’s rank-N machinery. Second, the `synth-library` fixture requires library composition (`List.map`, `List.foldr`, `List.flatten`, `List.replicate`) from the standard environment, so the provider index of Section 9 is on the baseline path, not a later milestone. Third, the classical candidates in `synth-manual` (Peirce’s law, double-negation elimination, and the like, produced with `synth-classical on`) count as positive outcomes; Leant2 must be able to offer `Classical.em`-based candidates, labeled as classical, when constructive search returns a certified fragment non-derivability (Section 7.7).

The baseline is a floor. Leant’s further suites also drive `:synth`: the Church behavior ledger (160 cells, 94 with historical acceptance), `test-recursive` (eight recursive-data behavioral scenarios with independent Lean replay), `test-behavioral` (bounded simplification and combined decision controls), and `test-context` (global class-projection discovery). These form the *extended tier* of Section 22: Leant2 should reach them after the baseline, and their historically accepted cells are regression targets, not gates for the first release.

## 1.4 What the predecessors established

The proposals read the same sources and reach the same conclusions about them.

**Leant’s representation boundary.** Leant elaborates queries in Lean and rechecks every candidate in Lean, but its search runs in Djex over a serialized fragment (`src/Leant/Synth/Fragment.hs`). The serializer retains structural connectives and selected polymorphic applications but treats term-indexed applications as opaque and describes recursion by bounded templates. This is the boundary Leant2 removes<sup>[all]</sup>.

**Djex’s two traditions.** Djinn is a checked propositional prover based on Dyckhoff’s contraction-free calculus LJ<sub>T</sub>; it terminates and can report that no derivation exists in its fragment. Exference is a ranked best-first search with class-evidence resolution and resource controls. Djex’s shared infrastructure enforces source ownership, scoped evidence, and independent checking. Leant2 keeps those disciplines without the Haskell compatibility surface<sup>[P6,P8]</sup>.

**Behavioral assertions.** At the inspected revision, Leant checks a named assertion against each exact candidate by `decide`, then its negation, then bounded `simp` at both polarities; it distinguishes passed, falsified, and inconclusive, and never treats proof failure as a counterexample. Search produces type inhabitants first and filters afterwards. Every proposal moves the assertion into the search state so that it constrains witnesses before a candidate exists, while retaining the three-way distinction<sup>[all]</sup>.

**The indexing gap.** Leant’s current diagnostics locate the remaining integer-indexing failure in bounded composition: a function-valued fold and the integer case step are each found in isolation and missed in composition within the original limits. This motivates the demand-directed component reuse and carrier invention of Section 9. No proposal claims to have solved that fixture<sup>[P1,P6,P7,P9]</sup>.

**The earlier rewrite study.** The Leant repository’s own rewrite analysis recommends Lean-hosted semantics, project-matched worker processes, exact source identities, and a kernel validation boundary; those are adopted. It also recommends a pure, neutral search representation with native metavariables confined to preparation and lowering. All nine proposals depart from that point: a neutral representation restricted enough to be easy to search recreates the loss of dependency information the successor is meant to remove. Leant2 searches over native, branch-local dependent constraints, and keeps a serializable plan only for replay, caching, and cross-process transfer (Section 4)<sup>[P1,P6,P9]</sup>.

## 1.5 What to take from Forge

Forge [4] is a certificate-producing proof-planning layer above `grind`. It pins the same toolchain as the nine proposals, Lean 4.34.0 with Mathlib 1cf325a0, and has reached a state none of the Leant2 prototypes has: compiled Lean with proved soundness theorems and a measured comparison. Four parts transfer directly.

1. **The orchestration frontend** (`lean/Forge/Frontend.lean`). The `forge` tactic runs a fixed list of workers, saves the full tactic state before each, restores it on any failure including resource limits, and on apparent success does not take the worker’s word: it instantiates the proof term and rejects it if it contains `sorry`, mentions a free variable outside the goal’s context, fails to type-check against the *original* goal, or depends on an axiom outside the policy, computed with `collectAxioms` over the constants the term uses. It then emits a replay record. This is exactly the proof-service contract of Section 11.3 and the audit half of the acceptance gate of Section 13; Leant2 should start from this code rather than rewrite it.
2. **Certificate checkers with proved soundness** (`lean/Forge/Checker/`). Cone, Farkas, affine-witness, polynomial-recurrence, and conserved-invariant checkers, each with a soundness theorem and an axiom audit that fails the build if a theorem uses an axiom outside `propext` and `Quot.sound`. The design contract `CertificateSpec` (a decoding `denotes`, a decidable `check`, and a proof `sound`) is the shape every proposal wants for its “certified domain plugin” (Section 11.6). Leant2’s behavioral lane should register these checkers as proof services for arithmetic obligations on day one.
3. **Two soundness pitfalls** documented in `docs/IDEAS-FROM-LEANT-DJEX.md` and verified against Lean 4.34.0. `Kernel.Environment.addDecl` skips the kernel when `debug.skipKernelTC` is set in the options the caller passes, so an acceptance step must pin that option off rather than merely not set it. `Lean.addDecl` in `CoreM` commits preliminary constant information immediately and, under `Elab.async`, defers the kernel check to a task, so observing that a declaration is present observes nothing. Acceptance must wait for a positive signal that the checker ran. Section 13 adopts both as invariants.
4. **A checkable fragment refutation**. Forge’s extension round supplies finite Kripke counter-models for intuitionistic propositional logic whose checker recomputes forcing from scratch. This is the object every proposal asks for when it says that LJT non-derivability must not be promoted to a Lean refutation: a certificate that *there is no derivation in the named fragment*, distinct from a proof of  $\neg T$ . Leant2’s constructive propositional lane should produce this certificate (Section 7.7).

Forge’s status lattice, with its rule that pruning must state its approximation polarity, coincides with the graded-authority principle of Section 11. Its Python search components are research code and are not reused; only the Lean checkers and the frontend are.

## 2 Principal decisions

The following decisions are held by all nine proposals. Each is stated once here; the rest of Part I gives the mechanisms and Part II the algorithms.

**Decision 2.1** (Exact Lean expressions are the only semantic authority). Targets, contexts, candidates, universe levels, instance dictionaries, and behavioral propositions are Lean `Expr`, `Level`, and local declarations in an identified environment. There is no second type language in which term indices, proof dependencies, or universes become approximations. Indexes, abstract domains, and solver encodings exist, but each is a lossy accelerator with an explicit reconstruction boundary: it proposes, and native unification and the kernel decide<sup>[all]</sup>.

**Decision 2.2** (The unit of backtracking is a partial program with all of its obligations). Solving a hole never commits its value before the remaining obligations that depend on it succeed.

An application creates jointly required children; a later proof obligation, index equation, or instance constraint can reopen an earlier data choice. Every prototype implements this as `search (children ++ rest)` inside a saved state, and the canonical failure it prevents is choosing 0 for  $\{n : \mathbb{N} // n = 1\}$  and then being unable to prove  $0 = 1$ <sup>[all]</sup>.

**Decision 2.3** (Typing, behavior, and executability are separate results). A term may inhabit the type, satisfy the contract, and compile to acceptable executable code; these are three outcomes, not three spellings of one. A kernel-accepted recursor term can be rejected by the code generator, as several prototypes observed. Executability is a field of the result, not a consequence of typing<sup>[P4,P6,P8,P9]</sup>.

**Decision 2.4** (Recursion enters only through typed schemas). The definition under construction is never an ordinary provider. Recursive calls are available only as *capabilities* that carry the decrease evidence for a specific smaller argument, and a recursion schema records its motive, branches, and termination obligations. One proposal’s first run failed precisely because unfiltered locals exposed a provisional self-reference<sup>[P2]</sup>.

**Decision 2.5** (Behavioral information has graded authority). A finite observation refutes the exact candidate it fails and nothing else. A proof certifies. A heuristic analysis may rank. Only a proof-producing over-approximation may prune a partial program, and only after exhibiting a valid input. Solver verdicts, evaluator timeouts, and failed proof attempts are not evidence<sup>[all]</sup>.

**Decision 2.6** (One acceptance gate). Every positive result crosses one gate that reconstructs the frozen query, closes the program and the proof over the exact telescope, kernel-checks both with kernel bypasses pinned off, audits transitive axioms and runtime dependencies, optionally compiles, and re-elaborates the exported source. Search components cannot construct an accepted result; the constructor is private to the gate<sup>[all]</sup>.

**Decision 2.7** (Negative answers are qualified). “No solution” has at least four distinct meanings: budget exhausted; a named finite grammar exhausted; no derivation in a named fragment, with a checkable certificate; and a Lean proof that no term of the type satisfies the contract. Only the last is an impossibility claim. A negative from a fragment or a budget never enters a cache as impossibility<sup>[all]</sup>.

**Decision 2.8** (Resources are never refunded by rollback). Restoring a metavariable context restores logical state only. Work charged for rule applications, unification, proof attempts, solver calls, and evaluation lives in a monotone ledger outside the snapshot. Cancellation, heartbeat exhaustion, and memory pressure are interruptions that unwind to the controller; they are never caught as “rule not applicable”<sup>[P1,P2,P5,P7,P9]</sup>.

Together with Decisions 1.1 to 1.3, these fix the shape of the system. Figure 1 shows the pipeline.

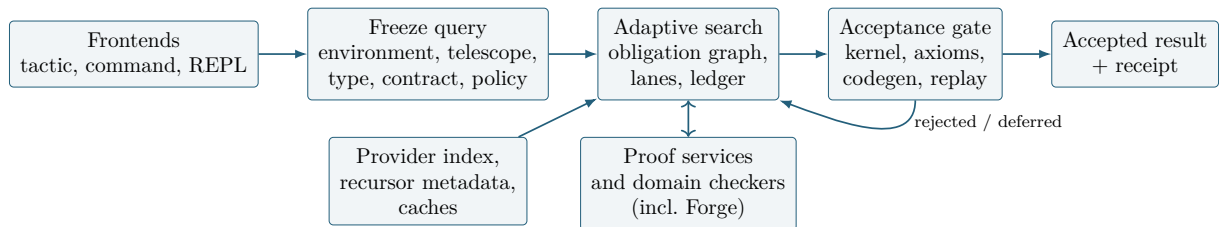


Figure 1: The pipeline. Everything left of the gate is untrusted search; the gate adds nothing to what the kernel checks except refusal.

### 3 The synthesis problem

#### 3.1 Queries and answers

**Definition 3.1** (Frozen query). A frozen query is a tuple

$$Q = (\mathcal{E}, U, \Gamma, T, \Phi, \mathcal{X}, \mathcal{G}, \Pi, \mathcal{B})$$

where  $\mathcal{E}$  is the environment with its import identities and revision;  $U$  is the universe-parameter context;  $\Gamma$  is the ordered dependent local context, including `let` values and local instances;  $T$  is the target with  $\mathcal{E}; U; \Gamma \vdash T : \text{Sort } u$ ;  $\Phi$  is the optional contract with  $\mathcal{E}; U; \Gamma, f:T \vdash \Phi(f) : \text{Prop}$ , taken as `True` when absent;  $\mathcal{X}$  is a finite set of typed observations;  $\mathcal{G}$  is the admitted construction grammar and provider inventory, stated in terms of constants and local terms rather than names;  $\Pi$  is the trust and execution policy; and  $\mathcal{B}$  is the resource policy<sup>[all]</sup>.

The query is elaborated once in the real project environment and frozen before search. Unknown identifiers are errors, not auto-bound implicits; universe parameters remain parameters and are never specialized; instance selection in the statement is part of the frozen question; and no metavariable owned by the caller may be assigned by search except through an explicit permission set returned as an atomic patch<sup>[P5,P6,P8]</sup>.

**Definition 3.2** (Answer). A verified answer to  $Q$  is a pair of judgments

$$\mathcal{E}, \Delta; U; \Gamma \vdash t : T \quad \text{and} \quad \mathcal{E}, \Delta; U; \Gamma \vdash p : \Phi(t),$$

where  $\Delta$  is a finite list of generated auxiliary declarations accepted in dependency order under  $\Pi$ . The two judgments are kept separate rather than packed into a subtype, because  $T$  may live in any sort and because a proof that specializes the candidate would describe a different program<sup>[P2,P3,P5,P8]</sup>.

**Proposition 3.3** (Pointwise packaging). *If  $\Phi(f) \equiv \forall x:A, R(x, f x)$ , then a term  $g : \prod_{x:A} \{y : B(x) // R(x, y)\}$  yields  $f := \lambda x. (g x).val$  with  $\forall x, R(x, f x)$ , and conversely, constructively. The engine registers this as a checked transformation and uses the packed form to carry the contract into recursive branches (Section 10); it never pattern-matches on the syntax of  $\Phi$  to decide correctness<sup>[P1,P3,P9]</sup>.*

#### 3.2 Kinds of constraint

Four kinds of behavioral input are supplied independently and are never silently converted into one another<sup>[P1,P7]</sup>:

- a *hard contract*  $\Phi$ , which must be proved;
- *observations*  $\mathcal{X}$ : typed inputs with precondition evidence and expected outputs, or a proposition about particular inputs. They refute candidates and guide search; passing them establishes only themselves;
- *construction restrictions*: grammar predicates such as “exclude `List.reverse`”, enforced on the transitive dependencies of the candidate, not on names;
- *preferences*: ranking signals such as “prefer using every input”, which can never reject a correct program, since some correct programs must ignore inputs.

Leant’s `:synth f : T where P` supplies a hard contract  $P$  elaborated under the binder  $f:T$ ; the baseline corpus contains no observations, restrictions, or preferences beyond this.

### 3.3 The result algebra

The nine proposals propose between five and nine result constructors. They are merged as follows; the internal representation carries orthogonal evidence fields, and the frontend may group constructors for display but the structured API preserves them<sup>[P4,P6,P8,P9]</sup>.

```
inductive Outcome where
| verified      (t : Accepted) (p : Accepted) -- exact type and exact contract
| typedOnly     (t : Accepted) (pending : Obligations)
| testedOnly    (t : Accepted) (passed : Observations) -- never satisfies a universal
  contract
| refutedCandidate (t : Candidate) (q : Accepted) -- proof of not (Phi t), or replayed
  counterexample
| impossible     (q : Accepted) -- proof of (t : T) -> Phi t -> False, or T -> False
| fragmentUnderivable (fragment : Name) (cert : Checked) -- e.g. Kripke countermodel for IPC
| grammarExhausted (grammar : Id) (ledger : Incompleteness)
| budgetExhausted (resumable : StateId) (ledger : Incompleteness)
| unsupported    (feature : String)
| infrastructure (error : String)
```

Each accepted object additionally records three independent flags, `kernelChecked`, `compiled`, and `evaluatedOnExamples`, together with the axiom inventory of the proof and the runtime-dependency inventory of the program (Decision 2.3). “Best” candidate means best among verified candidates under a declared cost model; “minimal” requires an exhausted cheaper prefix and is otherwise reported as “lowest cost found under this schedule”<sup>[P6,P8]</sup>.

The baseline mapping is: Leant’s displayed candidates correspond to `verified` (type-only queries have  $\Phi = \text{True}$ , so the contract proof is trivial); “provably uninhabited” corresponds to `impossible` or `fragmentUnderivable`; “no term found” corresponds to `budgetExhausted` or `grammarExhausted`; “none passed” corresponds to `refutedCandidate` results with no `verified` result.

### 3.4 Trust profiles

Three policy profiles are recognized, each fixing an axiom allowlist for proofs and a separate runtime policy for programs<sup>[P1,P3,P6,P9]</sup>:

- *strict constructive*: empty axiom inventory beyond explicitly accepted query assumptions; total, computable program; no `Classical.choice` on the computational path;
- *standard*: `propext`, `Quot.sound`, and `Classical.choice` permitted in proofs and erased transports, with actual use always reported; program still total and computable;
- *project-relative*: named project axioms permitted; results certified relative to them.

`sorryAx` is rejected in every profile. A result is never called “axiom-free” when it depends on `propext`; several prototypes’ indexed `cases` did, and the receipts say so. The default profile for the baseline is *standard*, which admits the classical candidates of `synth-manual`, labeled as such.

## 4 Representations and search state

### 4.1 Three representations, one type theory

Every proposal converges on the same division of labor, in three tiers<sup>[P3,P5,P8,P9]</sup>.

1. **Semantic objects.** Native `Expr`, `Level`, names, binder information, local contexts, local instances, and the environment. These are authoritative and are never replaced by their pretty-printed form.
2. **The search plan.** A persistent, serializable record of search actions: introduce this binder; apply this exact provider occurrence with these universe instantiations and this spine prefix; split this scrutinee with this motive; instantiate this recursion schema; ask this proof service with this budget. Plan holes carry a stable identifier, a scope identifier, a native target closed over a dependency-closed telescope, and a role. The plan can be replayed against the frozen query to rebuild native state with fresh metavariable identifiers. It exists for resumption, caching, explanation, parallel transfer, and worker restart; it does not define a competing typing relation.
3. **Accelerators.** Discrimination-tree keys, type-head graphs, abstract domains such as list lengths and affine summaries, finite grammars, and solver encodings. Each is lossy and each declares its reconstruction boundary and its approximation polarity.

**Invariant 4.1** (No naked open expressions cross a process boundary). Raw `MVarId` and `FVarId` are never durable. A task shipped to another process or session is a closed plan over an ordered telescope with scope-stable binder positions, exact level and constant names, an environment fingerprint, and a protocol version. The receiver reopens it with fresh identifiers and rechecks; a mismatched fingerprint rejects the package rather than reinterpreting it<sup>[P4,P8,P9]</sup>.

Within one worker, native snapshots (`Meta.saveState/restore`) are the fast path. A production engine mixes two profiles: hot branches keep a materialized native checkpoint; cold branches are stored as a checkpoint plus the plan suffix needed to rematerialize them, bounding memory<sup>[P5]</sup>.

## 4.2 Origin record and environment epochs

The frozen query is stored as an immutable *origin record*: toolchain identity, imported module identities, environment revision, the closed telescope,  $T$ ,  $\Phi$ , the component policy, the axiom and execution policy, and the semantic options in force (transparency, active local instances, coercion configuration, grammar restrictions). Fingerprints locate cache entries; they never replace structural revalidation<sup>[P9]</sup>.

A worker owns an *environment epoch*. Every handle, snapshot, cache entry, provider index, and proof-service reply is stamped with the epoch in which it was made. An undo, a `:reset`, an import change, or a worker restart bumps the epoch and invalidates everything stamped with the old one. Leant’s session model, in which `:reset` and declaration replay change the environment mid-transcript, maps directly onto epochs<sup>[P1,P6,P7]</sup>.

## 4.3 Branch state and the obligation graph

**Definition 4.2** (Branch state). A branch is  $S = (O, M, r, H, D, K, R, \mathcal{X}_S, \rho, c)$ : the origin record; the native metavariable and elaboration state; the root program and root contract-proof expressions; the set of open obligations; the dependency graph over them; the delayed-constraint store; the set of recursive-call capabilities in scope; the branch-local observation bank version; the plan suffix that regenerates the branch; and the branch cost vector<sup>[P2,P3,P7,P9]</sup>.

Each obligation  $h \in H$  records its goal metavariable, its local context  $\Gamma_h$ , its target  $T_h$ , its residual contract  $\Phi_h$  if any, and a *role*: program value, type or universe argument, instance evidence, index equality, contract proof, termination proof, invariant, or reconstruction check. Roles guide scheduling; they never affect typing.

The dependency graph has an edge  $h_1 \rightarrow h_2$  whenever assigning a metavariable reachable from  $h_1$  can change the type, context, admissibility, or residual contract of  $h_2$ . Reachability

is computed over local declaration types and values, delayed constraints, instance arguments, and the contract, not only over syntactic occurrence; read and write footprints are conservative over-approximations. Obligations that can assign metavariables reachable from one another form *coupled clusters*, the strongly connected components of  $D$ , and are scheduled as units. A constructor application whose fields depend on one another is one hyperedge, not a sequence of independent children<sup>[P3,P6,P7,P8]</sup>.

**Invariant 4.3** (Branch ownership). Every unassigned metavariable belongs either to the frozen ambient problem or to exactly one branch lineage. A branch may assign only its own search metavariables; it may not change the frozen statement, specialize the query’s universe parameters, publish declarations, or reuse another branch’s assignment without a checked context embedding. When Leant2 runs inside an unfinished proof, the permitted assignment set for ambient metavariables is computed at freeze time and any delta is returned as one atomic patch<sup>[P6,P7,P8]</sup>.

**Invariant 4.4** (Scope closure, type coherence, dependency coherence, evidence ownership). Every obligation’s target is closed under its own context; an unresolved proof obligation remains an obligation rather than being weakened; assigning a metavariable invalidates any observation, index entry, or summary whose recorded support depends on it; and a proof, counterexample, or summary is meaningful only for the exact expression, context, contract, and policy it was produced for<sup>[P8]</sup>.

## 5 Transactional search

### 5.1 The atomic attempt

The fundamental operation is the atomic attempt: snapshot all state the rule may write, run the rule, recurse on the rule’s new obligations *prepended to the remaining obligations of the branch*, and commit only if that whole continuation succeeds; otherwise restore, retaining only tagged diagnostics and the ledger charges<sup>[all]</sup>. The prototype form, common to all nine packages up to naming, is:

```
partial def solve (fuel : Nat) : List MVarId -> SearchM Bool
| []      => pure true
| g :: gs => do
  if fuel = 0 then return false
  if <- g.isAssigned then return <- solve fuel gs
  for rule in rulesFor g do
    let saved <- saveState
    try
      let children <- rule.apply g          -- may assign metavariables shared with gs
      if <- solve (fuel - 1) (children ++ gs) then return true
    catch e => if isInterrupt e then throw e -- never swallow cancellation
  restore saved
  return false
```

The production engine replaces recursion by explicit *cursor frames* (focused obligation, siblings, saved state, rule family, next alternative, cost allowance) so that a branch can be suspended, resumed, serialized, and scheduled fairly, but the semantics of the attempt are those above<sup>[P6,P8]</sup>.

**Invariant 5.1** (Commit criterion). A subgoal’s solution may be memoized as an alternative, but its assignments become irrevocable only when every dependent obligation in the enclosing coupled cluster has been discharged or the generalization has been independently justified. An AND-node is a joint obligation with correlated substitutions, not a Cartesian product of independent first answers<sup>[P2]</sup>.

What a snapshot must cover depends on the rule. A pure **MetaM** rule needs the metavariable context and caches. A rule that elaborates syntax needs the elaborator state, postponed synthetic metavariables, local instances, and diagnostics. A rule that creates declarations needs an environment transaction, and such rules are staged: they run against a scratch environment and are committed only by the acceptance gate<sup>[P2,P5,P9]</sup>.

## 5.2 Rule outcomes and blocked states

A rule returns one of **progress** (new obligations, changed holes), **solved**, **blocked**, **rejected** (optionally with a certificate), **unsupported**, or **interrupted**. Three kinds of “blocked” are distinguished and none is a contradiction: blocked on assignments (a delayed equation or instance goal waiting for named metavariables), unsupported by this lane (with the continuation preserved for another lane), and yielded for budget (with the charged work and the continuation)<sup>[P2,P6]</sup>. Delayed constraints carry dependency stamps and are woken only when a relevant metavariable is assigned; a flex-rigid equation that becomes a pattern, an instance goal whose output parameters become ground, and a contract that reduces after an assignment are all wake-up events<sup>[P4,P6,P9]</sup>.

Unification is a service, not a completeness oracle. Its results are classified as deterministic progress, blocked, speculative, or mismatch; a rigid-head mismatch rejects an alternative cheaply; a budget failure or a stuck higher-order equation is unknown and never enters a negative cache<sup>[P3,P7]</sup>.

## 5.3 Joint mode and frozen mode

Proof services are invoked in one of two modes, and the modes are distinct APIs<sup>[P1,P2,P7]</sup>.

- In *joint mode* a proof attempt may assign program holes within the current transaction. Solving  $?m = n + 1$  by unification determines the witness. Any such assignment is returned as part of the branch’s construction choice, is subject to the commit criterion, and must respect the executable policy.
- In *frozen mode* the candidate is closed and the service may not replace or specialize it. A proof of  $\neg\Phi(t)$  rejects exactly  $t$  in exactly this context. A verifier that could mutate its candidate would produce receipts about a different program.

A reopened data choice creates a new candidate identity; evidence transfers between identities only through an explicit checked equality with contract transport<sup>[P4]</sup>.

# 6 Adaptive scheduling

Decision 1.1 removes Leant’s engine switches and settings. What replaces them is a single scheduler that runs a *portfolio of lanes* over one obligation graph and adapts its allocation to the query and to progress. This section merges the scheduling designs of the nine proposals into that one engine.

## 6.1 Lanes

A lane is a policy over the same search actions; lane identity never changes what a candidate means. The initial lanes are<sup>[P1,P2,P7,P9]</sup>:

- *exact reuse*: locals, projections of known locals, and direct heads that already have the target type;
- *focused construction*: introductions, constructors, and application spines over locals and the goal’s own constructors;

- *library composition*: indexed retrieval and demand-directed composition of environment providers (Section 9);
- *dependent elimination*: case splits and transports (Section 8);
- *recursion*: schema instantiation, motive generalization, carriers (Section 10);
- *constructive propositional*: a terminating focused calculus over a reified propositional fragment with a certificate for non-derivability (Section 7.7);
- *classical*: `Classical.em` and `Classical.byContradiction` as explicit providers, enabled by policy after the constructive lane has returned a certificate;
- *proof*: the portfolio of Section 11.3;
- *behavioral*: counterexample-guided refinement and domain checkers (Section 11.4);
- *reference*: a conservatively enumerated fallback grammar with increasing cost bounds, which prunes only with a checked certificate.

Every lane exposes a bounded step, `step : Origin -> Task -> Quantum -> WorkerM StepResult`, returning a continuation, proposals, and a work delta. Every generator is a finite-step cursor, not a lazy stream<sup>[P3,P9]</sup>.

## 6.2 Allocation

Within a branch, the next obligation is chosen by *pressure*<sup>[P5,P6]</sup>:

$$\text{pressure}(h) = a \cdot \text{dependentUses}(h) + b \cdot \text{knownIndexTests}(h) \\ + c \cdot \text{behaviorConstraints}(h) - d \cdot \text{estimatedBranching}(h),$$

with aging to prevent starvation. A cheap proof field that determines a data field is preferred over the data field (proof-first); when no such field exists, witness-first search with continuation backtracking is retained. Coupled clusters are scheduled as units.

Across branches the scheduler runs weighted round-robin over lane families with best-first order within a lane. The priority of a job  $j$  is

$$s(j) = g(j) + \lambda_h \hat{h}(j) + \lambda_p \hat{p}(j) + \lambda_d \text{duplication}(j) - \lambda_r \text{relevance}(j),$$

where  $g$  is charged cost,  $\hat{h}$  an estimate of remaining construction,  $\hat{p}$  an estimate of proof debt, and the last two terms penalize syntactic near-duplicates and reward obligations that mention rigid heads and concrete observations. This is a heuristic order, explicitly not an admissible  $A^*$  heuristic<sup>[P1,P5]</sup>.

Fairness is guaranteed structurally, not by weights. The reference lane holds a reserved nonzero share of quanta (one in eight by default) and enumerates admitted plans by a *diagonal schedule* over the tuple (lane, expression-cost bound, recursion-scheme bound, verification allowance), increasing the sum with deterministic ties. Within a tier the heuristic queue is complemented by a reserved FIFO stream so that the fairness claim is honest. Queue caps in fast lanes are permitted, and every dropped state is recorded as an incompleteness event in the ledger rather than treated as a refutation<sup>[P2,P5,P7,P9]</sup>.

Verification is scheduled alongside generation. A candidate whose contract proof returns unknown keeps a verification continuation with an increasing proof budget in an *unresolved queue*; a bare rejection never shrinks the raw candidate window; a successful-result quota is not a raw-candidate quota<sup>[P7,P8]</sup>.

## 6.3 How the adaptive engine replaces Leant’s settings

Leant exposes `synth-steps`, `synth-window`, `synth-verify`, `synth-shown`, `synth-budget`, `synth-timeout`, `synth-classical`, `synth-library`, `synth-providers`, and `synth-engine`. Leant2 has none of them at first. The corresponding behaviors are internal:

- *engine*: the lane portfolio is always active; lane weights adapt to the query shape (propositional goals raise the constructive lane; goals whose head is a library inductive raise library composition; contracts raise the behavioral lane) and to progress (a lane that has yielded nothing in its last  $k$  quanta has its weight halved, subject to the reserved share);
- *steps and budget*: the work ledger has a global wall-clock deadline derived from the query’s size and a logical work budget that grows by the diagonal schedule; there is no per-query knob;
- *window, verify, shown*: candidates are streamed to the frontend as they are accepted; the number displayed is a frontend concern; verification is scheduled per candidate as above and never capped by a count;
- *classical*: the classical lane is enabled by the *standard* profile after the constructive lane has produced a certificate, so classical candidates appear later and labeled, which is the behavior Leant’s default configuration approximates;
- *library and providers*: retrieval tiers widen automatically (Section 9); provider discovery is always on, and a result records which tiers were used.

Because allocation adapts, Leant2 may find candidates that Leant’s fixed-window configurations missed (the three “no term found” controls and the two “none passed” verification-window controls of the baseline are examples), and it may rank candidates differently: the cost vector of Section 6.4 replaces Leant’s engine-specific ranking. Both are permitted by Definition 1.5.

## 6.4 Ranking and multiple answers

The cost of a candidate is a structured vector, not a scalar<sup>[P2,P3,P6,P8]</sup>:

(policy violations, term size, eliminations, recursion shape,  
transports, nonlocal providers, proof size, runtime estimate).

The first component is an admission constraint. Candidates are ranked lexicographically by a declared profile (small source by default), and a small Pareto frontier of behaviorally distinct candidates is exposed rather than the first result only. Diversity is counted over native terms modulo checked equivalence, not over printed strings; two candidates that agree on all observations are presented as distinct unless a checked equality merges them. “Use every input” is a preference, not a filter.

## 6.5 Resource limits and determinism

Three independent controls apply: logical work budgets (rule applications, unification steps, normalization steps, proof tasks, solver calls, candidate evaluations), wall-clock deadlines, and process-level limits (memory, output size, external process lifetime). Counters are monotone and charged even after rollback (Decision 2.8). Per-rule limits yield suspended branches; raising a proof budget never regenerates earlier terms. A deterministic mode fixes ordering and seeds so that a transcript replays exactly; a performance mode returns the first result from parallel workers and records which lane produced it<sup>[P3,P4,P9]</sup>.

Parallelism is restricted to closed request slices: whole lanes, candidate verifications, and connected components of the dependency graph with an independence certificate (disjoint owned metavariables, disjoint synthetic obligations, unchanged shared rigid context). Workers exchange closed plans and certificates, never live metavariable contexts<sup>[P1,P9]</sup>.

## 7 Native construction

This section fixes the rules; the search procedures that apply them, with pseudocode and traces, are in Section 15, and dependent elimination, retrieval, and carriers are in Section 16. The construction calculus is bidirectional. Checking  $\Gamma \vdash e \Leftarrow T$  drives introductions from the expected type; inference  $\Gamma \vdash n \Rightarrow T$  extends heads (locals, constants, projections) by spines and drives eliminations. Rules are labeled as *checked transformation*, *grammar-preserving normalization*, or *heuristic proposal*, and a fairness lane retains the alternatives that aggressive focusing drops<sup>[P3,P6,P9]</sup>.

### 7.1 Introductions

For a target that weak-head-normalizes to  $\prod_{x:A} B(x)$ , introduce a fresh local preserving binder information (explicit, implicit, strict-implicit, instance-implicit). Introduction is existence-preserving for every binder kind, including type parameters, indices, and instance arguments, so it is the default focusing policy for function targets. It does not yield the smallest term: an existing function of the target type must also be considered, so a cheap *exact-term lane* runs before eta-expansion, but a cheap inhabitant never suppresses contract-sensitive alternatives<sup>[P3,P4,P5]</sup>.

For an inductive target, enumerate all constructors rather than committing to the first; freshen the constructor’s universe levels, instantiate parameters, and unify the result with the target in one transaction. An index equation that is inconsistent rejects the constructor; one that is merely blocked keeps it and records a delayed constraint. In dependent pairs and subtypes, the data field remains a choice point until every dependent field and the contract are satisfied; proof irrelevance may deduplicate proofs of a fixed proposition, never data<sup>[P1,P6]</sup>.

### 7.2 Application spines

**Definition 7.1** (Spine expansion). For a head  $c : \prod_{a_1:A_1} \cdots \prod_{a_k:A_k} R(\vec{a})$ : freshen the universe parameters of this occurrence; for each admissible prefix length  $j \leq k$ , in a fresh transaction, create owned holes  $?a_1, \dots, ?a_j$  in telescope order with exact binder information, and unify the residual type  $\prod_{a_{j+1}} \cdots R$  with the target; keep the full argument vector so that later arguments can constrain earlier ones; classify each hole by role; discharge determined instance and proof arguments cheaply; suspend blocked constraints with dependency stamps; and yield the application together with its whole coupled continuation<sup>[P2,P3,P6,P8,P9]</sup>.

Partial applications are admitted because a useful application length changes after reduction or when a type variable is instantiated by a function type. Repeated use of one head is allowed. The prototypes build this on `MVarId.apply` with `newGoals := .all` (original argument order, which forces witness-first backtracking to be tested) and `approx := false`; the design treats `apply` as a first adapter and differential-tests any optimized spine planner against it<sup>[P2,P3,P6]</sup>.

Local providers keep their exact free-variable identities and universes; only global constants are freshened per occurrence. Named universe parameters of the query are rigid; fresh level metavariables of library constants are flexible; there is no implicit universe lifting and no lowering of a polymorphic query to `Type 0`<sup>[P1,P2,P4]</sup>.

### 7.3 Known-major projections

Applying `Sigma.snd` to an unknown pair leaves an unknown family and an unknown major premise. Instead, projections are first-class search actions over *known* locals: enumerate `Expr.proj` over local structure values using structure metadata, infer the exact dependent type,

and match. This fixed a real regression ( $p : \Sigma_n \text{Fin}(n + 1) \vdash \text{Fin}(p.1 + 1)$ ) generically, and projection-first ordering also fixed a heartbeat exhaustion caused by constructor growth before projections in another prototype<sup>[P1,P5]</sup>.

## 7.4 Polymorphic and higher-rank instantiation

Higher-rank arguments are Pi-typed argument holes and are constructed by introduction; there is no rank-N approximation. Type-valued holes are filled by a staged *type-invention ladder*<sup>[P1,P2,P6,P9]</sup>.

1. types forced by unification with the rigid target;
2. expected-type-directed construction of polymorphic arguments (introduce the binder rather than enumerate monotypes);
3. a bounded, cost-ordered *type frontier*: types in the telescope, output, and contract; types of locals and of provider domains; small products, sums, sigma and function types over those; registered carrier templates;
4. quantified types and accumulator carriers, with a separate size cost.

Pool exhaustion is a missed instantiation, not impossibility. Stuck higher-order equations are postponed, then bounded-instantiated with small lambda, projection, and application templates, each rechecked by Lean; there is no general higher-order unification. The Church and rank-N fixtures of the baseline are the primary regression set for this ladder.

## 7.5 Instances and dictionary identity

Two modes must not interfere. In *canonical instance mode*, instance obligations are Lean’s own resolution, delayed until their input parameters are known and woken on assignment. In *explicit dictionary mode*, a dictionary is data: local instances are named providers, the actual dictionary expression is stored at each method occurrence, and search may backtrack over dictionary choices. Two dictionaries of the same class type are never interchangeable without a proved equivalence; only **Prop**-valued classes enjoy proof irrelevance. Cache keys and candidate identity include dictionary payloads. The fixture in which the second of two **Inhabited Nat** values must be selected because the contract says so is a permanent regression<sup>[P1,P3,P5,P6,P8]</sup>. Leant2 never manufactures global instances.

## 7.6 Normalization and equality

Normalization is controlled work: memoized weak-head normalization, a small transparency ladder with per-operation budgets, escalation on failure, and charges that survive rollback. Definitional equality is an operation, not a canonicalizer: there are no pretty-printed normal-form cache keys. The order for an index mismatch such as  $\text{Vec } A (n + m)$  against  $\text{Vec } A (m + n)$  is cheap definitional equality, then registered proof-producing normalizers, then a bounded equality proof  $q : n + m = m + n$  followed by an explicit transport along the equality recursor with a Lean-checked motive; never a raw cast, and never index erasure<sup>[P3,P4,P6,P8]</sup>.

## 7.7 The constructive propositional lane and classical candidates

Leant’s baseline includes propositional goals proved constructively (**synth-basic**, **synth-manual**, **synth-prove**) and, with **synth-classical on**, classically. It reports **forall p : Prop, p or not p** as “provably uninhabited” because Djinn’s LJT search found no derivation. Leant2 reproduces this capability with two lanes and one rule.

The *constructive propositional lane* reifies the propositional fragment of a goal (connectives  $\rightarrow, \wedge, \vee, \neg, \leftrightarrow, \perp, \top$  over atoms that are opaque Lean propositions or types), runs a terminating focused calculus over it, and returns either a Lean proof term or a checkable certificate that no derivation exists in that fragment. The certificate is a finite Kripke countermodel, checked by recomputing forcing, as in Forge’s extension round [4]. The result is `fragmentUnderivable`, with the fragment named, and it is never promoted to `impossible`<sup>[P1,P4,P6,P8]</sup>.

The *classical lane* adds `Classical.em`, `Classical.byContradiction`, and `Decidable.em` as explicit providers. It is enabled by the *standard* profile and scheduled after the constructive lane has returned a certificate, so that constructive candidates are found first and classical ones are labeled by their axiom inventory. For the baseline this yields the classical candidates of `synth-manual` and, for `forall p : Prop, p or not p`, both a fragment certificate and a labeled classical candidate.

## 8 Dependent elimination

Case analysis is a motive-construction problem, delegated to Lean’s own machinery behind a wrapper that returns the branch goals, the old-to-new local mapping, and any transport obligations<sup>[P3,P4,P6,P8,P9]</sup>.

1. Compute the dependency closure of the scrutinee  $x : I \vec{p} \vec{i}$ : every later local whose type or value mentions  $x$ , its indices, or a selected dependent.
2. Generalize the closure into the motive, preserving order; inspect the real eliminator and its result-sort restriction; build the motive over the indices and the scrutinee.
3. Instantiate each constructor branch with its fields and index equations; reintroduce the dependent context with transformed types; record the branch-to-parent substitution.
4. Discharge impossible constructor and index combinations only by checked evidence: unification contradiction, no-confusion, or a proved local contradiction. Record for each removed branch whether it was removed by kernel-checked contradiction, by a trusted exact-fragment theorem, or by heuristic omission; only the first two support coverage claims.
5. Reconstruct the eliminator application.

Scrutinee selection prefers locals that occur in target indices, that a discriminating contract or observation demands, or that block reduction. Split history is tracked by discriminant identity, so repeated splitting of a recursive input is recognized as bounded unrolling and not as recursion. Elimination restrictions are enforced natively: `False` eliminates anywhere; `Nonempty A` is not an executable `A`; `Exists` is not a computational pair; quotients require a respect proof through their registered lift. Decidable conditionals require an admitted `Decidable` instance and a checked link between the condition and the branch facts<sup>[P2,P4,P7]</sup>.

Every prototype that eliminated an indexed family with `cases` picked up `propext`; the receipts report it, and the strict profile treats it as a policy decision rather than hiding it.

## 9 Library composition and carrier invention

### 9.1 The provider index

A persistent index is built at import over declaration types: identity, universe parameters, telescope, weak-head result shape, argument heads, constructor or projection role, computational-versus-proof classification, and registered specification lemmas. The index is a discrimination tree over normalized result heads with mandatory wildcard and reducible-head buckets; it may over-return but never silently under-returns, and an intentionally narrowed query is never negative evidence<sup>[P4,P5,P8,P9]</sup>.

Retrieval widens in tiers, each recorded in the result: locals and local dictionaries; the goal’s own constructors and projections; explicitly requested providers; registered project components and tagged *synthesis lemmas*; nearby namespaces and imports; a budgeted broad scan. A fixed cap such as fifty providers is a heuristic, never “exhaustive”<sup>[P1,P9]</sup>.

## 9.2 Component recipes and the join

A component is a recipe ( $\Delta \vdash e : \sigma$ , contract lemmas, cost, provenance) with  $\Delta$  the dependency-closed telescope. Reuse under  $\Gamma$  requires a telescope substitution  $\rho : \Delta \rightarrow \Gamma$  applied uniformly to  $e$ ,  $\sigma$ , and the contracts, followed by a native recheck. Forward synthesis stores cheap components in a typed shared DAG; backward synthesis demands result types; a *join* matches demanded premises against stored components and propagates assignments across the whole application<sup>[P6,P7]</sup>.

A *demand graph* from target holes and observations yields, per demand, a *candidate context slice*: the locals mentioned in the type, contract, and constraints plus a bounded predecessor closure. The slice is searched first and the full context second; slice solutions are transported by explicit weakening and rechecked. This is the direct response to Leant’s indexing diagnostic, in which a handler found standalone is missed in the full context<sup>[P7,P8]</sup>.

## 9.3 Providers with contracts

A provider is an exact term plus useful theorems: the declaration, its universe telescope and binder structure, result-head keys, instance dependencies, a use policy, and *specification entries* naming theorems  $\forall x, P(x) \rightarrow Q(x, g(x))$  that relate the operation to its inputs and outputs. Entries are checked at registration and never trusted as annotations; a summary must name its justifying theorem and instantiate it at the same occurrence. Multiple specifications per provider are selected by the demanded observation (length, order, multiset). Provider exclusion follows declaration dependencies, not namespace blacklists, so a withheld reference implementation cannot be retrieved through an alias<sup>[P3,P5,P8,P9]</sup>.

Parametricity is a ranking heuristic and a template source, never a pruning axiom: Lean’s axioms admit nonparametric functions<sup>[P2,P5]</sup>.

## 9.4 Carrier invention

When an eliminator or fold is applied and the continuation still consumes an argument after the traversal, the carrier is not the result type. The engine tries, in increasing structural cost, the carrier grammar

$$C ::= R \mid I \rightarrow C \mid C_1 \times C_2 \mid \text{Option } C \mid \Sigma d:D, C(d),$$

seeded by demanded types, with three demand rules: *parameter postponement* (a parameter consumed after the traversal becomes the domain of a function-valued carrier), *result pairing* (two demanded summaries become a product), and *accumulator generalization* (a changing argument becomes an accumulator). Generalization is *behavioral*: it includes arguments whose values change across calls, not only those the type depends on. The worked trace shared by four proposals is defaulted indexing, `List A -> Nat -> Option A`, as `foldr` with carrier  $\mathbb{N} \rightarrow \text{Option } A$ ,  $z(i) = \text{none}$ ,  $s(x, r)(0) = \text{some } x$ ,  $s(x, r)(k+1) = r(k)$ ; one proposal checked a hand-supplied instance of it, none discovered it automatically<sup>[P1,P4,P5,P7,P8,P9]</sup>.

Base and step are attempted jointly with the outer continuation; failures are memoized only under the full context, motive, and bound. Typed anti-unification of repeated residual obligations proposes shared helper types and let-bound helper holes solved under the intersection of scopes<sup>[P1,P2,P8]</sup>.

## 10 Recursion

Recursion is the area where the nine proposals agree most closely on design and have demonstrated the least. Every recursive definition in every prototype was hand-supplied; the tactics filled branch bodies. This section states the consensus design and marks its unimplemented parts; the schema-generation, motive, invariant, measure, and lowering algorithms, with worked traces, are in Section 17.

### 10.1 Routes, in order

Given a target whose telescope contains an inductive argument, the recursion lane tries, in order<sup>[P2,P3,P4,P5,P8,P9]</sup>:

1. a permitted library combinator with a matching specification entry (`map`, `foldr`, `foldl`, option combinators, registered recursors);
2. a nonrecursive eliminator or fold already in the local context, including Church-encoded values with function-valued carriers (Section 9.4);
3. a *structural schema* generated from the recursor metadata of the input type;
4. a strengthened or accumulator-generalized structural schema;
5. a well-founded schema over an explicit measure.

Mutual recursion is a jointly synthesized strongly connected component over a tagged sum with a shared termination argument; nested inductives use their actual recursors; both are later extensions<sup>[P3,P8]</sup>.

### 10.2 Schemas and capabilities

**Definition 10.1** (Recursion schema). A schema records the function telescope, the decreasing argument, the result family (motive)  $M(\vec{i}, x)$ , one branch per constructor with its fields and index equations, the set of *recursive-call capabilities* available in each branch, the termination obligations, and the lowering plan<sup>[P4,P7,P9]</sup>.

**Invariant 10.2** (Recursive calls are capabilities). A branch never contains an unrestricted local  $f : \forall x, T(x)$ . A recursive call is available only as  $\text{rec} : \prod_{y:D} \prec(y, x) \rightarrow M(y)$ , where the decrease evidence is a constructor field for structural recursion or an obligation  $\mu(y) < \mu(x)$  for well-founded recursion. The obligation is synthesized or the call is rejected; the program is never rejected because a measure failed<sup>[all]</sup>.

The failed first run of one prototype, in which unfiltered implementation-detail locals exposed the provisional definition and Lean rejected the resulting cycles, is the empirical reason for this invariant. The acceptance gate additionally audits the dependency graph of the declaration bundle for provisional self-reference<sup>[P2]</sup>.

### 10.3 Motive construction

**Definition 10.3** (Motive by dependency-closed generalization). Choose the discriminant  $z : D(\vec{p}, \vec{i})$ . Partition the surrounding variables into fixed parameters and generalized arguments, closed under dependence and under behavioral change (arguments whose values differ between the call and the recursive call). The candidate motive is

$$M(\vec{i}, z) = \prod \Delta(\vec{i}, z). R'(\vec{i}, z, \Delta),$$

checked by Lean at the recursor’s sort. The schema instantiates the recursor, exposes the method holes and induction hypotheses, translates the residual contract per branch, and requires a checked *final specialization* back to the original target<sup>[P2,P5,P6,P9]</sup>.

Motives are tried on a complexity ladder: the direct result family; generalize the dependent arguments; small subsets of the changing arguments; accumulator and invariant templates. The worked examples common to the proposals are `Vec.map` with  $M(n, xs) = \text{Vec } \beta n$ , `Vec.zip` with  $M(n, xs) = \text{Vec } \beta n \rightarrow \text{Vec } (\alpha \times \beta) n$ , vector lookup with  $M(n, xs) = \text{Fin } n \rightarrow \alpha$ , and accumulator reversal with the motive quantified over the accumulator. Motive-subset search is exponential in the number of changing arguments; the mitigation is the ladder plus *failure-directed generalization*: classify why an induction hypothesis cannot be applied and compute the smallest dependency-closed telescope extension that repairs it<sup>[P2,P6]</sup>.

## 10.4 Proof-carrying motives

The central idea for program-and-proof co-synthesis appears in all nine proposals under different names (certificate algebras, refinement motives, certified recursion, packed motives): strengthen the motive so that the induction hypothesis carries correctness<sup>[all]</sup>.

**Proposition 10.4** (Certificate-algebra composition). *Let  $C(x) = \{y : B(x) // R(x, y)\}$ . A native recursor application with motive  $C$  and checked method terms yields  $\prod_{x:D} C(x)$ , hence an output satisfying  $R$  for every input. For function-valued outputs the motive is  $C(x) = \prod_{a:S} \{y : B(x, a) // R(x, a, y)\}$ . The list instance is the checked theorem*

$$R([], z) \wedge (\forall x xs v, R(xs, v) \rightarrow R(x::xs, s(x, v))) \Rightarrow \forall xs, R(xs, \text{foldr } s \ z \ xs).$$

Each recursive branch receives the smaller output *and* its local specification, so a contract failure backtracks into the branch choice rather than rejecting a finished term. The alternative route, synthesize then prove by induction on the checked equations, is kept as a portfolio alternative and is preferred when the contract is not pointwise. Two proposals checked hand-written instances (`certifiedMap` with a functional `MapRel`; `certifiedAppend`; `indexFold` with its universal correctness)<sup>[P1,P5,P8]</sup>.

## 10.5 Accumulator invariants

When a helper with an accumulator is needed, the invariant is a bounded synthesis subproblem, never an assumption. The grammar is built from the public relation, the accumulator variable, the available operations and their registered algebraic summaries, and known unit elements; each candidate yields initiation, preservation, and finalization obligations proved by the proof portfolio. The example shared by six proposals is  $\text{go}(xs, acc) = \text{reverse}(xs) ++ acc$  for accumulator reversal; two of them proved the invariant by hand for the fold selected by a finite experiment, and none synthesized it<sup>[P2,P3,P4,P6,P8,P9]</sup>.

## 10.6 Termination

The well-founded schema is  $F(x) = \text{step}(x, \lambda y (h : y \prec x). F(y))$ . Measures come from a finite grammar: natural-number arguments, registered size functions, structural sizes, bounded differences such as  $n - i$  with truncated-subtraction semantics handled honestly, lexicographic tuples, and user-supplied relations. Each proposed call yields  $\mu(y) < \mu(x)$  as an ordinary obligation; a schema is registered only after Lean’s termination elaboration accepts it. Fuel is legal only when it is part of the interface or proved sufficient<sup>[P1,P3,P5,P8]</sup>.

## 10.7 The kernel/compiler boundary

Four prototypes independently produced an axiom-free recursor term that the code generator rejected (“code generator does not support recursor `T.rec` yet”). The consequence is a design rule<sup>[P4,P6,P8,P9]</sup>:

**Invariant 10.5** (Two products per recursion plan). A recursion schema produces a logical reconstruction, used for proof, and an executable definition, lowered from the plan through the equation compiler with pattern equations and, when needed, `termination_by` and decrease proofs. If the two differ, the engine proves a correspondence theorem  $\forall t, f_{\text{eq}}(t) = f_{\text{rec}}(t)$  and checks both, or proves the contract directly for the executable definition. The receipt records `kernelChecked` and `compiled` separately.

Lowering recovers the major argument, patterns, branch locals, capabilities, and generalized arguments from the plan, not from printed recursor text, and elaborates the result in a staging environment that contains only approved dependencies<sup>[P6]</sup>. The recursion-schema grammar tags each construction as genuine recursor, bounded unrolling, or nonrecursive, so that repeated case splits are not mistaken for recursion<sup>[P5]</sup>.

## 10.8 Status

Demonstrated: filling branch bodies inside hand-supplied structural skeletons on indexed vectors and lists; hand-written proof-carrying motives and their universal correctness theorems; the code-generator boundary. Not demonstrated: automatic schema generation, motive generalization, carrier discovery, invariant synthesis, measure synthesis, lowering. This is the gate that the roadmap of Section 23 treats as the decisive capability milestone<sup>[P8]</sup>.

# 11 Behavioral constraints

Behavioral information enters search from three directions: the type constrains shape, the contract propagates backward into holes, and observations and counterexamples constrain choices. None replaces the final proof<sup>[P3,P9]</sup>. The propagation rules, the refinement loop, and the pruning procedures are given as algorithms in Section 18; scheduling and caching in Section 19; proof services and the gate in Section 20.

## 11.1 Contract decomposition

A *contract decomposer* takes the elaborated contract  $\Phi$  and a construction step and returns local obligations together with a *reconstruction term* proving that they imply the original. Each extracted view is tagged *necessary*, *sufficient*, or *equivalent*, and the original  $\Phi$  stays authoritative<sup>[P2,P6,P8,P9]</sup>. The initial library of decomposition rules is:

- lambda introduction: a pointwise  $\forall x, \text{Post}(x, f\ x)$  becomes  $\text{Post}(x, ?r)$  for the body hole (Proposition 3.3);
- constructor application: a record invariant becomes projection equations on the fields;
- decidable conditional: the postcondition splits under the guard and its negation;
- provider application: a registered theorem  $\forall a, P(a) \rightarrow Q(g(a))$  turns the demanded  $Q$  into the *sufficient* obligation  $P(a)$  on the argument. Sufficiency does not license deleting arguments that fail  $P$ ; they remain in another lane unless a necessity theorem exists;
- conjunctions, constructor-aligned relations, and registered pre/postconditions.

Multi-call laws stay as higher-order obligations; an opaque higher-order provider stops propagation with a symbolic residual.

## 11.2 Typed observations and residual evaluation

An observation is a typed substitution into the input telescope, precondition proofs, an exact output observation, and an evidence policy. Dependent outputs are checked at their induced type; **Fin**  $n$  values carry their bound; dependent pairs generate the first field before instantiating the second's type; polymorphic functions are tested at several concrete types with distinct dictionary payloads. The default evaluator never runs effects; **I0** is never executed as a test; a timeout is unknown, never false<sup>[P3,P6,P7]</sup>.

Partial programs are evaluated *residually*: evaluation proceeds until a hole and yields a typed residual (known constructors, closures, suspended applications, conditionals, named holes with observed arguments). A residual is a constraint on a particular use of the hole; multiple observed calls of one hole constrain one shared function, recorded in a hole-observation table keyed by hole and argument tuple so that  $h(0) = 0$  and  $h(0) = 1$  are recognized as inconsistent. Recursive-call outputs under a sketch are symbolic and never support hard pruning alone<sup>[P2,P6,P7]</sup>.

## 11.3 Proof services

Proof obligations are discharged by a portfolio behind one adapter, in cost order: conversion and **rfl**; local assumptions; constructor proofs; **decide** by ordinary kernel reduction; curated **simp** with an explicit lemma set; **omega**; Forge's certificate checkers for cone, Farkas, affine-witness, recurrence, and conserved-invariant obligations [4]; **grind**; Aesop with query-specific rules; optional pinned Mathlib procedures. Induction is a deliberate schema choice coordinated with the recursion plan, not a generic proof step<sup>[P1,P3,P5,P6,P9]</sup>.

The adapter's contract is Forge's: save the full state before the worker, restore on any failure including resource limits, and on success instantiate the proof term and reject it if it contains **sorry**, mentions a free variable outside the goal's context, does not type-check against the original goal, or depends on an axiom outside the policy. A reply is one of **Proof** (with any permitted assignment patch, in joint mode), **Refutation**, **ReducedGoal** with a reconstruction proof, **Unsupported**, or **Unknown** with the budget consumed. A bare Boolean is not an adequate interface; a failed proof never falsifies a candidate; a result obtained under a branch guard discharges only guarded obligations<sup>[P2,P3,P7]</sup>. **native\_decide** is not used for proofs; native evaluation may rank or propose but never certifies<sup>[P8,P9]</sup>.

Proof debt is scheduled: obligations carry cost and readiness, index and arithmetic obligations wake on relevant assignments, and heavy tactics are delayed until the computational holes they mention are assigned or exposed as witness choices<sup>[P1,P5]</sup>.

## 11.4 Certified counterexample-guided refinement

**Definition 11.1** (Verifier).

$$\text{verify}(t, \Phi) \in \{\text{Proof}(p), \text{Counterexample}(c), \text{Unknown}(r), \text{Failure}(e)\},$$

where  $c$  is a value of the actual input type with proofs of its preconditions and a checked violation. A counterexample enters the bank only after replay in Lean's exact interpretation; a solver model is a proposal until decoded and replayed; an evaluator timeout adds nothing<sup>[P1,P4,P5,P8]</sup>.

The loop: choose the next candidate consistent with the bank; try to prove  $\Phi(t)$ ; on unknown, schedule a counterexample search and keep the candidate in the unresolved queue at a larger proof tier; on a certified counterexample, add it to the bank, repartition the observation buckets, and wake suspended work; publish only through the acceptance gate. "Failure of one is not success of the other": a proof attempt and a counterexample search for the same candidate run as separate resumable tasks<sup>[P6,P7,P9]</sup>.

**Proposition 11.2** (Progress of finite counterexample search). *With  $N$  candidates in a finite grammar, permanently retained valid counterexamples, and bank-consistent selection, at most  $N$  rejections occur; unknown verdicts do not count as rejections<sup>[P3,P8]</sup>.*

Finite domains are promoted to universal statements by an untrusted enumerator that proposes a literal and a kernel `decide` that proves the universal statement, decoupling synthesis time from proof size; the finite promotion theorem  $(\forall x, x \in L) \wedge (\forall x \in L, R(x)) \Rightarrow \forall x, R(x)$  is checked once<sup>[P6,P9]</sup>. Two prototypes implemented finite Boolean CEGIS in Lean with the universal correctness proved by ordinary `decide`; one implemented a proof-carrying certifier `certify : Affine → Option {c // Spec c.eval}` whose acceptance object is a Lean proof rather than a Boolean<sup>[P7,P8,P9]</sup>.

## 11.5 Reversible observational buckets

Candidates are grouped by their observation vector  $\text{obs}_{\mathcal{X}}(t) = (t(x_1), \dots, t(x_m))$ . A bucket holds an active representative *and* its members or their regeneration recipes; a new observation splits buckets and reactivates members; a second liveness rule rotates parked members even without new observations, since a parked candidate may have a shorter proof. Permanent merging is unsound: the Boolean pair (constant true, identity) agrees on `true` and differs on `false`, and three separate finite experiments showed that permanent deduplication on the initial bank loses the correct program (identity versus reverse;  $x$  versus  $y$  on  $(0,0)$ ). Observational agreement is provisional and never a cache equivalence<sup>[P1,P2,P3,P5,P9]</sup>.

## 11.6 Certified pruning

**Proposition 11.3** (Sound sketch pruning). *Let  $P$  be a partial program with completions  $\text{Comp}(P)$  and let  $A(P)$  be an abstract summary with  $\{\text{Beh}(t) : t \in \text{Comp}(P)\} \subseteq \gamma(A(P))$ . If some valid input  $\rho_0$  has a defined observation for every completion, and a checked necessary condition  $K$  on that observation satisfies  $\gamma(A(P)) \cap K = \emptyset$ , then no completion of  $P$  satisfies  $\Phi$ <sup>[P3,P5,P6,P8,P9]</sup>.*

A domain plugin implements Forge’s `CertificateSpec` shape: a typed reification, an interpretation into Lean, a soundness theorem for its transfer rules, and refusal cases. Its replies are `Unsupported`, `RankingHint` (no logical authority), `NecessaryConstraint` with proof, `Counterexample` with validity evidence, or `NoCompletion` with a certificate and its exact grammar scope. Missing transfer rules yield top, never bottom; an under-approximation may produce witnesses but may never justify pruning; grammar identity is part of every certificate’s applicability. Initial domains are list lengths and sizes, affine and linear arithmetic (Forge’s checkers), constructor tags, bounded machine integers, and small multisets. One proposal proved the length-abstraction necessity and sufficiency for its fold grammar; another proved an exact five-list separator family for affine folds and generated twenty-seven universal contracts from it<sup>[P3,P4,P5,P7]</sup>.

External solvers have three distinct uses, each with its own validation: proposing sketch choices, proposing counterexample inputs, and proving theory obligations by a reconstructed certificate. `unsat` on an under-approximate encoding excludes only that encoding. Typed SMT sketches encode dependent holes either *branch-expanded* (each witness alternative owns a specialized child grammar) or *native-first* (the solver picks the witness and Lean generates the residual); untyped multiplexers are not used<sup>[P2,P5,P6]</sup>.

## 12 Caching and equivalence

Three equivalences are kept apart: structural identity, checked semantic equality, and finite observational agreement<sup>[P8]</sup>.

Cache keys include the environment epoch and provider policy, the canonical dependent context preserving `let` values and instance identities, the target, the residual contract, the transparency setting, universe parameters, the assignment state of shared metavariables, the grammar identity, domain versions, and the observation-bank version; never a pretty-printed type. Positive entries are closed term templates over a canonical telescope, reinstantiated through an explicit context embedding and rechecked. Negative entries must state whether they are a semantic refutation, a bounded-grammar exhaustion with its bound and provider set, or an operational miss; a failure at fuel twenty says nothing about fuel one hundred; a timeout is never cached as uninhabitability<sup>[P1,P2,P4,P5,P7,P9]</sup>.

Nogoods store the smallest justified dependency slice, tagged with request, contract, instance selections, epoch, and grammar scope; a budget exhaustion is a penalty, not a nogood. Definitional-equality comparisons must not mutate branch unknowns. E-graphs are permitted only over a reified typed fragment with proof-carrying edges; proof irrelevance merges proofs, never dictionaries. Library learning stores successful closed subterms as checked helper schemas, with leakage rules for evaluation<sup>[P1,P2,P3]</sup>.

## 13 The acceptance gate

### 13.1 Pipeline

The gate is the only constructor of an accepted result. It runs, in a fresh native state<sup>[all]</sup>:

1. **Restore the origin.** Check the environment epoch and the origin record; reject stale generations. Reconstruct the exact type and contract from the frozen query, never from the candidate’s inferred type.
2. **Close.** Instantiate all expression and level metavariables; reject any remaining hole, dangling free variable, loose bound variable, or unresolved elaboration obligation. Close the program and the proof over the original telescope, preserving `let` values; generalize only permitted universes and locals.
3. **Bundle.** Collect auxiliary declarations in dependency order; check that no declaration references a provisional self-reference and that every helper body is present. The published theorem refers to the published definition by its exact generated name.
4. **Kernel-check.** Add the bundle through the ordinary checked declaration path in a scratch environment, with `debug.skipKernelTC` pinned to `false` in the options passed, and with a positive signal that the kernel task completed rather than mere presence of the constant (Section 1.5). Check the program at the frozen  $T$  and the proof at  $\Phi$  applied to *that* program, retained even when  $\Phi$  ignores its argument.
5. **Audit.** Compute the transitive axiom closure of the proof and the transitive runtime-dependency closure of the program separately; enforce the profile (Section 3.4), rejecting `sorryAx`, unapproved axioms, `implemented_by` and `extern` outside the runtime policy, `partial` and `unsafe` dependencies, and any use of the withheld reference implementation. Policies are over the dependency graph, not over strings.
6. **Compile.** When the execution policy requires it, compile the exact program and record the outcome; a kernel-accepted term that fails code generation is not `compiled`.
7. **Replay the source.** Delaborate the result in the pinned presentation context, re-elaborate the source against the frozen signature, and compare meaning with the accepted expression; on mismatch report a “newly checked source variant” rather than the original. Text is never the authority: implicit arguments, notation, and instance drift change on re-elaboration.
8. **Publish atomically** with a receipt. Nothing is published on any failure.

**Theorem 13.1** (Conditional acceptance soundness). *Assume the validation environment is the recorded one, the kernel and the imported environment are correct, and the gate accepts only*

*closed instances of the frozen contract. Then every published **verified** result satisfies the two judgments of Definition 3.2 relative to the displayed premises and the allowed axioms. No search step, heuristic, index, solver, or proof service occurs as a premise. This is an architectural implication, not a formal verification of the gate<sup>[all]</sup>.*

**Invariant 13.2** (Inference is not acceptance). Success of `inferType`, `isDefEq`, `Meta.check`, or a tactic’s “no goals” is never acceptance. Forge’s test workers demonstrate each failure: a goal assigned `True.intro` without a type check, a proof mentioning a phantom local, an admitted theorem, and a `Classical.choice` proof under a constructive policy [4].

## 13.2 Receipts

A receipt records the request identity; toolchain, import, and environment identities; provider and policy selections and the retrieval tiers used; the accepted declarations and the contract type; the axiom closure of the proof and the runtime-dependency closure of the program; the flags `kernelChecked`, `compiled`, `evaluatedOnExamples`; the source-replay outcome; search bounds and the incompleteness ledger; and the accounting of rejections, inapplicabilities, and infrastructure failures. Hashes are identity checks, not semantic equivalence. Remote-checker receipts, such as the AXLE transcriptions in the nine packages, are operational evidence and are never confused with local kernel artifacts<sup>[P3,P7,P8]</sup>.

## 13.3 Isolation

In-process private constructors protect against mistakes, not against hostile metaprograms. The strict deployment profile runs the gate in a separate process with the same kernel against the project snapshot, receiving only the serialized bundle and never executing proposed tactic code. Untrusted extensions (learned proposers, external solvers, plugin metaprograms) run in a sandboxed proposal worker with a restricted publication interface. An independent checker such as Lean4Lean [25] is an optional higher-assurance route, reported only when performed<sup>[P3,P6,P9]</sup>.

# 14 Frontends and deployment

## 14.1 Entry points

All entry points build the same frozen query and share the gate<sup>[P3,P6,P9]</sup>:

- a tactic, by `leant2`, which fills the current goal; when the goal is data, it may be combined with a contract as a subtype or with a separately supplied `satisfying` clause;
- a command, `##synth T satisfying fun f => P using [providers]`, for exploration without registering declarations;
- a declaration command, `synth_def name : T satisfying fun f => P`, which publishes the implementation and its correctness theorem atomically, at command-elaboration level so that recursive declaration bundles and the compiler are available;
- a batch API used by the worker and by test harnesses.

Unknown names in a contract are errors, not auto-bound implicits. The proposed syntax is a design sketch; none of the nine prototypes implements it.

## 14.2 Compatibility REPL for the baseline

To run the baseline corpus, Leant2 ships a small REPL adapter that accepts Leant’s transcript syntax: session declarations, `:synth T`, `:synth f : T where P`, `:reset`, `:quit`, and

**##eval/##check.** Every `:set` line is accepted and ignored (Decision 1.1). The adapter is an adapter, not a second elaborator: it feeds declarations to a session environment, bumps the epoch on `:reset`, and submits each query to the batch API. Its output is a new golden format that prints outcomes and accepted candidates with their axiom labels, and the baseline harness compares outcomes by Definition 1.5 rather than by text. Leant’s session-binding convention (`it1`, `it2`, ...) is preserved so that transcripts such as `synth-manual`, which evaluate `it2` after a query, replay.

### 14.3 Worker and controller

The core is a Lean library compiled with the project’s own toolchain; a globally compiled executable must not load incompatible `.olean` files. A lightweight controller, also in Lean, owns sessions, progress, cancellation, and durable requests; a replaceable project-local worker owns the environment, local contexts, metavariables, caches, and search state, and is launched under the project’s Lake configuration. Heavy solver calls and untrusted runtime evaluation go to subordinate processes. Protocol messages carry a version, the query identity, the epoch, and a cancellation token; replies carry request and frame identifiers so that a stale reply cannot satisfy a newer obligation. Durable state is source commands, the origin record, and plans; never pointers or metavariable identifiers. There is no Haskell component<sup>[P1,P2,P6,P7,P9]</sup>.

The narrow, version-sensitive adapter over Lean’s metaprogramming API is isolated in one module with toolchain-pinned regression tests for success, failure, rollback, and interruption of each operation. The nine prototypes exercised exactly this surface on Lean 4.34.0<sup>[P1,P4,P6,P9]</sup>:

```
Meta.saveState, Meta.restore, MVarId.intro1P, MVarId.apply, MVarId.cases,
MVarId.induction, mkConstWithFreshMVarLevels, whnf, Meta.check, addDecl,
collectAxioms.
```

### 14.4 Proposed module layout

| Module                          | Responsibility                                                    |
|---------------------------------|-------------------------------------------------------------------|
| <code>Leant2.Frontend</code>    | tactic, commands, REPL adapter; builds frozen queries             |
| <code>Leant2.Core.Origin</code> | origin record, epochs, policies, result algebra                   |
| <code>Leant2.Core.Plan</code>   | serializable search plan, replay, cross-process packaging         |
| <code>Leant2.Native</code>      | the version-sensitive adapter and transaction combinators         |
| <code>Leant2.Search</code>      | obligation graph, cursor frames, scheduler, ledger                |
| <code>Leant2.Rules</code>       | introductions, spines, projections, constructors, transport       |
| <code>Leant2.Elim</code>        | dependent elimination and motive construction                     |
| <code>Leant2.Index</code>       | provider index, tiers, component recipes, demand graph            |
| <code>Leant2.Recursion</code>   | schemas, capabilities, motives, carriers, lowering                |
| <code>Leant2.Contract</code>    | decomposer, observations, residual evaluation, buckets            |
| <code>Leant2.Proof</code>       | proof-service adapter; Forge checkers, <code>grind</code> , Aesop |
| <code>Leant2.Domain</code>      | certified abstract domains and solver bridges                     |
| <code>Leant2.Accept</code>      | the gate, audits, source replay, receipts                         |
| <code>Leant2.Worker</code>      | controller, worker, isolation                                     |

Core modules depend only on Lean and its standard library. Mathlib-backed proof methods, Forge’s Mathlib-dependent checkers, SMT adapters, and learned ranking live in optional packages pinned separately<sup>[P8]</sup>.

## Part II

# Algorithms

## 15 Algorithms: construction search

Sections 4.3–7.5 fix *what* the construction engine maintains and *why*. This section fixes *how*: the concrete procedures, in the order a worker executes them, with the data-structure operations they perform and two exact traces. Every procedure is a merge of the nine proposals’ versions; where they disagree, the best-specified version is taken and the alternatives are named. Pseudocode is Lean-like; `transaction` always means the atomic attempt of Section 5, and `ledger.charge` is never refunded (Decision 2.8).

### 15.1 The main loop over the obligation graph

#### 15.1.1 The reference recursion and what all nine prototypes ran

Every prototype is an instance of one recursive skeleton: pop an obligation, try each alternative inside a snapshot, and recurse on `children ++ rest`, so that the head goal’s solution is tentative until every remaining obligation succeeds<sup>[all]</sup>. The proposals differ only in rule order, in the apply configuration, and in how exceptions are classified:

- rule order: P1: intro, locals and projections, providers and constructors and `Eq.refl`, cases; P2: refl, exact local, intro, constructors, local spines, cases; P3: exact local, intro, constructors and providers, local spines; P4: intro, refl, locals, providers, constructors; P5: intro, refl, locals, projections, constructors, cases; P6, P8, P9: refl, intro, locals, (providers), constructors, cases; P7: intro, refl, locals, constructors, cases on non-recursive non-indexed locals;
- apply configuration: P2, P5, P6, P7, P9 pass `newGoals := .all`; P1 adds `synthAssignedInstances := false`; P3 passes `newGoals := .all`, `approx := false`, `synthAssignedInstances := false`, `allowSynthFailures := true`; P4 and P8 use the default configuration, whose `nonDependentFirst` order schedules a proof field before the witness it constrains (proof-first, see Section 15.1.5);
- interruption: P1 rethrows `Exception.internal`, P3 rethrows `isInterrupt`, P5 rethrows `isInterrupt || isMaxHeartbeat`; P2, P4, P6, P7, P8, P9 catch everything and say so is wrong for production;
- bounds: P3 and P8 iterate the fuel bound upward (1..24 and 1..17) from the original saved state; P1, P2, P5, P7 add a separate case-split bound; P3 additionally requires the root to contain no metavariables before reporting success.

#### 15.1.2 Cursor frames: the resumable production loop

The production engine keeps the semantics of the recursion but replaces the host call stack by explicit frames so that a branch can be suspended after a bounded quantum, resumed, serialized as a plan suffix, and scheduled fairly<sup>[P2,P6,P8]</sup>. A frame is exactly the data the recursive version keeps implicitly<sup>[P8]</sup>:

```
structure Frame where
  focus      : HoleId          -- obligation being refined
  siblings   : List HoleId     -- rest of the coupled continuation
  saved      : Snapshot        -- Meta/Elab state before the current alternative
  family     : RuleFamily      -- exact/intro/ctor/proj/spine/elim/proof/recur
  cursor     : RuleCursor      -- resumable: next alternative of family for focus
  allowance  : Cost            -- remaining charged work for this frame
```

```

def advance (fr : Frame) (q : Quantum) : WorkerM StepResult := do
  fr.saved.restore           -- undo the previous alternative entirely
  ledger.charge q           -- monotone; survives rollback
  match <- fr.cursor.next q with
  | .exhausted =>
    -- parent frame restores ITS snapshot and advances its own cursor
    return .pop fr
  | .yielded cursor' =>
    -- out of budget: keep the continuation, resume after a fair turn
    return .suspend { fr with cursor := cursor' }
  | .alt a =>
    match <- transaction (runRule a fr.focus) with
    | .progress children =>
      let H <- (children ++ fr.siblings).filterM fun h =>
        return !(<- h.isAssigned)
      if H.isEmpty then
        return .candidate (<- instantiateRoot) -- goes to the gate
      let h := selectGoal H -- Section alg:select
      return .push { focus := h, siblings := H.erase h,
        saved := <- saveState, family := firstFamily h,
        cursor := start h, allowance := split fr.allowance }
    | .solved => -- as progress []
    | .blocked deps =>
      park fr.focus on deps
      if all siblings parked then return .suspend fr
      else return .push (frame for the next unparked sibling)
    | .rejected cert => record cert; loop -- next alternative
    | .unsupported k => return .handoff (k, fr) -- another lane owns k
    | .interrupted => throw interrupted -- never "rule failed"

```

The frame stack is the branch’s plan suffix  $\rho$  (Definition 4.2): replaying the `alt` choices of the stack against the origin regenerates the branch with fresh identifiers. P2’s portfolio loop (`successors`, `yielded`, `candidate`, `checkedPrune`, `failedAttempt`, `interrupted`) and P8’s (`partialPlan`, `completeTerm`, `checkedCounterexample`, `verified`, `suspended`, `finished`) are the same event algebra seen from the scheduler; the merged form above is P8’s frame with P2’s outcome names.

**Invariant 15.1** (Restore discipline). A frame restores its own snapshot before every alternative, not after a failure. A successful child leaves its state installed until the child is popped; a popped child’s parent restores. Mixing “restore on failure” and “child owns the successor state” is the source of stale assignments and is forbidden<sup>[P8]</sup>. The `rollback_probe` regression of P9 (assign a hole by `isDefEq`, restore, check the hole is again unassigned) is the unit test for this invariant.

### 15.1.3 Goal selection

The prototypes take the leftmost unassigned goal; the production selector is<sup>[P2,P3,P4,P6,P8]</sup>:

```

def selectGoal (H : List HoleId) : HoleId :=
  -- wake-ups reclassify blocked holes before selection
  let ready := H.filter fun h => (blockedOn h).isEmpty
  -- coupled clusters (SCCs of D) are scheduled as units
  let clusters := stronglyConnected D ready
  let c := argmax (fun c => pressure c + age c) clusters
  -- proof-first when a cheap proof field can determine a data field
  match c.find? (fun h => h.role = .proof && determinesDataHole h) with
  | some h => h
  | none => argmax pressure c -- witness-first fallback

```

**pressure** is the score of Section 6; P6’s form ( $w_1$  information gain +  $w_2$  dependents –  $w_3$  cost) and P4’s “most constrained eligible goal subject to dependency barriers” are the two inputs merged there. Readiness is “the variables the chosen rule needs are sufficiently instantiated”, not “no metavariable remains in the type”<sup>[P8]</sup>. A cyclic dependency is never diagnosed as inconsistency: the cluster is branched over as a unit until a constructor or type choice breaks the cycle<sup>[P2,P8]</sup>. Scores are written to the trace so that a composition failure can be attributed to scheduling rather than hidden behind “no term found”<sup>[P2]</sup>.

#### 15.1.4 Rule outcomes, the three blocked states, and wake-ups

`runRule` returns exactly one of `progress`, `solved`, `blocked(deps)`, `rejected(cert?)`, `unsupported(feature)`, `yielded(cursor, charged)`, `interrupted`<sup>[P2,P9]</sup>. The three non-failure, non-success states are kept distinct because the scheduler treats them differently<sup>[P2]</sup>: `blockedOnAssignments(deps)` is parked and woken by an assignment to any of `deps`; `unsupportedByLane(feature, k)` is routed with its continuation `k` to another lane; `yieldedForBudget(k, charged)` is resumed after a fair turn or a raised bound. None enters a negative cache.

Delayed constraints carry dependency stamps<sup>[P4,P5,P6,P8,P9]</sup>:

```
structure Delayed where
  c      : Constraint    -- flex-rigid eq / instance goal / index eq / contract
  deps   : MVarIdSet     -- metavariables whose assignment can change c
  stamp  : Version       -- assignment version at which c was last examined

def onAssign (m : MVarId) : WorkerM Unit := do
  for d in delayed do
    if m in d.deps && d.stamp < version m then requeue d -- exactly these
    -- wake-up events: a flex-rigid equation became a pattern; an instance
    -- goal's input parameters became ground; a contract reduced after
    -- substitution; recursor(?x, ..) = y with ?x now a constructor
```

A constraint is re-examined only when a dependency changed; P5 states the requirement (“retry after a relevant metavariable assignment, not after every unrelated step”) and P6 names the four wake-up events used above.

#### 15.1.5 Interleaving proof and data obligations

Proof obligations and data obligations live in the same frame stack; what differs is whether a proof rule may assign data holes<sup>[P1,P2,P4]</sup>.

```
def runProofRule (h : HoleId) (svc : ProofService) : WorkerM RuleOutcome := do
  match mode h with
  | .joint =>
    -- inside the branch transaction: assignments are a branch choice
    let (reply, patch) <- svc.run h.goal
    checkOwnership patch -- only branch-owned metavariables
    checkExecutable patch -- a proof may not pick a noncomputable witness
    return .progress patch.newObligations -- subject to the commit criterion
  | .frozen =>
    -- the candidate is closed; the service may not assign anything
    let closed <- closeOverTelescope h.candidate
    match <- svc.run (spec closed) with
    | .proved p => return .solved
    | .refuted q => return .rejected (some q) -- exactly this identity
    | .unknown r => return .yielded (retryLarger h) r.charged
```

In the prototypes, `MVarId.refl` on a goal  $?m = n + 1$  is a joint-mode rule: `isDefEq` assigns  $?m$ . P2 calls this “proof search may choose data inadvertently” and requires that it be a charged, transactional branch choice; that is exactly what running it under `advance` gives.

## 15.2 Introductions

```
def introRule (g : HoleId) : WorkerM RuleOutcome := g.withContext do
  let T <- whnfBudgeted (<- g.getType)    -- expose only the outer Pi; charged
  unless T.isForall do return .rejected none
  let (x, g') <- g.intro1P                -- keeps binder name and binderInfo
  return .progress [g']                  -- x is rigid w.r.t. earlier holes
```

`intro1P` preserves explicit, implicit, strict-implicit, and instance-implicit binders; the fresh local’s identity, not its printed name, enters the state<sup>[P1,P2,P3,P8,P9]</sup>. A universe or type parameter of the query introduced this way is rigid: the search may not specialize it to `Nat` to make an example executable<sup>[P3]</sup>. Introduction is existence-preserving, so the focused lane applies it eagerly; it is not size-preserving, so the exact lane below runs first when the profile rewards direct reuse<sup>[P3,P4,P5,P9]</sup>.

### 15.2.1 Exact-local lane and projections

```
def exactLane (g : HoleId) : WorkerM (List Alternative) := g.withContext do
  let mut alts := []
  for d in (<- getLCtx) do
    -- P2's initial run admitted these and produced self-reference cycles
    if d.isImplementationDetail then continue
    alts := alts ++ [transaction do
      unless <- isDefEq (<- inferType d.toExpr) (<- g.getType) do
        return .rejected none
      g.assign d.toExpr; return .solved]
    -- known-major projections over a local structure value (alg:proj)
    let S <- whnf (<- inferType d.toExpr)
    if let .const sName _ := S.getAppFn then
      if let some info := getStructureInfo? (<- getEnv) sName then
        for i in [:info.fieldNames.size] do
          alts := alts ++ [spineAlternative g (Expr.proj sName i d.toExpr)]
  return alts
```

P3’s prototype runs this lane before introduction (“exact local terms are tried before expanding applications”) and P2’s runs it before introduction too; the others introduce first. The design merges them: exact lane, then introduction, with the caveat of Section 7.2 that a cheap inhabitant never suppresses contract-sensitive alternatives, because the continuation, not the lane order, decides acceptance<sup>[P3,P5]</sup>.

### 15.2.2 Constructor application for inductive targets

```
def ctorRule (g : HoleId) : WorkerM (List Alternative) := g.withContext do
  let T <- whnf (<- g.getType)
  let .const I _ := T.getAppFn | return []
  let .inductInfo info <- getConstInfo I | return []
  -- ALL constructors are alternatives, never only the first
  return info.ctors.map fun c => transaction do
    let c' <- mkConstWithFreshMVarLevels c -- fresh levels per occurrence
    -- open the telescope; params are unified away by the result match
    match <- spineUnify c' T with
    | .mismatch => return .rejected none -- rigid index clash: 0 vs n + 1
```

```

| .blocked cs => delay cs; yield fields -- index eq blocked: keep c
| .progress  => yield fields           -- unassigned args, telescope order

```

Rejecting a constructor requires an inconsistent index equation; a blocked one keeps the constructor and records a delayed constraint, or is reported as a bounded omission, never as impossibility<sup>[P3,P6,P9]</sup>. Fields are the constructor’s actual telescope in order, including inherited and proof fields; a data field remains a choice point until every dependent field and the contract are satisfied<sup>[P1,P3]</sup>. In the prototypes this rule is literally `goal.apply (<- mkConstWithFreshMVarLevels ctor)` with the constructors read from `InductiveVal.ctors`<sup>[P1,P2,P5,P6,P7,P8,P9]</sup>.

### 15.3 Spine expansion

This is the central algorithm. Six proposals give pseudocode (P2 `planApplication`, P3 `applyHead`, P6 `expandApplication`, P7 `expandHead`, P8 `applyHead`, P9 Algorithm 2 SPINES); P9’s prefix-by-prefix loop is the most explicit and is taken as the spine, with P6’s per-prefix restore, P3’s cheap discharge step, and P7’s classification of the unification result merged in.

```

def spines (g : HoleId) (h : Head) (b : Nat) : WorkerM Unit := g.withContext do
-- 1. freshen: globals get fresh level metavariables per occurrence;
--    locals keep their exact fvar identity and universes
let h' <- match h with
| .global c  => mkConstWithFreshMVarLevels c
| .local d   => pure d.toExpr
| .proj s i d => pure (Expr.proj s i d.toExpr)
let mut e := h'
let mut U <- inferType h'
let mut args : Array HoleId := #[]
let base <- saveState
for j in [0:b+1] do -- admissible prefix lengths 0..b
  base.restore
  -- 2. this prefix: residual type against the target, own transaction
  match <- transaction (unifyClassified U (<- g.getType)) with
  | .mismatch    => pure () -- rigid clash: next prefix
  | .speculative => exploration.offer (e, args) -- approx: exploratory lane
  | .progress | .blocked cs =>
    -- 3. roles from binderInfo and sort:
    --    term | type | instance | proof | index | level
    let roles := args.map classify
    -- 4. discharge determined arguments cheaply
    let mut kids := []
    for a in args do
      if <- a.isAssigned then continue -- fixed by result unification
      match roles a with
      | .instance =>
        if inputParamsGround a then tryOnce (synthInstance a)
        else delay a
      | .proof =>
        -- bounded refl / assumption / decide; else it is a child
        unless cheapClose a do kids := kids ++ [a]
      | _ => kids := kids ++ [a]
    delayAll cs -- with dependency stamps
    -- 5. yield the application with its WHOLE coupled continuation
    yieldAlternative g e kids
  -- 6. extend by one binder, reducing U only enough to expose it
  U <- whnfCoreUntilPi U
  let .forallE _ dom body bi := U | break
  let a <- mkFreshHole dom g.lctx bi -- exact domain, binderInfo, owned
  args := args.push a
  e := mkApp e (mkMVar a)

```

```
U := body.instantiate1 (mkMVar a)
```

Steps and their justification:

1. *Freshen per occurrence.* Two occurrences of one global do not share level assignments merely because they print alike; a single occurrence shares its assignments across its whole spine; locals are never freshened, since freshening a local dictionary or universe turns a fixed choice into a new one<sup>[P1,P3,P4,P5,P6]</sup>.
2. *Prefixes.* Partial applications are admitted because the useful arity changes after reduction or when a type variable is instantiated to a function type, and because a partially applied head may be the demanded higher-order argument<sup>[P6,P8,P9]</sup>. Repeated use of one head is allowed  $(f(f(x)))$ <sup>[P8]</sup>.
3. *Telescope order with binder info.* Later argument types are instantiated with the actual earlier holes; there is no “types first, then values” split, because  $x$  may constrain  $A$  and the result may constrain  $B$  in  $\prod_A \prod_{x:A} \prod_{B:A \rightarrow \text{Sort}} \prod_{k:\prod_a B a} B x$ <sup>[P3,P4]</sup>.
4. *Classification of the unification outcome.* `unifyClassified` runs `isDefEq` transactionally and reports **progress** (assigned, nothing postponed), **blocked** (postponed flex-rigid equations, stuck instance goals, or a level constraint waiting on assignment), **speculative** (solved only by first-order approximation of a higher-order equation), **mismatch** (rigid head or constructor clash), or **unknown** on a budget exception, which is never negative evidence<sup>[P3,P5,P7,P9]</sup>. The fast lane accepts **progress** and **blocked**; the exploration lane also takes **speculative**<sup>[P6]</sup>.
5. *Cheap discharge.* Instance arguments whose input parameters are ground are synthesized once; proof arguments are tried with a bounded reflexivity/assumption/decision step; anything else becomes a child in the continuation. Determined arguments (assigned by step 4) never become goals<sup>[P3,P4,P8]</sup>.
6. *Yield with the continuation.* The alternative carries **kids**, and **advance** prepends them to the siblings; there is no path by which an argument is “solved independently”<sup>[P2,P6]</sup>.

### 15.3.1 What the prototypes did and what the production version adds

The prototypes are the first adapter: `MVarId.apply` already opens the telescope, matches the residual type at every arity, creates argument metavariables with the binder’s context, and processes instance arguments<sup>[P1,P2,P3,P6,P8]</sup>. With `newGoals := .all` the returned goals are the unassigned arguments in telescope order, so the witness of a dependent pair precedes its proof and witness-first backtracking is exercised<sup>[P6]</sup>; with `approx := false` no first-order approximation is silently committed, and with `synthAssignedInstances := false` an instance argument already fixed by unification is not re-synthesized and possibly replaced<sup>[P1,P3]</sup>. P3’s `allowSynthFailures := true` turns a failed instance synthesis into a retained instance goal instead of an exception. The production version adds, over `apply`: resumable prefix enumeration with per-prefix restore; explicit argument provenance (assigned by unification versus searched); the outcome classification of step 4; delayed constraints with stamps instead of `apply`’s postponed list; the cheap-discharge pass; local-versus-global freshening; and a differential test of any optimized planner against `apply` on a dedicated corpus, because neither an `apply` failure nor a timeout is evidence that the request is impossible<sup>[P2,P6,P9]</sup>.

## 15.4 Known-major projections

Applying the polymorphic constant `Sigma.snd` to an unknown pair leaves the family and the major premise unknown; the result-directed unifier need not reconstruct the pair<sup>[P1]</sup>. Projections are therefore enumerated only over *known* locals, as in `exactLane` above: read `getStructureInfo?` for the local’s weak-head type, form `Expr.proj S i d`, infer its exact dependent type (the field

type instantiated at  $\mathbf{d}$ , beta-reduced), and run it through `spines` as a head<sup>[P1,P5]</sup>. Repeated projections are explored at increasing cost, never as an eager closure over recursive structures<sup>[P1]</sup>.

Evidence: P1’s expanded prototype initially missed  $p : \Sigma_{n:\mathbb{N}} \text{Fin}(n+1) \vdash \text{Fin}(p.1+1)$ ; adding exactly this rule made it pass, and the term is `p.2`, whose inferred type  $(\lambda n. \text{Fin}(n+1)) p.1$  is definitionally  $\text{Fin}(p.1+1)$ . P5’s first prototype ordered constructors before projections and, on the two-dictionary witness  $\{n : \mathbb{N} \mid n = \text{second.value}\}$ , grew `Nat.succ` chains until the heartbeat limit; moving projections before constructor expansion (the order in `solve` of P5’s `NativeCore.lean`) made the file compile. P7 and P9 solve  $(s : \text{Sigma } B) \rightarrow B \ s.1$  through the local-spine rule instead, since `apply s.2` is not needed when the search reaches `Sigma.snd` through the pair’s own eliminator; the projection rule is the cheaper, generic route.

## 15.5 Type invention and higher-rank instantiation

### 15.5.1 The staged ladder

A type-valued hole (role `type`, sort `Sort ?u`) is attempted in stages of increasing cost, and only after the cheaper stage has been exhausted for the current bound<sup>[P1,P2,P6,P8,P9]</sup>:

```
def typeHole (a : HoleId) (level : Nat) : WorkerM (List Alternative) := do
  -- stage 0: forced by unification with the rigid target or later args
  if <- a.isAssigned then return []
  -- stage 1: the hole's own type is a Pi: introduce, do not enumerate
  if (<- whnf (<- a.getType)).isForall then return [introRule a]
  -- stage 2: bounded, cost-ordered type frontier
  let seeds := typesIn origin.telescope ++ subtypesOf origin.target
    ++ typesIn origin.contract ++ (<- localTypes)
    ++ providerDomains (retrieved a) ++ rigidTypeVarsInScope a
  let frontier := closeUnder seeds level -- grammar below, size <= level
  let mut alts := []
  for C in frontier do
    alts := alts ++ [transaction do
      let u <- mkFreshLevelMVar
      -- Lean checks the sort; u is solved by the level unifier
      unless <- isDefEq (<- inferType C) (mkSort u) do return .rejected none
      unless <- isDefEq (mkMVar a) C do return .rejected none
      return .solved]
  -- stage 3: quantified types and accumulator carriers, separate size cost
  if level >= carrierLevel then alts := alts ++ carrierAlternatives a level
  return alts
```

The frontier grammar at level  $\ell$  is

$$\tau ::= s \mid \tau_1 \rightarrow \tau_2 \mid \tau_1 \times \tau_2 \mid \tau_1 \oplus \tau_2 \mid \Sigma x:\tau_1. \tau_2 \mid \text{Option } \tau \mid F \vec{\tau} \mid \prod x:\tau_1. \tau_2,$$

with  $s$  a seed,  $F$  an admitted type constructor, the last production admitted only at stage 3, and  $\text{size}(\tau) \leq \ell$ <sup>[P1,P2,P4,P9]</sup>. A type proposed under a binder may mention the binder and may not escape it<sup>[P4,P8]</sup>. Every candidate is checked by Lean at the expected sort; the level  $u$  is a fresh metavariable solved by Lean’s level unifier, and named universe parameters of the query are never assigned<sup>[P1,P3,P4,P6]</sup>. Exhausting the frontier at level  $\ell$  is a missed instantiation, not impossibility<sup>[P6]</sup>. P4 additionally allows an arithmetic adapter or a bounded expression grammar to propose *term* indices for index-valued holes under the same ownership rules.

Stage 1 is why no rank restriction exists: for an argument of type  $\prod_{\alpha:\text{Type } u} \alpha \rightarrow \alpha$ , `introRule` opens  $\alpha$  and  $x : \alpha$  as rigid locals and the exact lane closes the body with  $x$ ; the callback is constructed inside its private scope and  $\alpha$  cannot escape into a hole created outside it<sup>[P2,P3,P5,P8]</sup>. All nine prototypes ran a rank-2 fixture this way (`higherRank`, `rankTwo`, `usePolymorphic`, `rankN`, `microRankN`, `polymorphicArgument`); P3, P5, P7, P9 also ran a two-universe variant

with independent `Type u` and `Type v` callbacks, which is a representative fixture and not the original two-universe Church regression of Leant<sup>[P1,P5]</sup>.

### 15.5.2 Bounded higher-order instantiation for flexible heads

A stuck equation  $?F a_1 \cdots a_n \doteq t$  that is not a pattern is first delayed; if its dependencies are all assigned and it is still stuck, the exploration lane instantiates  $?F$  from a bounded template family, each instantiation being an alternative rechecked by `isDefEq`<sup>[P3,P5,P7,P8]</sup>.

```
def flexTemplates (F : HoleId) (n : Nat) (t : Expr) (depth : Nat) :=
  projections n          -- fun x1 .. xn => xi
  ++ constants t         -- fun x1 .. xn => c, c a rigid subterm of t
  ++ imitations t n depth -- fun x1 .. xn => C (?G1 x1..xn) .. (?Gk x1..xn)
                        -- with C the rigid head of t
  ++ localApps n depth  -- fun x1 .. xn => xi (?G x1..xn)
```

There is no general higher-order unification; the templates are Lean expressions and Lean decides each one. P7 restricts the first implementation to a pattern-oriented policy plus delay, and places imitation/projection in a separate expensive lane; P3 and P5 name the same template set. A timed-out or stuck attempt is `unknown`, never a negative cache entry<sup>[P3,P7,P9]</sup>.

### 15.5.3 Universe policy

Levels are expressions, not kinds: retain maxima and parameters, let Lean generate and solve level constraints, and never lower a polymorphic query to `Type 0` and present the result as a solution of the original<sup>[P1,P4]</sup>. `Prop` is impredicative, `Type` is predicative and non-cumulative, so a polymorphic function type quantified over `Type u` is not itself an instantiation for a hole of sort `Type u`; P9's `universeNegative` control asserts exactly that failure. A lift is a typed construction, never a numeric relaxation<sup>[P3]</sup>. No fixed list of levels appears in the semantic kernel; a bounded list may appear only in the heuristic instantiation lane<sup>[P3]</sup>.

## 15.6 Instance and dictionary handling

```
def instanceArg (a : HoleId) : WorkerM RuleOutcome := do
  match origin.instanceMode with
  | .canonical => -- Lean's own resolution
    if !(<- inputParamsGround a) then
      return .blocked (inputParamMVars a) -- woken on assignment
    match <- transaction (synthInstance? (<- a.getType)) with
    | some inst => a.assign inst; return .solved
    | none      => return .rejected none -- none under frozen local instances
  | .explicitDictionary => -- a dictionary is data: a choice
    let candS := (<- localDictionaries (<- a.getType))
    ++ providerDictionaries (<- a.getType)
    return .alternatives (candS.map fun d => transaction do
      a.assign d
      recordDictionary a d -- payload enters identity and cache key
      return .solved)
```

The two modes never interfere: canonical mode follows the local-instance set, priorities, output-parameter behavior, and postponement of Lean's synthesizer, delayed until input parameters are known and woken by `onAssign`<sup>[P1,P2,P3,P6]</sup>; explicit mode enumerates dictionaries as ordinary providers, backtracks over them, stores the exact dictionary expression at each method occurrence, and emits it in the term even where a pretty-printer would omit it<sup>[P1,P5,P6,P8]</sup>. Two dictionaries of

one class type are never interchanged without a proved equivalence; only **Prop**-valued classes enjoy proof irrelevance; **Decidable** is data<sup>[P5,P6]</sup>. No prototype implemented canonical-mode search; P7's **fromInstance** and P8's **preserveDictionary** extract a field from a supplied instance by structural search, and the fixture in which the second of two same-typed dictionaries must be chosen because the contract says so ran in P1 (default 11 versus 29), P3 (**microDictionaryChoice**), P5 (**chooseDictionary**), and P9 (**exactDictionary**)<sup>[P1,P3,P5,P9]</sup>. Leant2 never manufactures a global instance to solve a local hole<sup>[P7]</sup>.

## 15.7 Worked traces

### 15.7.1 Witness-first backtracking through the continuation

The query is P9's **chooseSecond**,

$$\{ f : \forall A:\text{Type}, A \rightarrow A \rightarrow A \mid f \mathbb{N} 11\ 29 = 29 \},$$

run by P9's **search** (order: **refl**, **intro**, local spines, constructors, cases; **newGoals** := **.all**; fuel 80). The same query with  $f : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$  ran in P3 (**microChooseSecond**) with the same outcome, and the  $A \rightarrow A \rightarrow A$  form is the motivating example of P1. Each line is one **advance**; indentation is the frame stack.

```

g0 : {f // f Nat 11 29 = 29}                                ctx: empty
refl g0              -> mismatch (not an Eq)
whnf: Subtype _, no Pi -> intro inapplicable
locals: none
ctor Subtype.mk.{?u} -> ?u := 2, ?alpha := (forall A, A -> A -> A),
                        ?p := fun f => f Nat 11 29 = 29
                        goals (.all, telescope order):
                          [?val, ?prop : ?val Nat 11 29 = 29]
?val : forall A : Type, A -> A -> A                        siblings [?prop]
refl ?val            -> mismatch
intro1P              -> A : Type, ?b1 : A -> A -> A
intro1P              -> a : A,    ?b2 : A -> A
intro1P              -> b : A,    ?b3 : A      siblings [?prop]
refl ?b3             -> mismatch
local A : Type -> mismatch (Type vs A)                    restore
local a : A   -> progress []; ?b3 := a; ?val := fun A a b => a
?prop : (fun A a b => a) Nat 11 29 = 29  ctx: empty (from g0)
refl ?prop          -> whnf lhs = 11; isDefEq 11 29 fails  restore
intro               -> inapplicable
locals: none (A, a, b are not in ?prop's context)
ctor Eq.refl -> ?x := 11; 11 =?= 29 fails                  restore
cases: no locals -> frame exhausted, pop
restore frame(?b3).saved
  -- undoes ?b3 := a and the delayed assignment of ?val
local b : A   -> progress []; ?b3 := b; ?val := fun A a b => b
?prop : (fun A a b => b) Nat 11 29 = 29
refl ?prop      -> whnf lhs = 29; isDefEq 29 29;
                  ?prop := Eq.refl 29; solved
siblings empty -> candidate
result: Subtype.mk (fun A a b => b) (Eq.refl 29)
checked: #eval chooseSecond.val Nat 11 29 = 29
theorem chooseSecond_all A a b : chooseSecond.val A a b = b := rfl

```

The decisive step is the restore of **frame(?b3).saved**: because **?prop** was in **?b3**'s continuation, the failure of  $11 = 29$  unwinds the witness rather than being reported as an unrelated impossible subproblem. Under **nonDependentFirst** (P4, P8) the same query is scheduled proof-first: **?prop** comes first, **refl** must solve  $?val \mathbb{N} 11\ 29 \doteq 29$ , which is a non-pattern flex-rigid equation, so it is delayed; the witness is then searched and the delayed equation is woken at each assignment

of `?val`. Both orders are legitimate; the selector of Section 15.1.3 prefers proof-first when the proof field is a pattern equation that determines the data hole directly (as in  $\{n : \mathbb{N} \mid n = 1\}$ , where `refl` assigns  $?n := 1$  in P4’s `chooseOne` and P8’s `double`) and falls back to witness-first otherwise<sup>[P4,P6,P8]</sup>.

### 15.7.2 Spine expansion with a dependent provider

The query is P3’s `microIndexed`: context  $A : \text{Type } u$ ,  $n : \mathbb{N}$ ,  $x : A$ ,  $xs : \text{IndexedVec } A \ n$ , target  $\text{IndexedVec } A \ (n + 1)$ , with the provider

$$\text{cons} : \{n : \mathbb{N}\} \rightarrow A \rightarrow \text{IndexedVec } A \ n \rightarrow \text{IndexedVec } A \ (n + 1).$$

P3’s order is exact lane, intro, constructors and providers, local spines; fuel iterates upward from 1.

```
g0 : IndexedVec A (n + 1)
exact lane: A : Type u -> isDefEq (Type u) (IndexedVec A (n+1)) fails
          n : Nat      -> fails
          x : A        -> fails
          xs : IndexedVec A n
                                -> isDefEq: n == n + 1, both rigid -> mismatch
whnf: not a Pi                  -> intro inapplicable
ctor IndexedVec.nil.{?u}
  result IndexedVec ?A 0; ?u := u, ?A := A; 0 == n + 1
  -> Nat.zero vs Nat.succ n: rigid mismatch; rejected; restore
ctor IndexedVec.cons.{?u}
  spine prefix 4 (apply picks the saturating prefix):
    args [?A : Type ?u, ?n : Nat, ?head : ?A, ?tail : IndexedVec ?A ?n]
    residual IndexedVec ?A (?n + 1) == IndexedVec A (n + 1)
    -> ?u := u, ?A := A, ?n + 1 == n + 1 -> ?n := n      (progress)
    roles: ?A type (determined), ?n index (determined),
           ?head term, ?tail term
    children (.all, unassigned, telescope order):
      [?head : A, ?tail : IndexedVec A n]
  ?head : A                      siblings [?tail]
  exact lane: A fails; n fails; x : A -> isDefEq A A; ?head := x
  ?tail : IndexedVec A n          siblings []
  exact lane: A, n, x fail; xs : IndexedVec A n -> ?tail := xs
  siblings empty; root has no metavariables -> candidate at fuel 3
result: @IndexedVec.cons.{u} A n x xs
```

Three features of Section 15.3 are visible: the level `?u` and the implicit index `?n` are determined by result unification and never become goals (provenance: assigned by unification); `nil` is rejected by a rigid constructor clash without any argument search; and `?tail`’s type mentions the determined `?n`, so choosing a different vector would have re-constrained the index. The designed-but-not-run extension is P4’s

$$\text{get} : (n : \mathbb{N}) \rightarrow \text{Vec } A \ n \rightarrow \text{Fin } n \rightarrow A$$

at target  $A$  with locals  $v : \text{Vec } A \ 3$  and  $i : \text{Fin } 3$ : the residual  $A \doteq A$  is **progress** at prefix 3 with `?n` still unassigned, so `?n` is classified as an index hole and parked; choosing  $v$  for the vector argument assigns  $?n := 3$  through `isDefEq`, `onAssign` wakes the parked hole, and the `Fin ?n` argument is now demanded at `Fin 3`, which the exact lane closes with  $i$ . A backend that synthesized “some vector” and “some bounded natural” separately would have to repair the mismatch afterwards, which is what native spine expansion exists to avoid<sup>[P4]</sup>.

## 16 Algorithms: dependent elimination, retrieval, and carriers

Sections 8, 9, and 9.4 state what the engine does; this section states how, as procedures over the branch state of Section 4.3. Every procedure below runs inside one transaction of Section 5, assigns only branch-owned metavariables (Invariant 4.3), and returns its new obligations to the scheduler of Section 6 rather than solving them itself; the construction rules that consume those obligations are in Section 15. Pseudocode uses the proposals' own names where they had them. Where a procedure was only designed, the text says so; the only parts that ran are the `cases` calls in the nine prototypes and the hand-written `indexFold` check in P8.

### 16.1 The case-analysis transaction

#### 16.1.1 What the wrapper returns

Every prototype that split a local called `MVarId.cases` and kept only the subgoal identifiers (`branches.toList.map (.mvarId)`), discarding the `fields` and `subst` components of each `CasesSubgoal`<sup>[P1,P2,P5,P6,P7,P8,P9]</sup>. That is sufficient for a bounded existence search and insufficient for Leant2, because contracts, observations, and recursive capabilities must follow the same branch substitution as the goal<sup>[P2,P3]</sup>. The production wrapper therefore returns a record, not a goal list<sup>[P3,P4,P6]</sup>:

```

CaseResult := {
  scrutinee   : FVarId,           -- x : I p i
  closure     : List FVarId,      -- dependents generalized with x
  motive      : Expr,            -- checked at the eliminator's sort
  branches    : List CaseBranch,
  discharged  : List DischargeRecord, -- impossible branches, with provenance
  reconstruct : List (BranchTerm) -> Expr -- eliminator application
}
CaseBranch := {
  ctor      : Name,
  goal      : MVarId,           -- branch context already reintroduced
  fields    : List FVarId,      -- constructor fields, telescope order
  indexEqs  : List (FVarId x Expr), -- surviving index equations, if any
  subst     : FVarSubst,        -- parent local -> branch local/term
  transports : List MVarId      -- equality obligations created by step 5
}
DischargeRecord := (ctor, evidence : kernelChecked p | trustedFragment thm | heuristicOmission)

```

`subst` is the branch-to-parent map that the contract translator, the observation bank, and the capability table apply; `transports` are goals of the form  $i = j$  that the wrapper could not close by unification and that Section 16.2 owns. Only `kernelChecked` and `trustedFragment` records support a coverage claim<sup>[P4]</sup>.

#### 16.1.2 Scrutinee selection

The prototypes disagreed on which locals may be split: P1, P2, P8, and P9 split any local whose weak-head type is an inductive application; P5 excludes `Nat`; P6 splits only heads on a whitelist; P7 splits only non-recursive, non-indexed inductives and marks each split local as used<sup>[P1,P2,P5,P6,P7,P8,P9]</sup>. All of them bounded the number of splits per path (`splits`). The unified procedure keeps the bound and replaces the whitelists by a score; a low score delays a split, it never forbids it<sup>[P2,P3,P7]</sup>.

```

scoreScrutinee(x, branch):
  ty := whnfR (typeOf x); require ty is an inductive application I p i

```

```

s := 0
if x or one of its indices occurs in an index of the target then s += 3
if a contract clause or observation discriminates on x then s += 3
if x blocks weak-head reduction of a term the target needs then s += 2
if x is Prop-valued and the target sort forbids its eliminator then return none
if I is recursive and no recursion plan covers x then s -= 2
if splitHistory(branch) already contains the origin of x then s -= 3
if I has no indices and one constructor then s -= 1
return s

```

Split history is keyed by discriminant origin, not by the current `FVarId`: splitting `xs` produces a tail whose origin is still `xs`, so a second split is recognized as bounded unrolling of one input, which is legitimate for finite cases and never a substitute for a recursion plan (Section 10)<sup>[P2,P5,P9]</sup>.

### 16.1.3 The transaction

P3 and P4 give the four-step plan and P5 the same plan with generalization made explicit; P6 adds the requirement that the substitution map be preserved per branch. Merged<sup>[P3,P4,P5,P6,P8]</sup>:

```

caseSplit(branch, x):
  -- 1. dependency closure of x and its indices
  (I, params, idx) := decompose (whnfR (typeOf x))
  closure := []
  for y in localsAfter x in telescope order:
    if typeOf y or valueOf y mentions x, any of idx, or any member of closure:
      closure := closure ++ [y]
  -- 2. generalize / revert
  -- fresh index variables j with equations h : j = idx, when idx are not variables
  (goal1, j, h) := generalizeIndices(branch.goal, x) -- Lean: generalizeIndices
  goal2 := revert(goal1, closure) -- Lean: MVarId.revert
  -- 3. eliminator and motive
  elim := casesOnOf I; require sortOf(target goal2) admissible for elim (alg:elim-restrictions)
  motive := fun j x => Pi closure. target goal2 -- checked by inferType
  -- 4. one branch per constructor
  for c in constructorsOf I:
    g := instantiate elim with motive, params, c -- Lean: MVarId.induction / casesOn
    fields := intro (fieldsOf c) in g
    eqs := index equations c.resultIndices(fields) = j
    r := unifyEqs(g, eqs) -- Lean: unifyEqs / subst
    match r with
    | contradiction p => discharged += (c, kernelChecked p); continue
    | solved (g', sigma) => reintroduce closure in g' with types under sigma
                          branches += {c, g', fields, residualEqs r, sigma ++ h, []}
    | stuck eq => branches += {c, g', fields, [eq], sigma, [mkTransportGoal eq]}
  -- 5. reconstruction
  reconstruct := fun terms => elim motive params (fun fields => terms c) ... x
  return CaseResult{x, closure, motive, branches, discharged, reconstruct}

```

The bracketed Lean names are the operations the prototypes used or that `MVarId.cases` composes internally on Lean 4.34.0: `generalizeIndices`, revert of dependents, the `casesOn` application, `unifyEqs` for the constructor index equations, and reintroduction through `FVarSubst`<sup>[P1,P5,P6,P8]</sup>. The first implementation calls `MVarId.cases` and reads back `fields` and `subst` instead of reimplementing steps 2 to 4; a custom fast path is admitted only against differential tests over that wrapper<sup>[P5,P8]</sup>. If step 3 yields a motive whose generalization exceeds a size bound, the rule declines or enumerates a bounded alternative closure; generalizing less than the closure is never admissible<sup>[P4]</sup>.

### 16.1.4 Impossible-branch discharge

A branch is removed only by evidence, in this order<sup>[P1,P4,P5,P8]</sup>:

```

discharge(c, eqs, g):
  if unifyEqs finds a constructor clash    (e.g. 0 = n + 1)  -> kernelChecked (noConfusion
    proof)
  else if unifyEqs finds an occurs-check cycle          -> kernelChecked (structural proof
    )
  else if a registered exact-fragment procedure (omega on Nat, decide on finite types)
    proves eqs -> False in the branch context          -> trustedFragment (its theorem)
  else if a bounded local contradiction search closes False -> kernelChecked (the proof)
  else
    impossible                                           -> keep the branch; it is not

```

Heuristic unification failure, an opaque index expression that does not reduce, or an SMT answer about an approximation of the indices never removes a branch; if the lane still drops it for cost, the record says `heuristicOmission` and the result cannot claim coverage<sup>[P4,P5,P8]</sup>. Every prototype that split an indexed family observed `propext` in the resulting term's axiom inventory; the receipt reports it (Section 13)<sup>[P2,P4,P7,P8,P9]</sup>.

## 16.2 Equality repair and transport

An index mismatch is a construction problem with an explicit search action, not a normalization step<sup>[P3,P4,P6]</sup>. The unified order is P3's repair rule with P6's example and P4's equality graph:

```

repairEquality(e : S, target T, branch):
  -- 1. definitional, on the transparency ladder, budgeted and charged
  for tr in [reducible, instances, default]:
    if isDefEq(S, T) under tr within budget: return e    (record tr if stronger than reducible)
  -- 2. registered proof-producing normalizers (simp sets, arithmetic normal forms)
  for nrm in registeredNormalizers:
    if nrm proves q : S = T within budget: return castAlong(q, e)
  -- 3. common family and index equation
  (F, i, j) := commonFamily(S, T)    -- S == F i, T == F j, with i, j rigid-headed
  if none: return blocked(S, T)      -- suspended constraint, watchers on its mvars
  q := newGoal (i = j) role := indexEquality    -- proved by the proof portfolio
  motive := fun k => F k                -- checked by inferType at the recursor's sort
  return Eq.rec (motive := motive) e q    -- transport; never a raw cast

```

For  $v : \text{Vec } A(n + m)$  against  $\text{Vec } A(m + n)$ : step 1 fails at every transparency because addition on open naturals does not reduce; step 2 succeeds if an arithmetic normalizer is registered; otherwise step 3 finds  $F = \text{Vec } A$ ,  $i = n + m$ ,  $j = m + n$ , opens  $q : n + m = m + n$  (closed by `Nat.add_comm` or `omega`), and returns the recursor application. The index is never erased and  $n + m$  is never treated as definitionally  $m + n$ <sup>[P3,P6,P8]</sup>. Each transport adds one unit to the `casts` component of the cost vector, so a differently oriented construction with no transport outranks it; the simplification phase later tries to remove casts by checked equalities, and failure to simplify is not evidence of ill-typing<sup>[P2,P4]</sup>.

Transport is also an elimination action on locals: from  $h : a = b$  and  $x : \beta(a)$ , propose  $\beta(b)$  in either orientation of  $h$ <sup>[P4,P5]</sup>. To find  $h$  the branch keeps an *equality graph* over index and dependent-argument expressions whose edges carry proofs, not union-find representatives; a path is used only by inserting its composed proof into a transport; congruence closure may propose a path, and dependent congruence that would need heterogeneous equality returns unknown in the first implementation<sup>[P4]</sup>. P3 additionally routes arithmetic index equations to solvers whose output must be a proof in the actual local context with the source operation's semantics (truncated subtraction, natural division).

### 16.3 Elimination restrictions

The admissibility test in step 3 of `caseSplit` reads the recursor, never a table of type names<sup>[P4,P7,P8]</sup>:

```
admissibleElim(I, targetSort, profile):
  rec := recursorOf I
  if rec eliminates into targetSort (large elimination or targetSort = Prop): return ok
  -- I is a Prop with restricted elimination and the target is data
  if I = False: return ok -- False.elim into any sort
  if I = Eq or I is a singleton Prop: return ok -- Lean grants these
  if I = Nonempty or I = Exists:
    if profile allows Classical.choice: return ok(labelled noncomputable)
    else return refuse("proof-to-data extraction")
  return refuse(I, targetSort)
quotientElim(q : Quot r, target):
  create holes f : A -> target and respect : forall a b, r a b -> f a = f b
  return Quot.lift f respect q -- both holes coupled, never a representative
decidableIf(c : Prop, branch):
  inst := instance obligation Decidable c -- delayed until c is determined
  then-goal under h : c, else-goal under h : not c
  contract clauses split under the same h; a Bool guard must be linked to c by a proved iff
```

P7's prototype keeps a negative control, `fail_if_success have : Nonempty A -> A`, showing that the native recursor check rejects the extraction without a special rule; the same file accepts `False-elimination`<sup>[P7]</sup>. Quotient outputs go through registered lift components with their respect theorems; searching for representatives is a different, usually noncomputable request<sup>[P3,P4,P8]</sup>. A condition synthesized for an `if` has its own proof and execution obligations; an SMT predicate with no Lean interpretation cannot become a branch assumption<sup>[P2]</sup>.

### 16.4 Worked elimination traces

**Indexed constructor (P4).** Target  $A \rightarrow \text{Vec } A \, n \rightarrow \text{Vec } A \, (n + 1)$ . `Pi` introduction exposes  $a : A$  and  $xs : \text{Vec } A \, n$ . The result-head key  $(\text{Vec}, 2)$  retrieves the two constructors. `nil` : unifying  $\text{Vec } A \, 0$  with  $\text{Vec } A \, (n + 1)$  is a rigid clash, rejected in its transaction. `cons` : the spine creates  $?k : \mathbb{N}$ ,  $?h : A$ ,  $?t : \text{Vec } A \, ?k$  and unifies  $\text{Vec } A \, (?k + 1)$  with the target, assigning  $?k := n$  by injectivity of the successor; the exact-local lane solves  $?h := a$  and  $?t := xs$  in the same continuation. This ran in P4's `vcons` and P2's `vectorTail`<sup>[P2,P4]</sup>.

**Indexed elimination (P2, P5, P8, P9).** Target  $\text{Vec } A \, (n + 1) \rightarrow A$ . After introduction  $v : \text{Vec } A \, (n + 1)$ ; `scoreScrutinee`:  $v$  is the only inductive local, the target is not indexed by it, but nothing else applies, so it is split. Step 1: closure is empty. Step 2: `generalizeIndices` introduces  $j : \mathbb{N}$ ,  $h : j = n + 1$  and retypes  $v : \text{Vec } A \, j$ . Step 3: motive  $\lambda j v. A$  (the target does not depend on  $v$ , so the motive is constant; the eliminator still refines  $j$ ). Step 4, `nil`: equation  $0 = n + 1$ ; `unifyEqs` closes the branch by `Nat.noConfusion`; record `(nil, kernelChecked)`. Step 4, `cons`: fields  $k : \mathbb{N}$ ,  $a : A$ ,  $t : \text{Vec } A \, k$ ; equation  $k + 1 = n + 1$  solved to  $k := n$  by injectivity; branch goal  $A$  with  $a$  in context; the exact-local lane closes it. Step 5 rebuilds the `casesOn` application; the term's axiom inventory contains `propext`, contributed by the equation elimination<sup>[P2,P5,P8,P9]</sup>. No conversion to an unindexed list occurs.

**Elimination inside a recursion schema (P4, P6).** Target  $(A \rightarrow B) \rightarrow \text{Vec } A \, n \rightarrow \text{Vec } B \, n$  with  $n$  abstract: constructors alone cannot decide between `nil` and `cons` because  $n$  is rigid, so the recursion lane owns the split. With motive  $M(n, xs) = \text{Vec } B \, n$  (Section 10), the `nil` branch refines  $n := 0$  and `nil` becomes admissible; the `cons` branch refines  $n := k + 1$ , exposes  $x : A$ ,

$xs : \text{Vec } A \ k$ ,  $ih : \text{Vec } B \ k$ , and the constructor lane reduces the goal to a  $B$  (solved by  $f \ x$ ) and a tail (solved by  $ih$ )<sup>[P4,P6]</sup>. P2's and P4's prototypes filled these branches inside a hand-written skeleton; P9 reports that the first attempt, an `induction` tactic inside an executable `def`, was rejected by the code generator until the plan was lowered to pattern equations (Section 10).

**Dependent lookup (P8).** Target  $\Pi n. \text{Vec } A \ n \rightarrow \text{Fin } n \rightarrow A$ . The index argument is consumed after the traversal, so the motive is  $M(n, xs) = \text{Fin } n \rightarrow A$ , not  $A$ . In the `nil` branch the reintroduced index has type  $\text{Fin } 0$  and is discharged by splitting it: its bound  $i < 0$  is a checked contradiction. In the `cons` branch a second split, now on the index, gives zero (return the head) and successor (apply the recursive result to the smaller  $\text{Fin } k$ ); the index conversion is a transport obligation of Section 16.2, not an erased annotation. Designed only; the vector-head prototype validates the elimination step, not this trace<sup>[P2,P8]</sup>.

## 16.5 Provider index construction and retrieval

### 16.5.1 Building the index

P1, P5, P8, and P9 converge on a discrimination tree over normalized result shapes with explicit wildcard buckets; P6 adds the recipe map for local components; P7 adds the typed forward DAG. Merged<sup>[P1,P5,P6,P7,P8,P9]</sup>:

```
buildIndex(env, generation):
  idx := DiscrTree.empty; wild := []; reducible := DiscrTree.empty
  for decl in env, decl admitted by the provider policy:
    ty := decl.type; tele := piTelescope ty
    resHead := whnfR (result tele) -- reducible transparency only
    key := (head resHead, arity tele, argHeadPattern resHead,
            role decl (ctor | proj | thm | def | recursor | lift),
            isProp (result tele), universeShape decl, binderDeps tele)
    entry := {name, levelParams, ty, key, visibility, computable?, specEntries decl, provenance}
    if head resHead is a bound variable or metavariable: wild += entry
    else idx.insert (pattern resHead) entry
    if head resHead unfolds under reducible to a different head:
      reducible.insert (pattern (whnfR' resHead)) entry -- alias bucket
    if decl is a structure: register projections and constructors explicitly
  return Index{idx, wild, reducible, generation}
```

The key deliberately omits detail: a hit is a proposal, and every hit is instantiated with fresh universe levels and matched natively before it becomes a search action, so the index may over-return; it must never silently under-return, so the wildcard and reducible buckets are always consulted at the widening tier<sup>[P3,P7,P9]</sup>. Locals, projections of known locals, and constructors of the goal's own family form a separate small inventory searched first<sup>[P1,P5,P7]</sup>. Specification entries are stored with the declaration they justify and are instantiated only at the same occurrence<sup>[P5,P8]</sup>.

### 16.5.2 Retrieval tiers and widening

```
retrieve(goal, branch, tier):
  T := whnfR (typeOf goal); pat := pattern (resultOf T)
  match tier with
  | 0 => locals(branch) ++ projectionsOfKnownLocals ++ localDictionaries
  | 1 => constructorsOf (headOf pat) ++ projectionsOf (headOf pat)
  | 2 => explicitlyRequestedProviders ++ registeredComponents ++ synthesisLemmas
  | 3 => idx.getMatch pat ++ reducible.getMatch pat -- nearby namespaces first
  | 4 => idx.getUnify pat ++ wild -- flexible heads
```

```
| 5 => budgetedBroadScan(env) -- charged, resumable
record (tier, cap) in the branch's restriction trail
return candidates ranked by (tier, key distance, registered cost, learned relevance)
```

A lane consumes tiers in order and widens on exhaustion; an exhausted tier or a cap such as fifty providers is a restriction recorded in the result, never an `impossible` verdict<sup>[P1,P3,P9]</sup>. Learned premise ranking may reorder within a tier and must operate only on accessible declarations<sup>[P1]</sup>.

### 16.5.3 Typed abstract graph refinement

For large inventories P4 and P9 adopt TYGAR-style abstraction as an accelerator, P8 as the coarse layer of the demand graph<sup>[P4,P8,P9]</sup>:

```
abstractCompose(goalType, inventory, k):
  A := abstraction with nodes = (rigid head, bounded argument pattern), edges = providers
  loop up to k times:
    path := shortestPath(A, abs(availableTypes), abs(goalType))
    if none: return noProposal -- advisory only, never negative evidence
    t := elaborate path natively in the branch (spines of alg:search, transports of alg:
    transport)
    if t typechecks: return proposal t
    d := explain mismatch (index Vec A 0 vs Vec A (n+1), universe, dictionary identity)
    A := refine A by the distinction d -- split one node; never ban the provider
  return noProposal
```

The abstraction keeps nominal heads, index dependencies, universe constraints, and dictionary identities as labels so that a failed path yields a concrete refinement; the exact branch stays authoritative and no completeness theorem is claimed for the abstract graph<sup>[P4,P9]</sup>.

## 16.6 Component recipes, saturation, and the join

### 16.6.1 Recipes and telescope substitution

A solved subterm is stored as a recipe over exactly the locals it depends on<sup>[P6,P7,P9]</sup>:

```
storeRecipe(e, branch):
  Delta := dependencyClosedTelescope(fvarsOf e ++ fvarsOf (typeOf e)) -- keeps let values
  sigma := abstract (typeOf e) over Delta; body := abstract e over Delta
  return Recipe{Delta, body, sigma, contracts := lemmasProvedAbout e (also abstracted),
    cost := size body, provenance := branch.decisionTrail, generation}
reuseRecipe(r, goal, Gamma):
  freshen universe variables of r
  for rho in telescopeSubstitutions(r.Delta, Gamma): -- exact, typed, in order
    e' := instantiate r.body rho; s' := instantiate r.sigma rho
    if isDefEq(s', typeOf goal) and check e' : s' in Gamma:
      contracts' := instantiate r.contracts rho -- the SAME rho
  yield (e', contracts')
```

A name match or a matching pretty-printed type is never enough; the substitution is applied uniformly to term, type, and contracts and the result is rechecked in  $\Gamma$ <sup>[P6]</sup>. A recipe never contains a stale `FVarId`<sup>[P7,P9]</sup>.

### 16.6.2 Forward saturation and the backward demand graph

```
forwardSaturate(branch, bound):
  dag := branch.forwardDag; queue := locals(branch)
```

```

while queue nonempty and charged work < bound:
  v := pop queue
  for d in [projections of v, constructor equations of v, registered forward rules applied to v]:
    key := if isProp (typeOf d) then typeOf d else (d, typeOf d)  -- data keyed by expression
    if key not in dag: dag.add key (derivation d); push queue d
  return dag
demandGraph(branch):
  G := nodes := targetHoles ++ observationDemands
  for n in G unexplored:
    for p in retrieve(n, branch, currentTier):
      add hyperedge argumentDemands(p, n) --p--> n  -- each node carries telescope,
                                                    -- assignments, instances, scope

  seed solved demands from forwardSaturate
  return G
join(G, dag):
  for hyperedge (args --p--> n) with every arg matched by a dag entry or a recipe:
    instantiate p with the matched entries in one transaction
    propagate the resulting type/instance assignments across the whole application
    yield the application with its continuation

```

Two demands with different vector lengths or different dictionary payloads are distinct nodes unless an exact transport identifies them<sup>[P6,P8]</sup>. Saturation is bounded and demand-driven; it improves the inventory, it does not close the imported environment under all theorems<sup>[P2,P9]</sup>.

### 16.6.3 Candidate context slices

```

contextSlice(demand n, branch, depth):
  S := locals in typeOf n ++ locals in contract(n) ++ locals in pendingConstraints(n)
  repeat depth times: S := S ++ locals that a retrievable provider can turn into a member of S
  S := dependencyClosure S
  return S
solveSliced(n, branch):
  r := search n under context contextSlice(n)  -- high-priority lane
  if r = solution e: e' := weaken e into the full context via an explicit renaming map
    recheck e' : typeOf n in the full context; return e'
  else: continue in the full-context lane  -- slice failure is not evidence

```

This is the direct algorithmic answer to Leant’s indexing diagnostic, in which the handler was found standalone but not amid the outer list and index variables<sup>[P1,P6,P7]</sup>.

## 16.7 Carrier invention

### 16.7.1 Grammar and demand rules

The carrier grammar is P8’s, seeded by types already demanded; P1’s list  $R, I \rightarrow R, S \times R$ , invariant-carrying pairs is its first cost layer<sup>[P1,P8]</sup>:

$$C ::= R \mid D \rightarrow C \mid C_1 \times C_2 \mid \text{Option } C \mid \Sigma d:D, C(d),$$

where  $R$  and  $D$  range over types already present in the telescope, the target, the contract, and the domains of retrieved providers; nothing is drawn from unrelated global types.

```

proposeCarriers(eliminator E, goal, branch):
  R := result type demanded after E is applied
  rest := binders of the goal telescope consumed AFTER the traversal of E’s major premise
  seeds := [R]

```

```

-- parameter postponement (P8; "backward demand" in P1, "remaining telescope" in P5)
for q in rest, in original order: seeds += [typeof q -> R]
seeds += [Pi rest. R] -- all remaining arguments at once
-- result pairing (P8; "two mutually needed summaries" in P4)
for (s1, s2) in summaries demanded by the contract of the recursive step: seeds += [s1 x s2]
-- accumulator generalization (P8; "behavioral generalization" in P5)
for a in arguments whose value differs between a call and its recursive call: seeds += [
  typeof a -> R]
-- invariant-carrying carriers (P1, P5, P9)
if a pointwise contract Q is available: seeds += [{r : R // Q r}] and dependent variants
return enumerate seeds by increasing structural cost, each checked at E's carrier sort

```

Backward-demand detection is the trigger: the rule fires when, after choosing  $E$ , the continuation still consumes an argument  $I$ ; then  $I \rightarrow R$  is tried before forcing  $E$  to produce  $R$  directly<sup>[P1,P5,P9]</sup>. Generalization is behavioral as well as typal: P5's policy first reabstracts all remaining non-fixed arguments in their original order, then tries a small family of subsets suggested by the contract and by candidate recursive calls, always adding mandatory type dependencies; binder provenance is kept after eager introduction so that an already-introduced index can be reabstracted<sup>[P5]</sup>. Carriers are never guessed from unrelated global types<sup>[P5,P9]</sup>.

### 16.7.2 The carrier frontier

P1's Algorithm 5, with P6's recipe reuse and P8's cost accounting<sup>[P1,P6,P8]</sup>:

```

carrierFrontier(E, goal, branch):
  for C in proposeCarriers(E, goal, branch): -- increasing cost, fair
    key := (fullContext(branch), motive C, E, bound)
    if failMemo contains key: continue -- memoized only under the full key
    t := transaction:
      instantiate E at carrier C -- base z : C, step s : fields -> C
    -> C
    base, step := coupled holes; outer := continuation applying the fold result to rest
    solve (base, step, outer) jointly (alg:search) -- never base-then-step-then-outer
    if t succeeded: yield t -- alternatives stay on the frontier
    else failMemo += key
    -- reduced-context attempt (P1): a step solved in its small telescope becomes a recipe
    if step solvable in contextSlice(step): storeRecipe(step); retry reuseRecipe in full
    context

```

A failure in the reduced context never prunes the full context, and a success there is not a solution of the whole query until the outer continuation and the contract are closed<sup>[P1]</sup>. Cost is a vector, with carrier complexity separate from program size; the first accepted carrier is not claimed minimal<sup>[P1,P7]</sup>.

### 16.7.3 Worked trace: list indexing as a right fold

Request  $\text{List } A \rightarrow \mathbb{N} \rightarrow \text{Option } A$  with the relational lookup specification ( $\text{lookup } [] i = \text{none}$ ;  $\text{lookup } (x :: xs) 0 = \text{some } x$ ;  $\text{lookup } (x :: xs) (k + 1) = \text{lookup } xs k$ ). Merged from P8's trace, P6's recipe step, and P5's template<sup>[P1,P4,P5,P6,P7,P8,P9]</sup>:

1. Introduction exposes  $xs : \text{List } A$  and  $i : \mathbb{N}$ ; the target is  $\text{Option } A$ . Retrieval tier 2 returns  $\text{List.foldr} : (A \rightarrow C \rightarrow C) \rightarrow C \rightarrow \text{List } A \rightarrow C$  with a flexible carrier  $?C$ .
2. Spine expansion on  $\text{foldr}$  with major  $xs$  leaves  $i$  consumed after the traversal: backward demand fires.  $\text{proposeCarriers}$  yields, in order,  $C_0 = \text{Option } A$  and  $C_1 = \mathbb{N} \rightarrow \text{Option } A$  (parameter postponement of  $i$ ).

3.  $C_0$  fails: the step  $A \rightarrow \text{Option } A \rightarrow \text{Option } A$  cannot depend on  $i$ , and the outer continuation must still consume  $i$  from a result that ignores it; the cons/successor clause of the contract is unprovable for every step, which the proof lane reports as a failed obligation, not as impossibility. The key is memoized.
4.  $C_1$  opens the coupled holes

$$z : \mathbb{N} \rightarrow \text{Option } A, \quad s : A \rightarrow (\mathbb{N} \rightarrow \text{Option } A) \rightarrow \mathbb{N} \rightarrow \text{Option } A,$$

with the outer continuation foldr  $s z x s i$ .

5. Base: the contract clause for  $[]$  demands  $\forall i, z i = \text{none}$ ; constructor-directed propagation instantiates  $z := \lambda i. \text{none}$  and closes the clause by reflexivity.
6. Step: introduce  $x : A, r : \mathbb{N} \rightarrow \text{Option } A, i : \mathbb{N}$ . The contract clauses discriminate on  $i$ , so **scoreScrutinee** selects  $i$  and **caseSplit** yields the zero and successor branches (no indices; no discharge). Zero branch: the clause demands some  $x$ ; the constructor lane fills it. Successor branch with field  $k$ : the recursive hypothesis states that  $r k$  implements lookup in the tail at  $k$ ; the clause lookup  $(x :: xs) (k + 1) = \text{lookup } xs k$  reduces the obligation to that hypothesis; the exact-local lane fills  $r k$ .
7. The step is solved in the slice  $(x, r, i)$  and stored as a recipe; **reuseRecipe** reintroduces it in the full context by the substitution that maps its telescope to the fold's binders.
8. Final specialization: the candidate is  $\lambda x s i. \text{foldr } s z x s i$  with

$$z(i) = \text{none}, \quad s(x, r)(0) = \text{some } x, \quad s(x, r)(k + 1) = r(k),$$

and the contract follows by induction on  $x s$  and cases on  $i$  from the fold equations.

Lowered, this is P5's template and P8's checked **indexFold**:

```
fun xs => List.foldr (fun a smaller i => match i with
  | 0 => some a
  | n + 1 => smaller n) (fun _ => none) xs
```

P8 wrote this by hand, proved it equal to a structural reference for all lists and indices, and audited it axiom-free; no proposal discovered it automatically, and step 6 requires a registered relationship between the specification's cons/successor clause and its tail clause or a proof procedure that exposes it<sup>[P5,P8]</sup>.

**Variants.** For a Church-encoded list instantiated at carrier  $\mathbb{N} \rightarrow A$  with default  $d : A$ , the same equations become  $b(i) = d, s(x, k)(0) = x, s(x, k)(j + 1) = k(j)$ ; correctness is by induction on the encoded list and cases on the index, with no parametricity assumption about arbitrary inhabitants of the Church type<sup>[P1]</sup>. For an integer-indexed interface the carrier is  $\text{Int} \rightarrow A$  (or  $\text{Int} \rightarrow \text{Option } A$ ), the step has type  $A \rightarrow (\text{Int} \rightarrow A) \rightarrow \text{Int} \rightarrow A$ , and the branch contract distinguishes negative, zero, and positive inputs, making the integer case operation relevant; the negative-index policy (default, **none**, or other) is part of the specification and the proof, never guessed<sup>[P3,P6,P7,P9]</sup>. Leant's own diagnostics found the function-valued carrier and the integer-case step separately and failed to compose them; carrier invention plus the slice-and-recipe reuse above is the proposed repair, and no proposal reran that legacy query<sup>[P1,P7]</sup>. The same rule turns a left fold into a right fold with carrier  $S \rightarrow S$ : foldr  $(\lambda x k a. k(\text{step } a x)) (\lambda a. a) x s a_0$ <sup>[P4]</sup>.

## 16.8 Helper discovery by typed anti-unification

P1 names the mechanism, P8 gives the rule, P2 the cost and circularity discipline<sup>[P1,P2,P8]</sup>:

```
discoverHelpers(branch, bound):
  R := residual obligations that recur (same head shape, comparable structure) across branches
```

```

for cluster in structurallyComparable R, |cluster| >= 2:
  -- 1. close each occurrence over exactly its dependency telescope, in order
  closed := [ (dependencyClosedTelescope o, typeOf o) | o in cluster ]
  -- 2. typed anti-unification: differing well-scoped subterms become parameters
  (Delta_h, T_h, args) := antiUnify closed      -- args_o : Delta_h for each occurrence o
  if size T_h > bound: continue
  -- shared indices, instances, and proof assumptions stay explicit parameters;
  -- a dependency is never dropped because it occurs only in a type
  -- 3. propose a let-bound helper hole
  h := newGoal (Pi Delta_h. T_h) role := auxiliary
  replace each occurrence o by h args_o      -- checked in o's original context
  solve h under the intersection of the permitted scopes; validate every application
  charge the helper body once to program size; charge all search and verification work
  require the dependency graph of h to be acyclic (no use of the target's own contract)
  yield the plan with h

```

For recursion the rule composes with accumulator generalization: a changing argument fixed in an induction hypothesis is abstracted, the generalized helper is synthesized, and the original call specializes it (Section 10)<sup>[P8]</sup>. A helper theorem and a helper function are different products; a theorem whose proposition forbids data elimination cannot serve as an executable witness (Section 16.3), and every generated helper enters the acceptance gate in dependency order<sup>[P2,P8]</sup>. Anti-unification proposes; it never proves that the helper suffices<sup>[P1]</sup>.

## 16.9 What ran and what did not

Of the procedures in this section, only `MVarId.cases` with discarded `fields` and `subst`, split bounds, and the prototypes' scrutinee filters have executed; every one of the nine prototypes exercised them, and the indexed vector-head elimination ran in P2, P5, P8, and P9. P7's negative control for `Nonempty` extraction ran. The transport rule, the equality graph, the provider index, retrieval tiers, the abstract graph, recipes, the join, context slices, carrier proposal, and helper discovery were designed and not implemented. The `indexFold` carrier was hand-supplied and kernel-checked in P8; its discovery by `carrierFrontier` is the acceptance test for Section 9.4, and the baseline corpus's indexing transcripts (Section 13) are the regression set.

## 17 Algorithms: recursion synthesis

Section 10 fixes the design: recursion enters through a schema, recursive calls are capabilities (Invariant 10.2), motives may carry certificates (Proposition 10.4), and every plan has two products (Invariant 10.5). This section supplies the procedures. Each algorithm is the best-specified version found in the nine proposals, merged where they agree and marked where they differ. Pseudocode is written in the proposals' own vocabulary. Nothing here has been implemented: every recursive definition in every prototype was hand-supplied, and the tactics filled only branch bodies (Section 17.8).

### 17.1 Structural schema generation from recursor metadata

#### 17.1.1 The plan record

**Definition 17.1** (Recursion plan). A plan (called `RecSketch` in P9, a recursion descriptor in P4, a schema in P1 and P7) is the record

```

RecPlan :=
  telescope    : the elaborated function telescope (z_1 : A_1) ... (z_n : A_n)
  major       : index k of the discriminant z_k : D p i

```

```

fixed      : parameters that never change across recursive calls
generalized : Delta, an ordered dependency-closed sub-telescope
motive     : M : (i : Indices) -> (x : D p i) -> Sort u, checked by Lean
recursor   : the native recursor (or registered fold) instantiated at M
branches   : one Branch per constructor (fields, index equations,
             capabilities, residual contract, program/proof holes)
termination : none | structural | TerminationPlan (Section alg:rec-wf)
finalSpec  : the checked term mapping (rec ... major) back to the target
lowering   : the executable-lowering plan (Section alg:rec-lowering)

```

The fields are those of P4’s descriptor (telescope, decreasing arguments, admitted call forms, motive, generalized parameters, termination evidence per call) and P9’s sketch (adds the lowering plan)<sup>[P4,P7,P9]</sup>.

### 17.1.2 The generation algorithm

The following merges P2’s `planStructuralRecursion`, P5’s structural-plan generator, P6’s `deriveMotivePlans`, P7’s `proposeStructuralSchemas`, and P9’s `StructuralSketches`. They agree on every step; P2 is the only one that names the final specialization as a checked obligation, and P5 is the only one that states that generalization must go beyond the type-dependency closure<sup>[P2,P5,P6,P7,P9]</sup>.

```

generateStructuralPlans(goal, contract, ctx, budget):
  -- goal = (z_1 : A_1) -> ... -> (z_n : A_n) -> R(z)
  intro the telescope, keeping binder provenance of every z_j
  for major in rankMajorPremises(goal, contract): -- step 1
    meta := exactInductiveAndRecursorInfo(typeOf major)
    -- meta.params, meta.indices, meta.ctors, meta.rec, meta.minors
    for Delta in generalizationLadder(goal, major, meta, budget): -- step 2
      Delta := dependencyClose(Delta) -- step 3
      M := abstractTarget(goal, contract, major, Delta) -- step 4
      unless leanCheck(M : Indices -> D p i -> meta.rec.motiveSort):
        continue -- never guess the sort
      recApp := instantiateRecursor(meta.rec, meta.params, M)
      branches := []
      for (ctor, minor) in zip(meta.ctors, meta.minors): -- step 5
        br := branchTelescope(minor, M) -- fields, index eqs, IH types
        if impossibleByIndices(br): record evidence, skip -- Section sec:elimination
        br.caps := [capability(ih) for ih in br.inductionHyps] -- step 6
        br.contract := specialize(contract, ctor, br.fields, Delta) -- step 7
        br.holes := coupled(programHole(br.target), proofHole(br.contract))
        branches.append(br)
      fs := finalSpecialization(recApp, goal) -- step 8
      unless leanCheck(fs : originalTarget): continue
      yield RecPlan(telescope, major, fixed, Delta, M, recApp,
                    branches, structural, fs, lowering := pending)

```

The steps in detail.

1. *Major premise.* Rank the telescope’s inductive arguments (P7: “ranked recursive inputs”; P9: “each relevant inductive argument within the scheme budget”). The ranking prefers arguments that occur in the contract, arguments with recursive constructor fields, and arguments whose indices appear in the target family<sup>[P7,P9]</sup>. Mutual components, parameters, indices, and recursive fields must be kept distinct in the metadata read<sup>[P1]</sup>.
2. *Generalization ladder.* Enumerate  $\Delta$  in increasing structural complexity (Section 17.1.3).
3. *Dependency closure.* If a generalized variable occurs in the type of a later local, that local is generalized as well, in telescope order<sup>[P2,P4]</sup>. P5 adds the *behavioral* closure: a variable whose value changes between the call and the recursive call must be generalized even when

nothing depends on it by type (the index  $i$  of safe indexing, the accumulator of reversal). P4 computes this set from the recursive-call substitution of the template: parameters changed by that substitution are generalization candidates<sup>[P4,P5]</sup>.

4. *Motive*.  $M(\vec{i}, x) = \prod \Delta(\vec{i}, x)$ .  $R'(\vec{i}, x, \Delta)$  where  $R'$  is the target after replacing the original indices and discriminant by the motive's bound variables. When a certificate is requested,  $R'$  is the packaged family of Section 17.2. The motive is a Lean expression checked at the recursor's motive sort; the planner never infers the universe by counting surface binders<sup>[P2]</sup>.
5. *Branch telescope*. Each minor premise of the instantiated recursor is a Pi-telescope: non-recursive fields, recursive fields, then one induction hypothesis  $M(\vec{i}, x')$  per recursive field at that field's own indices. Read it back as an exact local context (P4 runs Lean's dependent elimination in a sandboxed state rather than substituting constructors textually). A tree gives two hypotheses; a function-valued recursive field gives a dependent family of hypotheses; none of these is flattened into an untyped "smaller results" array<sup>[P4,P5,P6]</sup>.
6. *Capabilities*. Every induction hypothesis becomes a capability (Section 17.3); the constructor field it belongs to is its structural evidence<sup>[P6,P7]</sup>.
7. *Per-branch specialization*. Substitute the constructor pattern into the residual contract and observations; simplify with constructor equations; put the branch's index equations into the context<sup>[P7,P9]</sup>.
8. *Final specialization*. Elaborate the term that applies the recursor result to the original discriminant and the original values of  $\Delta$ , and check it against the original target before the plan is admitted. This is what prevents a run from solving a conveniently generalized problem and reporting success on the original one<sup>[P2]</sup>.

### 17.1.3 The motive complexity ladder and failure-directed generalization

P2 specifies the ladder; P7 the motive grammar; P6 the sources of extra candidates<sup>[P2,P6,P7]</sup>:

```
generalizationLadder(goal, major, meta, budget):
  yield {}
  yield localsDependingOn(major, meta.indices) -- rung 0: direct result family
  for S in smallSubsets(changingArguments(goal, major), budget.motive):
    yield rung1 + S
  for T in accumulatorTemplates(contract, registeredSummaries):
    yield rung1 + T
  -- motive grammar per rung (P7): target family | (trailing args -> family)
  -- | {y : family // post} | tuples of these
```

Rung 2 is exponential in the number of changing arguments. All proposals that discuss it agree that enumeration is bounded and ranked, and P2 proposes *failure-directed generalization* to drive the ranking<sup>[P2,P6]</sup>:

```
failureDirectedGeneralization(plan, failedBranch):
  -- called when an induction hypothesis or capability cannot be applied
  reason := classify(failedBranch)
  -- IH fixed at wrong value of z_j      -> need z_j in Delta
  -- IH mentions index not in scope      -> need index generalization
  -- proof needs fact about z_j at call   -> need z_j quantified in R'
  ext := smallestDependencyClosedExtension(plan.generalized, reason.vars)
  schedule plan' := plan with generalized := ext, ahead of unrelated rungs
  -- the failed obligation is not evidence that the program is impossible
```

Two recorded failures are the regression tests for this component. In P7, a first `Vec.toList` fixed an ambient index  $n$  across recursive calls until its signature was changed to `{n : Nat} -> Vec A n -> List A`. In P9, induction on `Aligned` whose second index was the compound expression `(listMap f xs).val` failed until the indices were generalized to variables<sup>[P7,P9]</sup>.

## 17.2 Proof-carrying motives and certificate algebras

### 17.2.1 Motive shapes

For  $f : \prod_{x:D} B(x)$  with contract  $\forall x, R(x, f(x))$ , the certified motive is  $C(x) = \{y : B(x) // R(x, y)\}$  (P1 “refinement recursion”, P3 “packed motive”, P5 “certificate carrier”, P8 “certified motive”, P9 “pointwise packaging”, Proposition 3.3). With a trailing telescope  $a : S$  the motive is  $C(x) = \prod_{a:S} \{y : B(x, a) // R(x, a, y)\}$ , and this is where generalizing an index or accumulator becomes a substantive choice<sup>[P3,P5]</sup>. P8 notes that a general list recursor branch also receives  $xs$  itself, which matters when the proof depends on the original tail<sup>[P8]</sup>.

### 17.2.2 Branch obligations

For lists the instantiated minor premises are exactly the types of P8’s checked `certifiedRecursor`<sup>[P8]</sup>:

```
-- CertifiedFold.lean (hand-written, checked, empty axiom inventory)
def certifiedRecursor (R : List a -> b -> Prop)
  (base : {v : b // R [] v})
  (step : (x : a) -> (xs : List a) ->
    {v : b // R xs v} -> {v : b // R (x :: xs) v}) :
  (xs : List a) -> {v : b // R xs v}
| [] => base
| x :: xs => step x xs (certifiedRecursor R base step xs)

theorem fold_contract (R : List a -> b -> Prop) (base : b) (step : a -> b -> b)
  (hbase : R [] base)
  (hstep : forall x xs v, R xs v -> R (x :: xs) (step x v)) :
  forall xs, R xs (xs.foldr step base)
-- proof: induction xs; nil => hbase; cons x xs ih => hstep x xs _ ih
```

A branch state therefore contains a data hole and a proof hole that constrain each other: in the cons branch the data hole has type  $B(x::xs)$  and the proof hole has type  $R(x::xs, ?y)$  with  $?y$  the data hole, and the context contains  $v : B(xs)$  and  $h : R(xs, v)$ <sup>[P3,P5]</sup>.

### 17.2.3 Contract failure backtracks into the branch choice

Because the proof hole mentions the data metavariable, a failed proof restores the state to before the data assignment and tries the next data candidate. P2’s **Choice** trace is the minimal test: the first constructor is assigned, the dependent proof fails, the joint continuation restores the state and the second constructor succeeds<sup>[P2]</sup>. P3’s length example shows what the motive does and does not decide: with  $r : \text{List } A$ ,  $h : |r| = |xs|$ , both  $x::r$  and  $r \mathrel{++} [x]$  satisfy  $|ys| = |x::xs|$ , so the motive prunes but does not invent intent; an order-sensitive contract decides between identity and reverse<sup>[P3]</sup>.

```
certifiedBranch(br):
-- br.context has fields, and (v_i, h_i) for each recursive field
for y in candidates(br.target, br.context ++ [v_i]): -- Section sec:spines
  snapshot := state
  assign dataHole := y
  if prove(proofHole : R(ctorPattern, y), br.context ++ [h_i]): -- Section sec:proof-services
    return Subtype.mk y proof
  restore snapshot -- the proof failure rejects y, not the branch
fail(branch) -- feeds failureDirectedGeneralization
```

### 17.2.4 The alternative route and the choice between them

P4 specifies the other route as a two-pass template: construct the structurally recursive function with body holes, complete them, pass Lean’s termination and compilation checks, then generate a theorem with the matching induction principle and prove the original contract from the checked defining equations<sup>[P4,P6]</sup>.

**Decision 17.2** (Integrated versus separate certification). Use the certified motive when the contract is pointwise,  $B(x)$  is data-valued, and the per-branch obligation is expected to be local (P9’s Proposition 3.3 is the bridge). Use synthesize-then-prove when the contract relates several calls or several inputs (injectivity, an involution law), concerns effects or a representation relation, or needs an invariant stronger than the pointwise postcondition; when the executable declaration should stay proof-free; or when a strong library theorem about the executable form exists. Both routes remain in the portfolio and both must end at the same acceptance contract (Section 13)<sup>[P2,P5,P6,P9]</sup>.

The integrated route prunes earlier but inflates types and proof terms; the separate route keeps simpler executables and can reuse library theorems<sup>[P2]</sup>. Three hand-written instances were checked: `certifiedMap` with the functional relation `MapRel` (P1), `certifiedAppend` with the contract `zs = xs ++ ys` (P5), and `certifiedRecursor`, `fold_contract`, and `indexFold_correct` (P8)<sup>[P1,P5,P8]</sup>.

## 17.3 Recursive-call capabilities

### 17.3.1 Data structure

P2, P6, P7, and P9 each list the fields; the union is<sup>[P2,P6,P7,P9]</sup>:

```
Capability :=
  family      : the declaration under construction (never a provider)
  motive      : M, the current result family
  fixed       : the parameters that must be passed unchanged
  varying     : Delta, the generalized arguments the call may re-supply
  smaller     : the admissible argument (a constructor field for structural
               recursion; a bound variable y for well-founded recursion)
  evidence    : Structural(field path from the recursor minor premise)
               | WellFounded(obligation : r(y, x), status : open | proved p)
  resultType  : M(indices(smaller), smaller) instantiated -- exact, from Lean
  build       : the expression constructor that assembles the call
               (an induction-hypothesis variable in a recursor term;
               a recursive call in an equation skeleton after lowering)
```

The result type is read from the actual constructor field and recursor information, never computed by subtracting one from the outer index; this is what makes the same object correct for indexed, nested, and mutual families<sup>[P7]</sup>.

### 17.3.2 Consuming a capability

```
applyCapability(cap, branch, args):
  -- args supplies values for cap.varying only; cap.fixed is implicit
  check args are well-typed against Delta under the branch context
  match cap.evidence with
  | Structural(path) =>
    -- nothing to prove: the recursor supplied the hypothesis
    term := cap.build(path, args)
  | WellFounded(obl, open) =>
```

```

-- the call is admitted only with the decrease proof
obl := (r(cap.smaller, currentMajor)) -- Section alg:rec-wf
p := prove(obl, branch.context)      -- may fail; may be deferred
if p = none: return blocked(obl)     -- reported, never assumed
term := cap.build(cap.smaller, p, args)
spec := instantiate(branch.contract, cap.smaller, args) -- the IH's contract
return (term : cap.resultType, spec)

```

A call with a new accumulator on the structurally smaller argument is valid; a call on the unchanged list is not made valid by changing the accumulator (P7). For a structural list branch  $a::xs$  the capability authorizes  $xs$ , not  $a::xs$ <sup>[P7]</sup>. Each call also instantiates the recursive specification at the smaller input, so the branch prover sees the induction hypothesis's contract<sup>[P7]</sup>.

### 17.3.3 The self-reference audit

The rule has three enforcement points, all motivated by P2's failed first run, in which unfiltered local declarations exposed the provisional definition, a local-provider loop reused it, and Lean's termination check rejected the cycles (error recovery then inserted `sorryAx`)<sup>[P2]</sup>:

```

auditNoSelfReference(plan, candidate):
-- 1. during search: the provider inventory excludes the declaration
--    under construction and every isImplementationDetail local
assert family(plan) not in providerInventory(branch) -- P2 repair
-- 2. at assembly: every occurrence of family(plan) in candidate
--    is the build output of some capability with closed evidence
for occ in occurrences(family(plan), candidate):
  assert occ.origin is Capability and occ.origin.evidence is closed
-- 3. at acceptance: the dependency closure of the declaration bundle
--    contains no provisional constant, no sorryAx, and passes Lean's
--    own structural or well-founded elaboration
assert dependencyClosure(bundle) has no provisional or sorry constants

```

Filtering implementation-detail locals (following Lean's own `assumption`) repaired the prototype; the second and third checks are production requirements that the prototype does not implement<sup>[P2]</sup>.

## 17.4 Accumulator introduction and invariant discovery

### 17.4.1 The lane

P7's four stages are the skeleton; P4 supplies the trigger (the recursive-call substitution) and the constraint view; P3 and P6 supply the summary grammar; P9 names the three obligations<sup>[P3,P4,P6,P7,P9]</sup>:

```

discoverAccumulatorInvariant(plan, publicContract):
-- stage 1: extract recurring quantities
Q := recurringSubexpressions(residualBranchContracts(plan))
  ++ parametersChangedBy(recursiveCallSubstitution(plan)) -- P4
-- stage 2: propose a helper with those quantities as parameters
go := helperSignature(plan.telescope ++ [acc : typeOf q | q in Q])
-- stage 3: candidate invariants from a bounded grammar
for I in summaryGrammar(publicContract, acc, admittedOps, units, budget):
-- stage 4: three obligations, all proved or the candidate is dropped
  init := forall x, I(x, acc_0) -> publicContract(x) -- finalization
  base := I(base pattern, acc) for the base branch -- initiation
  step := forall acc, I(xs, acc') -> I(ctor xs, acc) -- preservation
    where acc' is the accumulator passed to the recursive call
  if proveAll([init, base, step]):

```

```

yield plan' with motive generalized over acc and contract I
-- a failed proof leaves I unresolved; it never refutes the program
-- cost: new fields, type size, and grammar tier are charged separately
-- from branch synthesis (P3), so the lane cannot add state indefinitely

```

The grammar of relational summaries is built from the public relation, the accumulator variable, the admitted operations with registered algebraic summaries (concatenation, length, sum, a monoid fold), and their unit elements; it prefers accumulator types already present in the request or library specifications over invented types<sup>[P3,P6]</sup>. P8 calls the same procedure *target-directed*: start from the postcondition and generalize only where recursion requires it (a changing accumulator, a function carrier's future argument, a paired auxiliary measure), rather than enumerating invariants from a global language<sup>[P8]</sup>. Counterexamples from the observation bank eliminate templates cheaply before proof is attempted<sup>[P7]</sup>. Family-specific templates (permutation-and-order for sorting, accumulator length as a function of an index) enter as plugins with documented grammars<sup>[P7]</sup>.

### 17.4.2 Worked: accumulator reversal

Public request  $f : \text{List } A \rightarrow \text{List } A$  with  $f(xs) = \text{reverse}(xs)$ . The tail-recursive plan (P6's `reverseAux`) is

```

def go : List A -> List A -> List A
| [],      acc => acc
| x :: xs, acc => go xs (x :: acc)

```

Stage 1: the recursive-call substitution is  $acc \mapsto x::acc$ , so  $acc$  is a changing argument and must be generalized; the public contract at  $acc = []$  is too weak to justify the call with a nonempty accumulator<sup>[P4,P6,P9]</sup>. Stage 2: the helper is  $go(xs, acc)$  with public result  $go(xs, [])$ ; the accumulator belongs to the helper, not to a changed public type<sup>[P7]</sup>. Stage 3: from the grammar (relation `reverse`, variable  $acc$ , operation `++`, unit `[]`) the candidates in cost order are  $go(xs, acc) = \text{reverse}(xs)$ ,  $go(xs, acc) = acc ++ \text{reverse}(xs)$ , and  $go(xs, acc) = \text{reverse}(xs) ++ acc$ . Stage 4 for the third candidate, with motive  $M(xs) = \forall acc, go(xs, acc) = \text{reverse}(xs) ++ acc$ :

```

initiation   : go [] acc = acc = reverse [] ++ acc          -- rfl
preservation : ih : forall acc, go xs acc = reverse xs ++ acc
              goal : go (x :: xs) acc = reverse (x :: xs) ++ acc
              unfold: go xs (x :: acc)
              ih at (x :: acc): reverse xs ++ (x :: acc)
              rhs: (reverse xs ++ [x]) ++ acc
              close by List.append_assoc, List.cons_append, List.nil_append
finalization : f xs = go xs [] = reverse xs ++ [] = reverse xs -- append_nil

```

The first candidate fails preservation (the induction hypothesis says nothing about  $x::acc$ ); the second fails at the same step. Two artifacts checked exactly this invariant by hand. P2's `ReverseCertificate.lean` proves `foldInvariant : xs.foldl (fun acc x => [x] ++ acc) acc = specReverse xs ++ acc` by induction  $xs$  generalizing  $acc$  with the three rewrites above, for the step  $[x] ++ acc$  that its finite CEGIS run selected fourth after  $acc$ ,  $[x]$ , and  $acc ++ [x]$  were refuted on 40 lists; P6's `reverseAux_spec` proves the same statement for `reverseAux` (with `propext` in its inventory). Neither invariant was generated<sup>[P2,P6]</sup>. The difference-list variant  $d(ys) = \text{reverse}(xs) ++ ys$  over the carrier  $\text{List } A \rightarrow \text{List } A$  is the same invariant read through a function carrier (Section 9.4)<sup>[P2,P8]</sup>.

## 17.5 Well-founded recursion

### 17.5.1 Schema and capability

The schema is  $F(x) = \text{step}(x, \lambda y (h : y \prec x). F(y))$  with a supplied proof that  $\prec$  is well-founded; the branch capability is  $\text{rec} : \prod_{y:D} \prec(y, x) \rightarrow M(y)$ , and  $M$  may be the certified motive so that the smaller call's certificate is available to the branch proof<sup>[P1,P3,P5,P8]</sup>. P5 fixes the artifact that the lowerer consumes<sup>[P5]</sup>:

```
TerminationPlan :=
  relation  : the chosen relation or measure expression mu
  wf        : proof that the relation is well-founded
  calls     : for each recursive occurrence:
              (argument y, branch context, decrease obligation, evidence)
```

### 17.5.2 Measure search

P1's Algorithm 4 is the most explicit; P3, P5, P8, and P9 give the same grammar<sup>[P1,P3,P5,P8,P9]</sup>:

```
measureSearch(plan, proposedCalls, budget):
  grammar := [ natural-number argument n
              , registered size (List.length, structural size, sizeof)
              , bounded difference n - i, admissible only with a branch
                hypothesis i < n (truncated subtraction: 0 - 1 = 0)
              , lexicographic tuples of the above, short
              , user-supplied well-founded relation ]
  for mu in grammar in cost order:
    wf := wellFoundedness(mu)           -- a theorem, retrieved or proved
    ok := true
    for call (y from x, ctx) in proposedCalls:
      goal := mu(y) < mu(x)             -- or r(y, x) for a relation
      p := prove(goal, ctx, [branch hypotheses, arithmetic (omega),
                             registered decrease lemmas], budget.perCall)
      if p = none: ok := false; break    -- reject mu, not the program
      record (call, goal, p) in plan.termination.calls
    if ok: yield TerminationPlan(mu, wf, calls)
  -- exhaustion reports "no measure found in this grammar", never "diverges"
```

Three rules are stated by every proposal that treats this lane. A sampled execution that decreases is not a measure, and an SMT verdict that an arithmetic approximation decreases is not a decrease proof<sup>[P2,P3,P5]</sup>. For a loop that increments  $i$  under  $i < n$ , the measure is  $n - i$  and its proof must use the branch condition and the exact truncated-subtraction semantics<sup>[P1,P7]</sup>. For Euclidean-style recursion a modulus-bound lemma is the decisive step and must be retrieved from the library or supplied with a component contract<sup>[P2,P6]</sup>. The definition is registered only after Lean's own termination elaboration accepts it; because well-founded unfolding is propositional, the generated equation lemmas are registered in the candidate's proof context, and ranking charges the cost of the reduction behavior<sup>[P2,P9]</sup>. Adding **partial** or a fuel parameter is not a repair unless fuel is part of the interface or proved sufficient<sup>[P4,P8]</sup>.

### 17.5.3 Mutual and nested recursion

Mutual recursion is a single strongly connected component synthesized jointly: the domain is the tagged sum  $\Sigma t : \text{Tag}, D_t$ , the measure is shared (typically lexicographic in the component size and the tag), each component's capabilities may call the other components only through the shared **rec**, and the summaries of the components are checked simultaneously by one

well-founded induction (proving each from an unchecked summary of the other is circular and prohibited). The alternative is a registered native mutual recursor with explicit motives per component<sup>[P3,P4,P5,P7,P8]</sup>. Nested inductives use their actual generated recursion principle; their induction hypotheses may be function- or container-shaped, and the branch solver must then synthesize the corresponding higher-order argument or traversal. Metadata the adapter does not support is reported as an explicit limitation, never approximated as an ordinary non-recursive field, while the family’s library components remain usable as ordinary providers<sup>[P2,P3,P6,P8]</sup>.

## 17.6 Executable lowering

### 17.6.1 The failure that motivates it

Four artifacts hit the same wall. P4’s first vector map used tactic induction inside an executable `def`: the axiom audit was empty and the code generator rejected the `Vec.rec` form. P6’s `countLeaves t := PilotTree.rec 1 (fun _ _ l r => 1 + r) t` was rejected for `PilotTree.rec` and kept as an expected-failure probe. P8’s bare `List.rec` definition passed axiom inspection and failed code generation. P9’s induction-tactic vector map did the same. In every case rewriting the same construction as pattern equations compiled and evaluated correctly<sup>[P4,P6,P8,P9]</sup>.

### 17.6.2 Equation-oriented lowering

P6’s five steps, with P9’s authorization protocol as the second stage<sup>[P6,P9]</sup>:

```
lowerToEquations(plan):
  -- 1. recover structure from the plan, never from printed recursor text
  major, patterns, locals, caps, Delta := plan.major, [ctor patterns],
    [branch fields], plan.branches[*].caps, plan.generalized
  -- 2. replace each induction-hypothesis use by a recursive call
  for br in plan.branches:
    body := br.solution
    for use in occurrences(cap.build for cap in br.caps, body):
      body := body[use := f fixed cap.smaller (use.varyingArgs)]
    equations.append(f fixed (pattern br) (Delta) := body)
  -- 3. hygienic definition syntax with explicit binders
  decl := def f (fixed) : (major : D p i) -> (Delta) -> M(i, major)
    with equations
    plus termination_by mu / decreasing_by proofs from plan.termination
  -- 4. elaborate in a staging environment with approved dependencies only
  env' := elaborate(stagingEnv(approved), decl)
  reject on termination-elaboration or code-generation failure
  -- 5. certify the executable constant
  either prove Spec(f) directly against f's equation lemmas
  or prove forall t, f t = plan.logicalCandidate t and transport the proof
  return declaration bundle (f, helpers as an audited DAG)
```

The two-stage protocol is: stage one produces the declaration bundle through Lean’s declaration elaborator (telescope, pattern equations, branch bodies, termination annotation); stage two, P9’s `Authorize`, restores the origin environment in an isolated context, lowers, rejects unresolved metavariables and leaked free variables, kernel-checks the bundle and its contract theorem, audits axioms and implementation dependencies, compiles when the request is executable, exports replay source, and publishes atomically, recording `kernelChecked`, `compiled`, and `evaluatedOnExamples` separately (Section 13)<sup>[P9]</sup>. The correspondence route was checked once: P6’s theorem  $\forall t, \text{countEq}(t) = \text{countRec}(t)$  between the noncomputable semantic model and the executable equation form, with `propext` in its inventory. A pointwise correspondence transfers

pointwise contracts; a predicate on the whole function needs function extensionality or a fresh proof, and the axiom policy applies to the transfer proof as well<sup>[P6]</sup>. Lowering may change implementation structure but never the type or specification; helpers it introduces form an audited dependency DAG<sup>[P6]</sup>. The plan grammar tags each construction as genuine recursor, bounded unrolling, or nonrecursive so that repeated case splits are never reported as recursion<sup>[P5]</sup>.

## 17.7 Worked traces

Each trace lists the motive, the branch goals, the capabilities, the final terms, and what was actually checked.

### 17.7.1 Indexed vector map

The family is `Vec.nil : Vec A 0` and `Vec.cons : A -> Vec A n -> Vec A (n + 1)`<sup>[P2,P6,P7,P9]</sup>.

```
target : {A : Type u} -> {B : Type v} -> (A -> B) -> {n : Nat} -> Vec A n -> Vec B n
major   : xs : Vec A n          fixed : A, B, f          generalized : {} (rung 0)
motive  : M (n : Nat) (xs : Vec A n) := Vec B n          -- checked at Type v
nil branch : index n := 0
  goal : Vec B 0
  caps : none
  solution : Vec.nil
cons branch : index n := k + 1, fields x : A, xs : Vec A k
  goal : Vec B (k + 1)
  caps : ih : Vec B k (structural, field xs)
  spine : Vec.cons ?h ?t with ?h : B, ?t : Vec B k
  ?h := f x (demand-directed application, Section sec:spines)
  ?t := ih (capability, no obligation)
final specialization : rec ... xs : Vec B n -- identical to target
lowered :
  def Vec.map (f : A -> B) : Vec A n -> Vec B n
  | .nil => .nil
  | .cons x xs => .cons (f x) (Vec.map f xs)
```

Checked: P2's `SkeletonChecks.lean` (repeated in P7 as `vecMap`) supplies the pattern skeleton and the recursive call have mapped `:= vectorMap f xs`, and `leant2_core` fills both branches; `vectorMap_nil` and `vectorMap_cons` hold by `rfl`; all three declarations have empty axiom inventories; evaluation maps `[1, 2]` to `[2, 3]` under increment. P7 adds a hand-written proof of `(vecMap f xs).toList = xs.toList.map f`. The type alone does not force elementwise mapping (a nonempty branch may repeat the transformed head); the checked `rfl` equations show that the tactic happened to produce the mapping branch<sup>[P2,P7,P9]</sup>.

### 17.7.2 Zip with a generalized second vector

Target `Vec A n -> Vec B n -> Vec (A x B) n`<sup>[P6]</sup>.

```
major   : xs : Vec A n          changing : ys (its length follows n)
rung 0 fails : ih : Vec (A x B) k is fixed at the original ys, which has
              length k + 1, so the cons branch cannot use it
rung 2 (behavioral generalization of ys) :
motive  : M (n : Nat) (xs : Vec A n) := Vec B n -> Vec (A x B) n
nil branch : goal : Vec B 0 -> Vec (A x B) 0
  intro ys; eliminate ys : Vec B 0 (Section sec:elimination):
  cons impossible (k + 1 = 0 refuted by no-confusion); nil => Vec.nil
cons branch : index n := k + 1, fields x : A, xs : Vec A k
  goal : Vec B (k + 1) -> Vec (A x B) (k + 1)
  caps : ih : Vec B k -> Vec (A x B) k (structural, field xs)
  intro ys; eliminate ys : Vec B (k + 1):
```

```

    nil impossible (0 = k + 1); cons y ys' with ys' : Vec B k
    spine : Vec.cons (x, y) (ih ys')          -- ih applied to the tail
final specialization : (rec ... xs) ys : Vec (A x B) n
lowered :
  def Vec.zip : Vec A n -> Vec B n -> Vec (A x B) n
  | .nil, .nil => .nil
  | .cons x xs, .cons y ys => .cons (x, y) (Vec.zip xs ys)

```

Checked: the lowered definition was hand-written in P6's pilot and reported **propext** (from the impossible-branch eliminations). The generalization step and the eliminations were not automated<sup>[P6]</sup>.

### 17.7.3 Safe list indexing

Target `lookupFin` :  $(xs : \text{List } A) \rightarrow \text{Fin } xs.length \rightarrow A^{[P2]}$ ; P8 states the vector form  $(n : \text{Nat}) \rightarrow \text{Vec } A \ n \rightarrow \text{Fin } n \rightarrow A$  with  $M(n, xs) = \text{Fin } n \rightarrow A^{[P8]}$ .

```

major      : xs : List A          changing : i (its type depends on xs)
rung 1 (dependency closure) : i : Fin xs.length depends on xs
motive     : M (xs : List A) := Fin xs.length -> A
nil branch : goal : Fin (List.length []) -> A
  intro i; i.isLt : i.val < 0; close by Nat.not_lt_zero (elim i)
cons branch : fields x : A, xs : List A
  goal : Fin (List.length (x :: xs)) -> A      -- = Fin (xs.length + 1) -> A
  caps : ih : Fin xs.length -> A      (structural, field xs)
  intro i; cases i.val:
    0      => x
    j + 1 => ih (Fin.mk j (Nat.lt_of_succ_lt_succ i.isLt))
            -- the data construction and the range proof are one obligation
final specialization : (rec ... xs) i : A

```

The point of the trace is the motive: the index stays inside it, so the cons branch's recursive result is  $\text{Fin } xs.length \rightarrow A$  and not a result specialized to the original index of the larger list<sup>[P2]</sup>. The related fold form with carrier  $\mathbb{N} \rightarrow \text{Option } A$  (`indexFold`, P8; the template in P5; P9's carrier sketch) was hand-written and proved equal to a structural reference for every list and index<sup>[P5,P8,P9]</sup>. No artifact checked `lookupFin` itself, and P2 warns that the engine must not substitute it for a user's integer-indexed request<sup>[P2]</sup>.

### 17.7.4 Accumulator reversal

The full trace is in Section 17.4.2. In plan terms: major  $xs$ , generalized  $\{acc\}$  found at rung 2 from the recursive-call substitution, motive  $M(xs) = \forall acc, \{r : \text{List } A // r = \text{reverse}(xs) ++ acc\}$  in the certified reading, capability in the cons branch  $ih : \forall acc, M(xs)(acc)$  consumed at  $x::acc$ , final specialization  $f(xs) = (\text{rec } xs [])$  with the finalization lemma `List.append_nil`<sup>[P2,P6]</sup>.

## 17.8 What was demonstrated and what was not

Demonstrated, in checked artifacts:

- Branch completion inside hand-supplied structural skeletons: `vectorMap/vecMap` (P2, P4, P7, P9) and `vectorReplicate` by hand-selected induction on  $n$  (P5), with the recursive call bound by the author<sup>[P2,P4,P5,P7,P9]</sup>.
- Hand-written certified folds and their universal theorems: `certifiedMap/MapRel` (P1), `certifiedAppend` (P5), `fold_contract`, `certifiedRecursor`, `indexFold_correct` (P8)<sup>[P1,P5,P8]</sup>.

- Hand-written accumulator invariants: `foldInvariant` for a fold step selected by finite CEGIS (P2); `reverseAux_spec` (P6)<sup>[P2,P6]</sup>.
- The code-generator boundary and its equation-oriented repair, including one checked correspondence theorem (P4, P6, P8, P9)<sup>[P4,P6,P8,P9]</sup>.
- The self-reference failure and its local-filter repair (P2)<sup>[P2]</sup>.

Not demonstrated by any artifact: selection of the major premise; dependency-closed or behavioral generalization; motive construction and its Lean check; instantiation of branch telescopes from recursor metadata; creation or consumption of capabilities by the engine; the certified-branch backtracking loop; accumulator or invariant discovery; measure search or any well-founded plan; mutual or nested plans; lowering from a plan; final specialization. Every algorithm in this section is therefore a specification with named regression tests (Sections 17.1.3, 17.3, 17.6), not a description of running code<sup>[P2,P7,P8,P9]</sup>.

## 18 Algorithms: contracts, counterexamples, and certified pruning

Section 11 fixes what behavioral information is allowed to do: tests refute, proofs certify, and only proof-producing analyses prune (Decision 2.5). This section gives the procedures. Every algorithm below operates on the frozen contract  $\Phi$  of Section 5.3; none of them replaces it. Provenance marks name the proposals whose text or code supplied each step; where the proposals differ, the better-specified version is used and the alternative is named.

### 18.1 Contract decomposition and backward propagation

A *contract decomposer* receives the elaborated property and one construction step and returns local obligations plus a *reconstruction term* showing that those obligations imply the original<sup>[P9]</sup>. Every returned view carries one of three tags<sup>[P2,P8]</sup>: *necessary* (a violation rejects; passing proves nothing), *sufficient* (solving it proves  $\Phi$  through the supplied implication; failing to solve it proves nothing), or *equivalent* (both). P8 names the same three outputs `sufficientObligations`, `necessaryConstraint`, and `heuristicHint`; the unified rule set adopts the tags and keeps the hint as a fourth, authority-free reply.

```
decompose(Phi, step, ctx) -> list of (obligation, tag) + reconstruction
-- Phi is the exact contract on the term being built at this node.
match step:
| Lambda x, body hole ?r, Phi = forall x, Post(x, f x):
    return [(Post(x, ?r) under ctx,x, equivalent)]
    reconstruction: fun x => proofOf(Post(x, ?r)) -- prop:pointwise
| Constructor K with field holes ?f1..?fn:
    substitute K ?f1..?fn into Phi; simplify by projection
    equations and registered constructor lemmas;
    for a dependent field, instantiate later field contracts
    with the ACTUAL earlier fields, never with fresh abstractions
    return field obligations tagged as the lemma proves them
    (equivalent when the constructor equations are iff; else sufficient)
| If c then ?a else ?b, with Decidable c:
    return [(c -> Phi(?a), equivalent), (not c -> Phi(?b), equivalent)]
    reconstruction: dite-elimination combining both branch proofs
| Provider application g ?h with registered theorem
    thm : forall a, P(a) -> Q(a, g a):
    if Q(a, g a) implies Phi(g a) by a checked bridge B:
    return [(P(?h), sufficient)]
    reconstruction: fun p => B (thm ?h p)
    else return [(Phi(g ?h), equivalent)] -- no propagation, no loss
```

```

| Match on x with branches for constructors K_i:
  route Phi under the branch equation x = K_i fields_i;
  each branch obligation is equivalent under its path assumption
| Conjunction Phi1 /\ Phi2:
  return [(Phi1, equivalent), (Phi2, equivalent)] on the SAME term
  reconstruction: And.intro
| Recursive call through a capability (alg:recursion):
  use only the induction hypothesis or the verified recursive
  summary at that smaller argument; nothing else
| anything else (multi-call laws, opaque higher-order providers):
  return [(Phi, equivalent)] with a symbolic residual; no decomposition

```

Three consequences are stated once and enforced by the tags. First, a *sufficient* provider rule may not delete arguments failing  $P$ : the theorem need not characterize every way of proving  $Q(g(h))$ , so an audit lane must retain a route that does not depend on the contract library<sup>[P4,P7]</sup>. Second, a contract that is too weak (length preservation for a reversal request) is never turned into a rejection of a correct provider<sup>[P7]</sup>. Third, a global relationship between two calls of  $f$  (idempotence, a homomorphism law, injectivity) has no independent per-output postcondition; it stays attached to the whole candidate and only necessary views are extracted from it<sup>[P2,P6,P8]</sup>.

### 18.1.1 Condition abduction as a guarded refinement rule

P2's appendix adds one decomposition rule that constructs a program correct on a proved region and then synthesizes the complement<sup>[P2]</sup>.

```

abduceGuard(Phi = forall x, Post(x, f x), e1, bank):
  -- e1 is a candidate body already found; it fails some bank inputs.
  for G in guards ordered by size:
    -- constructor tests, equalities,
    -- order comparisons, registered
    -- domain predicates

    if not decidable(G): continue
    rank G by how well it separates bank inputs e1 passes from those it fails
    try prove forall x, G x -> Post(x, e1 x)
    -- branch 1
    if proved with proof p1:
      synthesize e2 under hypothesis not (G x)
      -- branch 2, same loop
      if e2 found with proof p2 of forall x, not G x -> Post(x, e2 x):
        return (fun x => if G x then e1 x else e2 x,
                reconstruction from p1, p2, and the Decidable instance)
    return none
  -- no promise of a weakest guard; Phi stays authoritative

```

Separation on the bank ranks guards; it is not the proof of the branch region. The procedure searches progressively richer guards and never claims to find a weakest one<sup>[P2]</sup>.

## 18.2 Typed observations and residual evaluation

**Definition 18.1** (Typed observation). An observation is a typed substitution  $\rho$  into the input telescope, proofs of every precondition and dependent bound, the exact output observation in the fiber  $B(\rho)$ , and an evidence policy (native execution, a proof of the finite proposition, or a kernel-checked reduction)<sup>[P6,P8]</sup>. The three evidence strengths are stored separately in the manifest; none equals a proof of the universal contract<sup>[P6]</sup>.

### 18.2.1 The residual value language

Partial programs are evaluated under a legal sample until evaluation reaches a hole or an opaque operation. The result is a typed residual, never a default value<sup>[P2,P6]</sup>:

```

Residual ::= Known(constructor, fields : list Residual)
          | Closure(binder, body, env)
          | Suspended(opaque provider, args : list Residual)
          | Cond(path condition, thenResidual, elseResidual)
          | Hole(h, args : list Value, expected : Option Value, path : Phi)
          | Unknown(reason)           -- fuel exhausted, stuck opaque constant, effect

```

Fuel is explicit; **Unknown** is a first-class outcome and is never read as **false**<sup>[P2,P3,P6]</sup>. P2’s residual forms (known constructors, closures, suspended applications, conditionals, named holes with observed arguments) are adopted verbatim; P6 adds the path assumptions to the hole record.

### 18.2.2 Live-example propagation

```

propagate(sketch t, sample rho, expected y, table T, fuel):
  r := residual(t, rho, fuel)
  match (r, y):
  | (Known(K, fs), Known(K, ys)) -> for i: propagate(fs_i, ys_i)      -- decompose
  | (Known(K, _), Known(K', _)) with K /= K' -> Refute(rho)         -- constructor clash
  | (Closure(x, b, env), function observation (a |-> v)) ->
    propagate(b[x := a], v)                                           -- lambda: move to body
  | (Cond(c, r1, r2), y) ->
    if c decided under rho: propagate(chosen branch, y)
    else: record y under symbolic path c and under not c             -- both possibilities
  | (Suspended(g, args), y) ->
    if summary(g) has a checked inverse-image rule: apply it
    elif summary(g) gives a finite alternative set: branch on it
    else: keep the relational residual { z | g z = y }                 -- no guessed inverse
  | (Hole(h, args, _, path), y) ->
    T.insert(h, args, y, path)                                         -- constrain ONE use
    if T has (h, args, y') with y' /= y and args decidablely equal:
      Refute(rho)                                                       -- functional consistency
  | (Unknown(_), _) -> Unresolved(rho)                                 -- never a refutation

```

The hole-observation table  $T$  is keyed by hole identity and argument tuple; equal inputs induce equal outputs, so  $h(0) = 0$  and  $h(0) = 1$  are inconsistent for one deterministic  $h$  although an angelic interpreter assigning each occurrence independently would satisfy both<sup>[P2,P6]</sup>. P2’s regression script checks exactly this: one consistent valuation for the pair versus one spurious independent assignment. For higher-order arguments the argument-equality test may be unavailable; the table then retains uncertainty rather than inventing a comparison<sup>[P2]</sup>. Recursive-call outputs under a sketch are represented symbolically under the induction hypothesis or well-founded call specification; a model of those symbolic constraints does not prove that a single recursive implementation realizes them, so they never support hard pruning<sup>[P2,P7]</sup>. Two holes correlated by one sample are solved together, not independently<sup>[P7]</sup>.

### 18.2.3 Typed test generation for dependent inputs

```

generate(telescope (x1 : A1) ... (xn : An[x1..x_{n-1}]), registry):
  for i in 1..n:
    Ai' := instantiate Ai with the values already chosen
    gen := registry.lookup(head Ai')                     -- constructors as the start grammar
    if none: return Unsupported(Ai')                     -- symbolic, never invented
    xi := gen.sample(Ai')                                -- Fin n carries its bound proof;
                                                         -- a dependent pair chooses fst, then
                                                         -- samples the instantiated snd type
    if precondition on xi: attach its proof or discard the sample
  return rho with all validity proofs

```

Polymorphic targets are tested at several concrete types with distinct dictionary payloads, heterogeneous products, and asymmetric values, as falsification evidence only<sup>[P3,P6]</sup>. A generator for  $i : \text{Fin } n$  cannot sample  $n$  and  $i$  independently and forget the bound proof<sup>[P1]</sup>. The default evaluator never runs file, network, or process effects; IO values are never executed as tests<sup>[P3,P7,P9]</sup>.

## 18.3 The certified CEGIS loop

### 18.3.1 Certified counterexamples

**Definition 18.2** (Certified counterexample). For  $\Phi(t) = \forall x, P(x) \rightarrow R(x, t(x))$ , a certified counterexample is a triple  $(a, p : P(a), q : \neg R(a, t(a)))$  whose components are Lean terms checked in the frozen context<sup>[P7,P8]</sup>. The reusable bank stores the *observation*  $(a, p)$ ; the refutation  $q$  stays attached to  $t$ . A later candidate is rejected only after its own failure on  $(a, p)$  is established afresh<sup>[P7,P8]</sup>.

```

replay(candidate t, proposal a) -> Option CertifiedCounterexample
  -- proposals come from native execution, a solver model, or a generator
  decode a into the exact input type (fibers, Fin bounds, dictionaries)
  if decoding fails: return none -- a model is not a value
  p := prove P(a) (or discard if P(a) is refuted)
  q := prove not R(a, t a) by an approved route:
    kernel reduction + decide, a proof-producing evaluator,
    or a proved correspondence theorem for the evaluator used
  if q is Unknown: return none -- timeout is not evidence
  return some (a, p, q)

```

P7's prototype checks the general shape as `replay_sound`: a failed decidable observation refutes the universal specification. Shrinking must preserve typing and preconditions; an out-of-range index is not a smaller counterexample to a bounds-respecting program<sup>[P7]</sup>. Banks are keyed by the exact specification, environment, input types, and dictionaries; a counterexample under one dictionary selection is not applied to a problem whose printed function name merely coincides<sup>[P5,P8]</sup>.

### 18.3.2 The loop

The unified loop is P9's Algorithm 4 (bank  $E$  plus unresolved set  $W$ ) with P4's cheap bank check before verification, P5's four-valued verifier, P3's bank versioning, and P7's separation of proof and counterexample tasks<sup>[P3,P4,P5,P7,P9]</sup>. P1, P2, P6, and P8 give the same loop with a three-valued verifier and no explicit unresolved queue.

```

cegis(Q, stream S):
  E := typedExamples(Q); version := 0 -- bank and its version
  W := fairQueue() -- unresolved candidates
  while budget remains:
    t := choose fairly: next candidate from S consistent with E
    | resume (t, tier) from W with a larger tier
    if no source can produce anything:
      return Exhausted(declared grammar, bound) -- scoped to S only
    freeze t; typecheck t -- sec:joint-frozen
    for (a, p) in E: -- cheap necessary check
      if replay(t, a) = some cex: reject(t, cex); continue outer loop
    proofTask := prove Phi(t) under tier -- two RESUMABLE tasks,
    cexTask := seekCounterexample(t, Phi) -- scheduled fairly
    match first to complete (or both within the quantum):
    | Proof(p) -> submit (t, p) to the acceptance gate; return
    | Refutation(q), witness a ->

```

```

    cex := replay(t, a)
    reject(t, q)
    if cex = some c : E := E ++ [c]; version += 1
        repartition buckets; wake tasks stamped <= version
    | Refutation(q), no witness -> reject(t, q)      -- rejects t only
    | Counterexample(a) from cexTask ->
        match replay(t, a) with
        | some c -> reject(t, c); E := E ++ [c]; version += 1; repartition
        | none    -> record "model failed replay" without pruning authority
    | Unknown(r)      -> W.push(t, nextTier(tier), r)  -- keep alive
    | Failure(err)    -> report operational failure; do not infer falsehood
    return Unfinished(W, E) -- bottleneck attributed: construction, cex, or proof

```

Bank-consistent selection means the candidate generator uses  $E$  as a necessary finite condition (P3), ranks by failures on the growing bank (P2), and prefers small, diverse, branch-discriminating inputs when it proposes new tests (P2). Minimization is an optimization; a large checked witness remains valid evidence<sup>[P2]</sup>. Fairness is the property that a candidate passing every current test but resisting proof does not monopolize the search: candidate search and proof search have separate resumable frontiers linked by the candidate identity, and P5’s unit test `unknown_is_not_rejection` pins the required behavior (an unknown first candidate stays pending while a later candidate is certified)<sup>[P2,P5,P8]</sup>. In strict finite-grammar decision mode every candidate must eventually receive an exact verdict before exhaustion may be claimed; best-effort mode may stop with `Unknown` sooner, and the two modes never share an ambiguously named result<sup>[P8]</sup>.

**Proposition 18.3** (Finite CEGIS bound). *Let  $D$  be a finite input domain with  $|D| = N$ . If every candidate is checked exactly against all bank entries, verification either proves the finite-domain specification or returns a genuine violating input in  $D$ , and a satisfying candidate remains in the grammar, then at most  $N$  counterexample additions precede success<sup>[P8]</sup>. Dually, for a finite candidate set of size  $M$  with permanently retained counterexamples and bank-consistent selection, at most  $M$  rejections occur<sup>[P3]</sup>.*

*Proof.* A returned counterexample is not already in the bank, because the candidate satisfied every entry; each refuting round therefore adds a distinct input, and there are  $N$  of them. Once all inputs are present, bank consistency is the whole specification. For the dual bound, each counterexample permanently excludes the rejected candidate. Unknown verdicts satisfy neither argument and are not counted<sup>[P3,P8]</sup>.  $\square$

Neither bound limits the cost of one round; both are contracts on retention and bank updates, not runtime guarantees<sup>[P3,P8]</sup>.

### 18.3.3 Finite-domain promotion

When the input type has a certified complete enumeration  $L$ , a proof for every listed input is promoted by the checked theorem<sup>[P6]</sup>

```

theorem finiteValidation {I : Type} (samples : List I)
  (complete : forall i, i in samples) (R : I -> Prop)
  (checked : forall i, i in samples -> R i) : forall i, R i

```

(written here with ASCII binders; the source uses Lean notation). Completeness of  $L$  is the hypothesis that an arbitrary bounded generator does not supply: enumerating lists up to length four proves nothing about all lists<sup>[P1,P5,P6]</sup>. In practice the promotion is not even performed by the search: the search proposes a literal, and ordinary kernel `decide` proves the universal statement about that literal, so synthesis time and proof size are decoupled<sup>[P9]</sup>.

### 18.3.4 The finite Boolean CEGIS as implemented in Lean

Two packages implemented the finite case end to end in compiled Lean, without Mathlib, SMT, or `native_decide`<sup>[P8,P9]</sup>. The unified description follows P9’s dynamic-programming enumerator and P8’s fuel-indexed loop and truth-table targets.

```

-- Grammar (P9): e ::= x | y | neg e | conj e e | disj e e      (size = node count)
-- Grammar (P8): left | right | zero | one | neg | conj | disj | xor, size <= 4
enumerate(bound):
  -- exact-size DP; table[0] = []
  table[1] := leaves
  for n in 2..bound:
    layer := map neg (table[n-1])
    for left in 1..n-2: right := n-1-left
      for a in table[left], b in table[right]:
        layer := layer ++ [conj a b, disj a b (, xor a b)]
    table[n] := layer
  return table
  -- candidates := concat in size order

cegis(candidates, fuel, examples, proposals):
  if fuel = 0: return none
  match candidates.find? (fun e => examples.all (agrees e)) with
  | none   -> return none
  | some e ->
    match inputs.find? (fun p => not (agrees e p)) with
    | none   -> return some (e, examples, proposals ++ [e])
    | some p -> cegis(candidates, fuel-1, examples ++ [p], proposals ++ [e])

elab "synthesize_xor%" : term => do
  match (cegis ...).result with
  | none   -> throwError "finite CEGIS exhausted its declared grammar/budget"
  | some e -> return e.quote
  -- a literal syntax tree, not a proof
def synthesizedXor : BExpr := synthesize_xor%
theorem synthesizedXor_correct : forall x y, synthesizedXor.eval x y = (x != y) := by decide

```

The counts are exact. P9:  $a_1 = 2$ ,  $a_n = a_{n-1} + 2 \sum_{k=1}^{n-2} a_k a_{n-1-k}$ , so  $(a_0, \dots, a_8) = (0, 2, 2, 10, 26, 114, 402, 1722, 6890)$  and 9168 candidates up to eight nodes; fuel 5. P8:  $c_1 = 4$ ,  $c_n = c_{n-1} + 3 \sum_{i=1}^{n-2} c_i c_{n-1-i}$ , so  $(c_1, \dots, c_4) = (4, 4, 52, 148)$  and a pool of 208 syntax trees; fuel 5; all sixteen truth tables on two inputs solved with round counts  $(3, 4, 5, 3, 5, 3, 5, 5, 4, 5, 2, 5, 1, 5, 4, 2)$ , maximum five, consistent with four distinct counterexamples followed by success<sup>[P8,P9]</sup>. P8’s `synth_bool` elaborator additionally emits `Circuit.eval` (quoted circuit) as the implementation and proves `verify_sound`, `all_found`, and `all_correct` by `decide`; P9’s first attempt failed to prove termination of an enumerator written through a list combinator, and the final version uses the structurally decreasing fuel above<sup>[P8]</sup>. Both correctness theorems have empty axiom inventories; the generated function does not run CEGIS when called<sup>[P8,P9]</sup>.

## 18.4 Reversible observational buckets

**Definition 18.4** (Bucket). For the bank  $\mathcal{X} = \{x_1, \dots, x_m\}$  define  $\text{obs}_{\mathcal{X}}(t) = (t(x_1), \dots, t(x_m))$ . A bucket is a record

```

Bucket := { key : observation vector, bankVersion : Nat,
  active : Candidate,
  parked : list (Candidate | Recipe),
  lastRotated : Nat }
  -- the one being scheduled
  -- members or exact regeneration recipes

```

A recipe retains the original context, grammar choices, and frozen telescope; a promise to “enumerate later” is not a retention mechanism<sup>[P5]</sup>.

```

split(buckets, new observation x, version):
  for B in buckets affected by x (same context and domain stamps):
    groups := partition (B.active :: B.parked) by evaluation at x
    -- Unknown / timeout is its own group and is never merged
    -- with a returned value
    replace B by one bucket per group, key extended by that value,
    bankVersion := version; each new bucket's members become eligible
rotate(B, quantum): -- second liveness rule
  if B.active has been Unknown for k quanta and B.parked is nonempty:
    move B.active to parked; promote the next parked member
    -- a parked member may have a short certificate while the active one resists proof

```

Rotation is independent of splitting: proof-search diversity is a reason to reactivate members even when no distinguishing test arrives<sup>[P5]</sup>. Keys for open terms are stricter: parking is restricted to closed typed candidates, or to closures over the exact same frozen telescope with the same admissible instantiations and observed result family; a higher-order term may later meet a distinguishing context, a dependent term may live in a fiber whose index is unassigned, and an observation may depend on an unresolved dictionary<sup>[P5,P8]</sup>. Permanent merging requires a checked equality strong enough for every future use (for functions, pointwise equality plus the relevant extensionality principle), represented by a different API object than finite agreement<sup>[P1,P4,P5]</sup>.

Three finite ablations show what permanent merging loses<sup>[P1,P2,P3]</sup>:

- P1, `search_models.py`: candidates constant-true, identity, constant-false, not; initial test `true` gives bucket sizes (2, 2); keeping the first representative of each bucket yields no solution for identity after adding `false`, while retained derivations recover exactly `identity`.
- P2, `cegis.py`: identity and reverse agree on the two inputs of length at most one over  $\{0, 1\}$  and differ at (0, 1); permanently deleting reverse when identity is the first representative makes the later length-two counterexample unrecoverable.
- P3, `cegis.py`: all 354 length-viable fold steps agree on the initial bank  $\{\[], [0]\}$ , so a destructive control keeps only the first representative, identity; the counterexample  $[0, 1]$  rejects it and the control reports `finite_grammar_exhausted` although the grammar contains the correct reverse step. The working algorithm repartitions all 354 members on every new input.

P5's `behavior_lab.py` repeats the mechanism (empty versus identity split by the input  $((1), ())$ , bucket counts recomputed on each of three growing examples with all members retained).

## 18.5 Certified pruning

### 18.5.1 The soundness condition

Let  $s$  be a typed sketch,  $\Phi_s$  its valid path assumptions,  $\text{Comp}(s)$  its completions under the current grammar, and  $A(s, \Phi_s)$  an abstract summary with concretization  $\gamma$ . The condition that licenses pruning is<sup>[P3,P5,P8]</sup>

$$\{\mathcal{X}(e, \rho) : e \in \text{Comp}(s), \rho \models \Phi_s\} \subseteq \gamma(A(s, \Phi_s)),$$

together with an exhibited valid input  $\rho_0 \models \Phi_s$  and a defined observation for every completion at  $\rho_0$ ; otherwise an empty input domain makes the contract vacuous while the observation set is empty<sup>[P3]</sup>. Proposition 11.3 is the resulting statement; P6 checks its pointwise instance as

```

theorem safePruneAt {C I O : Type} (run : C -> I -> O)
  (completion : C -> Prop) (pre : I -> Prop) (contract over : I -> O -> Prop)
  (x : I) (hx : pre x)
  (sound : forall c, completion c -> over x (run c x))
  (exclude : forall y, over x y -> not (contract x y)) :
  not (exists c, completion c /\ forall z, pre z -> contract z (run c z))

```

whose proof is the four-line contradiction `exclude (run c x) (sound c hc) (hspec x hx)`<sup>[P6]</sup>. P5 states the same theorem with a necessary abstract condition  $N_\Phi$  in place of the pointwise `exclude`, and P2 states the target  $\forall e \in \text{Comp}(s), \neg \Phi(e)$  directly; the pointwise form is adopted because it is the one that was checked. The direction is not negotiable: an under-approximation may produce witnesses and counterexamples but never justifies absence<sup>[P1,P2,P4,P5]</sup>. A counterexample to one completion is not evidence about the family: `append xs ?tail` has the bad completion `?tail := []` and the good completion `?tail := ys` for the `append` contract, and P5’s test `counterexample_only_refutes_a_completion` pins this<sup>[P5]</sup>.

### 18.5.2 The domain-plugin protocol

```
DomainPlugin:
  recognize   : FrozenProblem -> Option DomainProblem      -- may decline
  reify       : Sketch -> Option AbstractTerm              -- typed; none = Unsupported
  interpret   : AbstractTerm -> Lean proposition about the exact candidate
  transfer    : per construct, an abstract rule + its assumptions
               (missing rule => top, never bottom)
  soundness   : theorem Beh(t) in gamma(A(t)) for every reified t
  necessity   : Option (theorem Phi(t) -> sigma(t) in N_Phi)
  checkPositive : proposal + certificate -> Lean proof or refusal
  replayNegative: model -> typed observation + refutation or refusal
  coverage    : Option (theorem relating the domain to the original grammar)
  reply       : Unsupported | RankingHint | NecessaryConstraint(proof)
               | Counterexample(validity evidence) | NoCompletion(certificate, grammar id)
```

This merges P7’s interface (`recognize`, `enumerateOrSolve`, `checkPositive`, `replayNegative`, `coverage`) with P3’s requirements (reification, interpretation, transfer rules, proof builders, refusal) and P4’s four proof-relevant boundaries (interpretation of syntax, meaning of summaries, justified transfer, reconstruction into the requested proposition)<sup>[P3,P4,P7]</sup>. A theorem about a callback’s length behavior must be instantiated with that exact callback occurrence and dictionary; it is not transferable to a similarly printed occurrence in another branch<sup>[P3]</sup>. The grammar identity in `NoCompletion` is part of the certificate’s applicability: a summary that assumes the committed prefix is preserved by `append` is void at a grammar position that may later delete or replace it<sup>[P5]</sup>.

### 18.5.3 The exact length abstraction

For the fold-step grammar  $s ::= \text{nil} \mid \text{single} \mid \text{tail} \mid \text{append}(s, s)$  with  $F_s([]) = []$  and  $F_s(x :: xs) = [s](x, F_s(xs))$ , assign<sup>[P3]</sup>

$$A(\text{nil}) = (0, 0), \quad A(\text{single}) = (0, 1), \quad A(\text{tail}) = (1, 0), \quad A(\text{append}(s, t)) = A(s) + A(t),$$

so that  $|[s](x, r)| = a_s|r| + b_s$  (`Step.summary_sound`, structural induction on  $s$ ; `append` adds lengths, so coefficient pairs add). If  $A(s) = (1, 1)$  then  $|F_s(xs)| = |xs|$  for every list (`Step.length_preserved`, induction on  $xs$ ). Conversely, if  $F_s$  preserves length on all lists of naturals then  $A(s) = (1, 1)$  (`Step.summary_necessary`): the input  $[0]$  forces  $b_s = 1$  and  $[0, 0]$  then forces  $a_s = 1$ . The condition is therefore equivalent to universal length preservation in this grammar, and necessity is what licenses pruning for a reversal contract: every term with  $A(s) \neq (1, 1)$  is a `NoCompletion` for the whole sketch family sharing that summary<sup>[P3]</sup>. Of the 3873 terms up to nine nodes ( $N_1 = 3$ ,  $N_3 = 9$ ,  $N_5 = 54$ ,  $N_7 = 405$ ,  $N_9 = 3402$ ;  $N_{2k+1} = C_k 3^{k+1}$ ), exactly 354 have summary  $(1, 1)$  and 3519 are eliminated before any counterexample round; the script cross-checks the summary rule on five recursive-result lengths for every term (19,365 concrete checks) as a regression, and the Lean theorems carry the universal argument<sup>[P3]</sup>. Length does not distinguish identity from reverse; that needs order-sensitive information<sup>[P3,P6,P7]</sup>.

P5’s laboratory uses the same abstraction with two list inputs,  $A(xs) = (1, 0)$ ,  $A(ys) = (0, 1)$ ,  $A(\text{reverse } t) = A(t)$ .

#### 18.5.4 The affine-fold separator theorem

For natural parameters, let  $F_{c,a,b,d}([]) = c$ ,  $F_{c,a,b,d}(x :: xs) = ax + bF_{c,a,b,d}(xs) + d$ , and  $S_{A,B,C}(xs) = A \sum xs + B|x| + C$ .

**Theorem 18.5** (Exact affine-fold certificate). *The following are equivalent<sup>[P4]</sup>: (1)  $F_{c,a,b,d} = S_{A,B,C}$  on every finite list; (2) the five equations  $c = C$ ,  $a = A$ ,  $bA = A$ ,  $bB = B$ ,  $bC + d = B + C$ ; (3) the two functions agree on the five lists  $[], [0], [1], [0, 0], [0, 1]$ .*

*Proof.* (1) implies (3) trivially. From (3):  $[]$  gives  $c = C$ ;  $[0]$  gives  $bC + d = B + C$ ;  $[1]$  gives  $a + bC + d = A + B + C$ , hence  $a = A$ ;  $[0, 0]$  gives  $b(B + C) + d = 2B + C$ , hence  $bB = B$ ;  $[0, 1]$  gives  $b(A + B + C) + d = A + 2B + C$ , hence  $bA = A$ . From (2), induction on the list:  $F(x :: xs) = Ax + (bA) \sum xs + (bB)|xs| + (bC + d) = A \sum(x :: xs) + B|x :: xs| + C$ <sup>[P4]</sup>.  $\square$

Direction (2) $\Rightarrow$ (1) is the Lean theorem `Leant2Affine.cerify`; the other directions are proved on paper and cross-checked exhaustively on the finite parameter domain (2187 candidate–specification pairs: 31 satisfy the equations, 2156 have an explicit separating list)<sup>[P4]</sup>. The theorem gives this domain a *complete* counterexample family: a candidate that fails its universal contract differs on one of the five lists, so the loop obtains a real separating input instead of treating a failed proof as evidence<sup>[P4]</sup>. Each accepted candidate is reconstructed as `apply certify <;> decide`; the generic theorem and all twenty-seven instances report only `propext` and `Quot.sound`<sup>[P4]</sup>. P7’s certifier is the same pattern for  $f_{a,b}(n) = an + b$  against  $f(0) = 1 \wedge \forall n, f(n+1) = f(n) + 2$ : `affine_spec_iff` proves  $\text{Spec}(f_{a,b}) \leftrightarrow (a, b) = (2, 1)$  and `certify` returns `Option {c : Affine // Spec c.eval}`, a proof-carrying value rather than a Boolean<sup>[P7]</sup>.

#### 18.5.5 Target-directed invariant discovery

```
discoverInvariant(schema, Post, recursiveCallSubstitution):
  V := parameters changed by the recursive-call substitution      -- accumulators
  template := Post generalized over V                               -- start from the target
  if the fold returns a function: quantify over its future argument
  if the proof needs output and an auxiliary measure:
    propose a paired carrier and a relational template
  for each instantiation of the finite template grammar
    (affine length relations, conjunctions of bounds, elementwise
     predicates, permutation relations, accumulator equations):
    obligations := [ init  : template holds at the initial state,
                   step   : each branch preserves it under its capabilities,
                   final  : template at the initial state implies Post ]
  if any bank counterexample refutes the template: discard it
  if all obligations proved: return (template, proofs)
  else keep the template unresolved                                -- a failed proof is not a refutation
```

Starting from the postcondition and generalizing only where recursion requires it couples the schema, the induction hypotheses, and the specification, instead of enumerating unrelated invariants from a global language<sup>[P8]</sup>. P7 gives the same four stages (extract recurring subexpressions, propose a helper, form candidates from the postcondition and library relations, validate preservation) and P4 the accumulator rule ( $g(xs, acc) = \text{reverse}(xs) ++ acc$  for a reverse helper, generalizing  $acc$  before induction)<sup>[P4,P7,P9]</sup>. A numeric parameter vector fitting the examples is not accepted until the base and step obligations are proved<sup>[P8]</sup>.

### 18.5.6 Abduction of nogoods

```
learnNogood(failedBranch):
  match cause:
  | CertifiedReject(cert, grammarId) | CheckedContradiction(indexAssignment):
    slice := smallest dependency set the certificate mentions
              (constructor choices, instantiations, path assumptions)
    store Nogood(slice, request, contract, instances, epoch, grammarId)
  | BudgetExhausted(method) -> store Penalty(method, budget); retry later
  | SolverUnsatCore(core)   -> store as a PROPOSED slice; promote only after
                              a Lean proof of the corresponding implication
```

A nogood identifies the request, contract, instance selections, environment version, and grammar scope; generalizing beyond the justified slice requires another proof<sup>[P3,P4,P5]</sup>.

## 18.6 Solver bridges

External solvers have three distinct uses with three validation paths<sup>[P3]</sup>: proposing choices for a finite sketch (decode the model into a typed plan and check it), proposing counterexample inputs (replay the decoded model against the exact candidate through `replay`), and proving a theory obligation (reconstruct through a certificate checker or a proof-producing integration). A single “solver succeeded” Boolean is inadequate<sup>[P3,P4]</sup>.

```
typedSketchQuery(sketch, examples | abstract semantics):
  selectors := one finite-domain variable per grammar choice
  for each dependent hole whose type depends on an earlier witness w:
    branch-expanded: for each alternative of w, emit the correctly
                      specialized child grammar guarded by w's selector
    native-first:    ask the solver only for w; let Lean elaborate the
                      residual dependent problem; issue a later query for its completion
    -- never one untyped multiplexer variable with the type attached later
  add theory constraints with the adapter's own semantics:
    Nat as Int with nonnegativity; truncated subtraction guarded;
    bitvectors modular; division/remainder with Lean's totalized cases;
    arrays with bounds and extensionality; user functions uninterpreted
  decodeModel(m): choices -> plan; replay in a fresh native branch; typecheck
  unsat: RankingHint immediately; NoCompletion only after the encoding theorem
         and a checked unsatisfiability certificate are accepted
```

Branch expansion exploits larger simultaneous constraints; native-first avoids an exponential family of heterogeneous child grammars; both reconstruct the same native term<sup>[P5]</sup>. `unsat` on an under-approximate grammar excludes only that grammar; `unsat` on a sound over-approximation supports stronger conclusions only after its encoding theorem and certificate are accepted; `unknown`, timeouts, unsupported proof rules, and malformed models remain distinct outcomes<sup>[P3,P6]</sup>. A solver may not invent universe expressions, inhabitants of unknown higher-order types, or an element of an empty type to make a counterexample convenient<sup>[P5]</sup>. The staged affine use is: enumerate a coefficient-bounded template, ask the solver for coefficients, substitute them into a native expression, prove the law in Lean; exhaustion of the bounds is template exhaustion, not impossibility<sup>[P9]</sup>. The no-solver profile remains fully functional: every experiment below ran without SMT<sup>[P4,P7,P8,P9]</sup>.

## 18.7 Worked traces

All numbers are those recorded in the source packages; none was re-executed for this document (Section 3.3).

**Affine fold for list summation (P1).** Grammar:  $F_{b,e,a,d}([]) = b$ ,  $F_{b,e,a,d}(x :: xs) = ex + aF(xs) + d$  with  $b, e, a, d \in \{0, 1, 2\}$ , 81 sketches in lexicographic order. Finite verifier: the  $1 + 3 + 9 + 27 + 81 = 121$  lists of length at most four over  $\{0, 1, 2\}$ , checked against `sum`<sup>[P1]</sup>.

| Proposed $(b, e, a, d)$ | Survivors before check | Counterexample     | Result           |
|-------------------------|------------------------|--------------------|------------------|
| $(0, 0, 0, 0)$          | 81                     | $[1]$              | reject           |
| $(0, 0, 0, 1)$          | 11                     | $[0]$              | reject           |
| $(0, 1, 0, 0)$          | 5                      | $[0, 1]$           | reject           |
| $(0, 1, 1, 0)$          | 1                      | none in the corpus | submit for proof |

The survivor count is recomputed over the whole grammar against the accumulated tests, so  $81 \rightarrow 11 \rightarrow 5 \rightarrow 1$ ; the selected sketch is the unique full-corpus survivor and is then proved equal to `List.sum` for every list by structural induction in `RecursionCertificates.lean`. The 121 tests select; the theorem certifies<sup>[P1]</sup>.

**Reverse as a left-fold step (P2).** Skeleton `xs.foldl step []`; step grammar `acc`, `[]`, `[x]`, and binary append, enumerated by exact node count up to seven nodes: 471 step expressions. Verifier: the 40 lists of length at most three over  $\{-1, 0, 1\}$ ; initial bank  $\{[]\}$ <sup>[P2]</sup>.

| Round | Selected step           | Checks this round | Counterexample      |
|-------|-------------------------|-------------------|---------------------|
| 1     | <code>acc</code>        | 1                 | $[-1]$              |
| 2     | <code>[x]</code>        | 3                 | $[-1, -1]$          |
| 3     | <code>acc ++ [x]</code> | 6                 | $[-1, 0]$           |
| 4     | <code>[x] ++ acc</code> | 10                | none: passes all 40 |

The article records the four successive selections and 20 candidate checks including revisits; the per-round checks and the three counterexamples above follow the script’s deterministic candidate and corpus order (`acc`, `[]`, `[x]`, then appends by size; lists by length, then lexicographically from  $-1$ ) and sum to the recorded 20, but the lists themselves are stated in the package’s `cegis-results.json` rather than in its article text. The result is exported as `xs.foldl (fun acc x => [x] ++ acc) []` and proved universally in `ReverseCertificate.lean` through the accumulator invariant, specialized to the empty accumulator<sup>[P2]</sup>.

**Length-guided reverse (P3).** Grammar as in the length abstraction above: 3873 terms up to nine nodes, ordered by size with identity before reverse; initial bank  $\{[], [0]\}$ ; verifier the same 121 lists<sup>[P3]</sup>.

| Search        | Round | Consistent | Selected step                   | Outcome                 |
|---------------|-------|------------|---------------------------------|-------------------------|
| plain         | 1     | 1309       | <code>single</code>             | counterexample $[0, 0]$ |
| plain         | 2     | 354        | <code>append single tail</code> | counterexample $[0, 1]$ |
| plain         | 3     | 177        | <code>append tail single</code> | passes all 121          |
| length-guided | 1     | 354        | <code>append single tail</code> | counterexample $[0, 1]$ |
| length-guided | 2     | 177        | <code>append tail single</code> | passes all 121          |

The  $(1, 1)$  summary removes the spurious `single` family before any counterexample round, saving one round; it cannot separate identity from reverse. The selected `AST Step.append Step.tail Step.single` is proved equal to `List.reverse` at every universe level by `selected_correct`<sup>[P3]</sup>.

**Affine certifier (P7).** Grammar  $f_{a,b}(n) = an + b$ ,  $a, b \in \{0, \dots, 4\}$ , 25 candidates in lexicographic order; bank initially empty; observations `base` ( $f(0) = 1$ ) and `step 0` ( $f(1) = f(0) + 2$ ). Candidate  $(0, 0)$  fails `base`; the next candidate surviving `base`,  $(0, 1)$ , fails `step 0`; thereafter

every candidate up to the winner (2,1) is rejected by the two stored observations before the certifier runs. The Lean theorem `run_counts`, proved by `decide`, fixes the executed counts: 12 candidates considered, 3 certifier invocations, 9 pruned by the bank, 0 inconclusive; an unfiltered baseline would invoke the certifier 12 times. The winner carries its universal proof in its type<sup>[P7]</sup>.

**Boolean exclusive-or (P8, P9).** P9, grammar without a primitive xor, 9168 candidates up to eight nodes, empty initial bank:

| Round | Proposal                             | New counterexample $(x, y)$ |
|-------|--------------------------------------|-----------------------------|
| 1     | $x$                                  | $(0, 1)$                    |
| 2     | $y$                                  | $(1, 0)$                    |
| 3     | $x \vee y$                           | $(1, 1)$                    |
| 4     | $\neg(x \wedge y)$                   | $(0, 0)$                    |
| 5     | $(x \vee y) \wedge \neg(x \wedge y)$ | none                        |

Five proposals, four counterexamples, an eight-node result, quoted and proved by `decide` with an empty axiom inventory; the independent Python enumerator reproduced the counts, the trace, and the absence of a smaller solution in the grammar<sup>[P9]</sup>. P8, whose grammar contains xor and whose pool has 208 trees, reports five rounds for mask 6 and the implementation `Circuit.eval (Circuit.left.xor Circuit.right)`; following its deterministic order by hand (leaves `left`, `right`, `zero`, `one`, then negations, then binary nodes with inputs checked in the order  $(0, 0), (0, 1), (1, 0), (1, 1)$ ) gives the proposals `left`, `right`, `one`, `left || right`, `left xor right` with counterexamples  $(0, 1), (1, 0), (0, 0), (1, 1)$ . That reconstruction is consistent with the recorded round count but is not itself recorded in P8<sup>[P8]</sup>.

*Remark 18.6.* Across the five traces the pattern is identical and is the pattern this section specifies: an untrusted enumerator or solver proposes; a finite bank refutes cheaply and is only ever extended by replayed counterexamples; a certificate, a `decide` on a quoted literal, or a structural induction proof supplies the universal claim; and the count of candidates examined is reported as an operation count, never as a speedup (Section 13). None of the traces exercises the unknown branch or the unresolved queue; those remain design requirements without prototype evidence<sup>[P9]</sup>.

## 19 Algorithms: scheduling, caching, and completeness

Section 6 fixes the lane portfolio, the pressure and priority formulas, the reserved reference share, and the three resource controls; Section 12 fixes what a cache key contains. This section supplies the procedures behind those design statements: the task record and the outer loop, the wake-up index over the obligation graph of Section 4.3, the two scheduling layers and the diagonal schedule, the cost-indexed reference enumerator, cost-vector ranking, the verification queue, cache lookup and storage, the non-refundable ledger of Decision 2.8, and the precise statements about the search that the engine commits to. Where the nine proposals give different pseudocode for the same step, the most fully specified version is taken and the alternatives are named in one sentence.

### 19.1 Tasks, cursors, and the step interface

**Definition 19.1** (Task). A *task* is the unit the scheduler dispatches. Merging the job record of P5, the cursor frame of P8, and the tier tuple of P9<sup>[P5,P8,P9]</sup>, it contains:

- *origin*: the query generation and the branch identifier that owns its metavariables (Invariant 4.3);

- *lane*: which lane policy produced it (Section 6);
- *cursor frame*: the focused obligation, its remaining sibling obligations under the same substitution, a saved branch state or a replay plan that reconstructs it, the rule family in use, the index of the next untried alternative, and the remaining cost allowance of this frame;
- *tier*: the tuple  $(i, s, r, v)$  of lane number, expression-cost bound, recursion-scheme bound, and verification allowance level under which the task is admitted;
- *priority components*: charged cost  $g$ , the estimates  $\hat{h}$  and  $\hat{p}$ , duplication and relevance features, and an age counter;
- *spent*: the work already charged to this task, broken down by counter (Section 19.9).

A task never stores a closure over a live metavariable context; a cold task stores a replay plan and is rematerialized from the nearest checkpoint<sup>[P1,P5,P9]</sup>.

Every lane implements the bounded step of Section 6. The result algebra merges the outcome lists of P2, P4, P7, P8, and P9; the three distinct meanings of “blocked” from P2 are kept as three constructors because the scheduler treats them differently<sup>[P2,P4,P7,P8,P9]</sup>.

```
step : Origin -> Task -> Quantum -> WorkerM StepResult

StepResult :=
| successors (children : Array Task)           -- new frames, same origin
| yielded   (cursor : Task) (work : Charge)    -- allowance used up, resumable
| candidate (t : Candidate) (obligations : Array Obligation)
| checkedPrune (cert : Checked)                -- exact completion set is empty
| blockedOn  (deps : Array MVarId) (cursor : Task)
| unsupported (feature : String) (cursor : Task)
| exhausted  (scope : GrammarFrontier) (cert : Option Checked)
| failed     (reason : Diagnostic)             -- this alternative only
| interrupted (reason : Interrupt) (resumable : Option Task)
```

A `successors` result is the only way a task creates children, and a `candidate` is not a success: it enters the verification queue of Section 19.7 and only the acceptance gate (Section 13) can turn it into an outcome. P1’s `Yield/Blocked/Finished` and P4’s `alternative/suspended/exhausted/unsupported` are coarser partitions of the same set.

## 19.2 The outer loop

The outer loop is P2’s `solve` with P8’s event dispatch and P5’s explicit job records<sup>[P2,P5,P8]</sup>. Its two invariants are that every step runs inside a transaction (Section 5) and that the charge for a step is recorded before the transaction can roll back.

```
solve(query):
  Q      := prepareAndFreeze(query)           -- immutable; Section 3
  ledger := newLedger(Q)                     -- monotone; never restored
  lanes  := initialLanes(Q)                  -- weights from query shape
  queues := forEach lane: (heuristicQueue, fifoStream)
  unresolved := emptyVerificationQueue()
  enqueue(queues, rootTasks(Q))
  while not ledger.deadlineReached() and (hasWork(queues) or hasWork(unresolved)):
    lane := nextLane(lanes, ledger)           -- weighted round-robin, Alg. below
    task := if lane = verification then unresolved.pop()
             else pickWithinLane(queues[lane]) -- best-first or reserved FIFO
    charge := ledger.beginNonrefundable(task)
    outcome := runTransaction(task.frame.state, fun s => step(Q.origin, task, quantum(lane)))
    charge.finish(outcome.work)               -- charged even if s was restored
    match outcome with
    | successors cs => forEach c in cs: enqueue(queues[c.lane], c)
    | yielded c w   => requeue(queues[lane], age(c))
```

```

| candidate t obs    => unresolved.push(newVerificationTask(t, obs, v := 0))
| checkedPrune cert  => closeExactBranch(task, cert); ledger.recordPrune(cert)
| blockedOn deps c   => park(c, deps)                                -- indexed wake-up, Alg. wake
| unsupported f c     => reroute(c, lanesSupporting(f)) or ledger.recordUnsupported(f)
| exhausted sc cert  => ledger.recordScope(lane, sc, cert)
| failed r           => ledger.recordLocalFailure(task, r); learnNogood?(task, r)
| interrupted r res  => if r.isCancellation then return stopWithUnknown(ledger)
                     else if res.isSome then requeue(queues[lane], res.get)

adaptWeights(lanes, lane, outcome)
drainAcceptances(unresolved, Q, ledger)    -- verified candidates to the gate
return outcomes(ledger, unresolved, queues) -- result algebra: never a bare failure

```

The quantum is not “one candidate”: it bounds syntactic expansions, native reduction and unification steps, proof-search fuel, and heartbeats together, so that provider retrieval, motive enumeration, and proof calls cannot monopolize the request inside a supposedly single step<sup>[P2,P6,P8]</sup>. A native primitive that cannot be suspended internally (an arbitrary tactic invocation) runs under a per-attempt limit; on timeout its checkpoint is restored and the attempt is either retired or re-enqueued at a larger allowance in a later tier, and the retried work is charged again<sup>[P5,P9]</sup>. P1’s `Rule.expand(branch, obligation, quantum)` and P4’s `Rule.prepare/Rule.step` pair are the same interface with the frame split out; the frame representation of P8 is retained because it makes the sibling list and the next alternative explicit, which the depth-first prototypes of P2, P3, P6, and P7 keep only on the host call stack.

### 19.3 Obligation selection, assignment versions, and wake-ups

Within a branch, the next obligation is the ready obligation with the highest pressure (Section 6), corrected by age. An obligation is *ready* when the variables its chosen rule needs are sufficiently instantiated, not when every metavariable in its type has disappeared<sup>[P8]</sup>. The scheduler maintains, per branch, a watcher index from each unresolved metavariable to the obligations whose target, telescope, contract, or residual mentions it (P4), an assignment version per metavariable (P6), and the strongly connected components of the dependency graph (P2, P6, P8)<sup>[P2,P4,P6,P8]</sup>.

```

selectObligation(branch):
  ready := [h | h in branch.pending, isReady(h), not parked(h)]
  clusters := sccs(branch.dependencyGraph restricted to ready)
  -- a coupled cluster is scheduled as one unit until a choice breaks the cycle
  score(C) := max_{h in C} (pressure(h) + alpha * age(h))
  return argmax_{C in clusters} score(C)

onAssign(branch, m : MVarId, value):
  branch.version[m] += 1
  for h in branch.watchers[m]:
    reclassify(h)          -- exactly the affected obligations
    -- flex-rigid became pattern? instance params known? contract reduces?
    if parked(h) and all deps of h now assigned or reclassified: unpark(h)
  for m' in freshMVarsIn(value): branch.watchers[m'] := obligations mentioning m'

```

Re-running every blocked solver after every assignment defeats dependency tracking; reclassification touches only the watcher set of the assigned metavariable<sup>[P1,P4]</sup>. A cyclic dependency is not diagnosed as inconsistency: the cluster is branched over as a unit, typically by choosing a constructor or a polymorphic instantiation first<sup>[P8]</sup>. The aging term  $\alpha \cdot \text{age}(h)$  grows by one unit each time the branch is stepped and  $h$  is not selected; it prevents “informative first” from starving a hard but necessary obligation<sup>[P3]</sup>. Proof-first ordering (a cheap proof field that determines a data field is selected before the data field) is a consequence of the pressure formula, since such a field has a high `dependentUses` count<sup>[P5,P7,P8]</sup>.

## 19.4 Two layers: heuristic best-first and the fair reference layer

Section 6 states that fairness is structural. Concretely, the scheduler runs deterministic weighted round-robin over lane families with best-first order inside a lane, reserves a fixed share of charged work for the reference lane and for the verification queue, and complements each tier’s heuristic queue with a FIFO stream<sup>[P2,P5,P7,P8,P9]</sup>. Lane-level fairness is measured in charged work, not in queue entries, so a lane that generates many internal plans cannot dilute another lane’s share<sup>[P8]</sup>.

```
-- credits are charged work units; weights w[L] sum to 1 over enabled lanes
nextLane(lanes, ledger):
  for L in lanes: credit[L] += w[L] * quantumSize
  runnable := [L | L in lanes, hasWork(L)]
  L := argmax_{L in runnable} credit[L] (deterministic tie: lane number)
  credit[L] -= quantumSize
  return L

pickWithinLane(queue L):
  -- every reserveEvery-th pick from a lane comes from its FIFO stream
  if L.picks mod reserveEvery = 0 then task := L.fifo.popFront()
  else task := L.heuristic.popMin(s) -- priority s(j) of the architecture section
  L.picks += 1
  return task
```

The reserved shares are engineering defaults: one quantum in eight for the reference lane (P5), one in eight for the verification queue, and no theoretical claim depends on either ratio<sup>[P5]</sup>. The FIFO stream is P9’s device for making the fairness claim honest within a tier; the alternative in P9 is to bound the number of heuristic overtakes per task, which gives the same guarantee with a different constant.

**Adaptive lane weights.** Initial weights come from the query shape (Section 6.3). The progress rule is<sup>[P2,P4]</sup>:

```
adaptWeights(lanes, L, outcome):
  if outcome in {successors, candidate, checkedPrune}: idle[L] := 0
  else idle[L] += 1
  if idle[L] >= k: -- k quanta with nothing to show
    w[L] := max(w[L] / 2, reserve[L]); idle[L] := 0
    renormalize(w) over lanes with w > reserve
  -- reserve[reference] > 0 and reserve[verification] > 0 always
```

Halving is subject to the reserve, so no enabled unfinished lane is ever starved; P4 states the same requirement as “weighted round-robin that revisits every enabled unfinished lane.”

**The diagonal schedule.** The reference lane admits tasks by tier  $(i, s, r, v)$  and visits tiers in increasing  $i + s + r + v$  with deterministic tie-breaking (P9); P4 states the same schedule as a joint increase of construction and proof bounds so that an expensive proof of a small program is not postponed forever behind larger programs with trivial obligations, and P1’s Algorithm 7 is the special case with a single bound  $k$ <sup>[P1,P4,P9]</sup>.

```
referenceLane.step(Q, task, quantum):
  (i, s, r, v) := task.tier
  if task.cursor = none: -- open the tier's finite grammar
    task.cursor := N(Q.telescope, Q.target, s, r) -- Section alg:enumerator
  match task.cursor.next(quantum) with
  | some term => recheck(term); return candidate(term, obligations(term)) with v
  | exhausted => ledger.recordScope(reference, tier := (i,s,r,v))
    enqueue nextTiers(i, s, r, v) -- all tuples of sum+1 not yet opened
```

```

        return exhausted((i,s,r,v), none)
    | yielded c => return yielded(c)

```

Queue caps are operational. When a fast lane drops a state because a queue is full, the drop is recorded as an incompleteness event with the lane, tier, and reason, or the state is spilled as a compact replay recipe; a dropped state is never a refutation<sup>[P2,P7]</sup>.

**How budgets and deadlines are derived (design).** Decision 1.1 leaves no per-query knobs, so the envelope is derived, and the proposals are consistent on the shape of the derivation but supply no measured constants; what follows is the design and is marked as such. The logical budget is not a fixed number: it is the union of the tiers the diagonal schedule has opened, and it grows while the run continues<sup>[P4,P9]</sup>. The wall-clock deadline for an interactive request is computed at preparation time from the query shape (telescope length, number of obligations after preparation, presence of a contract, and whether recursion is admitted) and is reported in the receipt together with the actual charges, not a configured maximum<sup>[P3]</sup>. The stopping rule of an interactive run is: stop after the first certified result once no queued task in the currently open tiers has a scalarized cost below the accepted candidate’s cost, so that “best under this schedule” is literally true; otherwise continue until the deadline or exhaustion<sup>[P4,P8]</sup>. An audit run has no early stop; it records every deliberate restriction and terminates only by the envelope (Section 19.10). Both wall-clock and reproducible work budgets are reported because they answer different questions<sup>[P2]</sup>.

## 19.5 The cost-indexed reference enumerator

The reference lane’s grammar is P2’s Appendix B enumerator  $N(\mathcal{G}, T, s)$ , which constructs normal forms at exact cost  $s$  under a fixed finite type/index grammar and provider inventory at the current widening level<sup>[P2]</sup>. P1 and P4 give the same construction as “a finite grammar at every cost bound” with explicit charges; P6 lists the same bounded dimensions (constructors, binder-type complexity, universe-expression complexity, spine-prefix choices, recursion schemes, proof fuel)<sup>[P1,P4,P6]</sup>.

```

-- N(Gamma, T, s, r): admitted terms of type T in context Gamma at cost exactly s,
-- with recursion-scheme bound r. Every yielded term is rechecked natively.
N(Gamma, T, s, r):
  if s = 0: return empty
  T' := whnfBounded(T)
  if T' is (x : A) -> B and etaNormalPolicy requires intro:
    yield fun x => e for e in N(Gamma, x:A, B, s - 1, r) -- lambda costs 1
  for head in inventory(Gamma) ordered deterministically: -- locals, ctors, providers
    (Delta, Tres) := telescopeOf(head)
    for prefix in admittedSpinePrefixes(Delta): -- partial applications
      if unifyResult(Tres[prefix], T') fails: continue -- transactional
      c0 := headCost(head)
      for partition (s_1..s_n) of (s - c0) over the n explicit args of prefix:
        yield head a_1 .. a_n for (a_1..a_n) in argsSeq(Gamma, Delta, prefix, [s_1..s_n])
  for plan in casePlans(Gamma, T', s, r) ++ recursionPlans(Gamma, T', s, r):
    yield plan.body for plan.body in N(Gamma + plan.hypotheses, T', s - plan.cost, r - plan.schemes)
  for lit in literalsOfSize(T', s): yield lit -- size-encoded literals

-- transactional sequential argument generator: earlier arguments change the
-- types of later ones, and a later failure resumes the earlier generator
argsSeq(Gamma, Delta, prefix, costs):
  gen(k, state):
    if k > n: yield state.args; return
    for a_k in N(Gamma, Delta_k[state.args], costs[k], r) under saveState:

```

```

    if assignArg(state, k, a_k) fails: restore; continue
    gen(k + 1, state)
    restore
    gen(1, emptyState)

```

The design rules that make each level finite are: a lambda consumes one unit; an application consumes its head cost and distributes the remainder over its arguments by a finite partition; constructor and case plans use analogous finite partitions; literals are enumerated by an explicit size encoding (bit length), never “any natural number at unit cost”; a recursion scheme that invents auxiliary parameters pays for their description size; a chosen eta-normal convention avoids counting equivalent lambda expansions repeatedly; and recursion enters as a finite set of priced plans whose branch bodies are enumerated under the plan’s capabilities (Section 17)<sup>[P1,P2,P4]</sup>. The apparent product over argument choices is implemented as a sequential generator inside one transaction, because an earlier argument changes the types of later ones<sup>[P2]</sup>.

The enumerator is fair because at fixed  $(s, r)$  and a fixed inventory it is finite, its cursor yields after every quantum (it is an explicit cursor, not a lazy stream), and the diagonal schedule opens every tier; combined with the verification queue it dovetails construction and checking allowances<sup>[P2,P3,P9]</sup>. Every claimed exhaustion result states exactly which dimensions were fixed: syntax sizes, literal sizes, type/motive sizes, provider set, recursion bound, and proof regime<sup>[P2]</sup>.

## 19.6 Cost vectors, profiles, and the Pareto frontier

The cost vector of Section 6.4 is used in three places: as the scalarized queue priority  $g(j)$  of a task, as the ordering of accepted results under a profile, and as the basis of the Pareto frontier shown to the user<sup>[P2,P3,P6,P7,P8]</sup>.

```

-- c(t) = (violations, size, elims, recShape, transports, nonlocal, proofSize, runtime)
scalar(c, profile) :=
  if c.violations > 0 then +infinity          -- admission constraint, not a penalty
  else sum_k profile.weight[k] * c[k]        -- profile: smallSource | fastRuntime |
    fewDeps

frontier.insert(t):
  if exists u in frontier with checkedEquivalent(u, t): merge evidence; return
  if exists u in frontier with dominates(u, t): return
  frontier := [u | u in frontier, not dominates(t, u)] ++ [t]
  if |frontier| > cap: drop the element with the worst scalar(., profile) among
    those that are observationally indistinct from a kept one

```

`dominates(u, t)` is componentwise  $\leq$  with at least one strict component on the vector without the violations entry. Diversity is counted over native terms modulo checked equivalence, never over printed strings: two spellings of one candidate earn one slot, and two candidates that agree on all current observations remain distinct until a checked equality merges them<sup>[P9]</sup>. The profile changes ranking only; the correctness predicate is unchanged<sup>[P2]</sup>. “Use every input” is a soft feature that applies only when the contract cannot disambiguate<sup>[P2,P9]</sup>.

**Decision 19.2** (When “minimal” may be said). A result is labeled *minimal* only when the ledger records that every tier with scalar cost below the result’s cost under the declared profile has been exhausted by the reference lane and every candidate in those tiers has been exactly rejected or checked-equivalent to the result. Otherwise the label is “lowest cost found under this schedule”<sup>[P6,P8,P9]</sup>.

## 19.7 Verification scheduling

Candidate generation and verification are two unbounded searches; a scheduler that fairly enumerates programs but discards each after one fixed failed proof attempt is not relatively complete for certified synthesis<sup>[P7]</sup>. The verification queue therefore holds *verification tasks* with their own allowance level  $v$ , separate proof and counterexample tasks per candidate, and wake-up indexing by lemma and by example bank<sup>[P7,P8,P9]</sup>.

```

newVerificationTask(t, obligations, v):
  return { candidate := t, proofTask := (obligations, allowance(v)),
          cexTask := (t, exampleBank.version), level := v }

verification.step(task, quantum):
  -- counterexample search first: it is cheap and can eliminate many waiters
  match cexTask.run(quantum) with
  | counterexample x cert => exampleBank.addChecked(x, cert) -- wakes all waiters
                          return refutedCandidate(t, cert)
  | none | inconclusive => ()
  match proofTask.run(quantum) with
  | proved p => return toAcceptanceGate(t, p)
  | disproved q => return refutedCandidate(t, q)
  | reduced goal' recon => proofTask := (goal', allowance(v)); requeue
  | unknown reason => task.level := v + 1
                      requeue at the tier (i, s, r, v + 1) -- never dropped

onNewCounterexample(x):
  -- wake-up, not restart
  for task in unresolved: if evaluates(task.candidate, x) violates: retire(task, x)
onNewLemma(l):
  for task in unresolved: if mentionsHead(task.proofTask.goal, l.head): reprioritize(task)

```

Quotas have the following meaning<sup>[P8]</sup>: a *successful-result quota* (how many verified candidates the frontend wants) is not a *raw-candidate quota*. A rejected candidate does not shrink the candidate window, the search continues while the work budget remains, and reaching a user-visible cap produces `budgetExhausted` with the reason `candidateBudget`, keeping the distinction between an algorithmic impossibility and a resource policy<sup>[P7,P8]</sup>. Duplicates and rejected candidates still consume work and do not authorize a refill loop<sup>[P8]</sup>. An independent proof (one that cannot assign a data hole) may run in parallel; a proof that may determine a data hole stays inside the branch transaction (Section 5.3)<sup>[P7]</sup>.

## 19.8 Memoization and equivalence

Section 12 lists the key fields. The procedures are<sup>[P1,P2,P4,P5,P7,P8,P9]</sup>:

```

structure CacheKey where
  envEpoch      : EnvironmentId      -- imports, instances, provider registrations
  providerPolicy : PolicyId
  telescope      : CanonicalTelescope -- dependency order; keeps let-values, instances
  target         : ClosedExpr
  contract       : Option ClosedExpr
  transparency   : TransparencyMode
  universes      : UniverseConstraints
  assignments    : AssignmentDigest   -- reachable shared-metavariable state
  grammar        : GrammarId          -- lane, tier bounds, recursion policy
  domainVersions : Array (Name x Nat)
  observationGen  : Nat                -- example-bank generation (behavioral keys only)

inductive NegativeScope where
  | refutation (q : Checked) -- semantic: no completion exists
  | exhausted (grammar : GrammarId) (tier : Tier) -- bounded: this grammar, this bound

```

```

| operational (method : Name) (budget : Charge) -- a miss; suppresses only identical
  retries

lookup(key, ctx):
  for entry in index[hash(key)]: -- hash is an index, not authority
    if not exactlyEqual(entry.key, key): continue
    match entry with
    | positive tmpl => (e, p) := instantiate(tmpl, freshMVars, freshUniverses)
                      emb := contextEmbedding(tmpl.telescope, ctx) -- explicit, checked
                      if check(emb(e) : key.target) and check(emb(p) : contract): return hit(e
, p)
    | negative sc   => if sc covers the requested grammar and tier: return miss(sc)
  return none

store(key, result):
  match result with
  | solved (e, p)   => index.add(key, positive(abstractOverTelescope(e, p, key.telescope)))
  | refuted q       => index.add(key, negative(refutation q))
  | exhaustedTier tr => index.add(key, negative(exhausted(key.grammar, tr)))
  | timeout m budget => index.add(key, negative(operational(m, budget))) -- short-lived

```

A positive entry is a closed template with explicit parameters, never a bare expression with old local identifiers; it is instantiated and rechecked in the destination context through the embedding<sup>[P1,P2,P4]</sup>. A negative entry at fuel twenty does not suppress a search at fuel one hundred, a failure with one provider set does not exclude another, and a timeout is never stored as uninhabitability<sup>[P2,P3,P4]</sup>. Open components (with residual constraints) are cached within one query only; persistent storage of raw metavariable states is not done<sup>[P6]</sup>. Environment epochs are persistent revisions: a change to an import, local instance, component registration, or semantic option invalidates the affected keys selectively<sup>[P9]</sup>.

**Nogoods and conflict learning.** Within one request, a branch that fails because a specific constructor choice and a specific index assignment imply a checked contradiction records that small assignment set as a nogood, tagged with request, contract, instance selections, epoch, and grammar scope. The slice stored is the smallest justified one: the dependency support of the contradiction, not the whole branch. A proof tactic's failure or a budget exhaustion has no such logical status and may only lower priority under the same budget<sup>[P4,P9]</sup>.

```

learnNogood?(task, reason):
  match reason with
  | contradiction cert deps => nogoods.add({ assignments := supportOf(cert, deps),
                                             scope := (Q.id, contract, instances, epoch,
grammar) })
  | _                       => penalize(task) -- not a clause
checkNogoods(branch): reject a frame whose assignment set extends a stored nogood in scope

```

**The three equivalences.** Structural identity (same representation modulo justified renaming) is used to avoid repeated work. Checked semantic equality (a native definitional check with explicit transparency and rollback, or a proved propositional equality with a transported contract) may justify replacing a candidate. Finite observational agreement,  $e \sim_X e'$  iff  $\text{obs}_X(e) = \text{obs}_X(e')$ , is used for ranking and batching only<sup>[P3,P4,P8,P9]</sup>. The observational bucket procedure is:

```

bucket(X, e): key := obsSignature(X, e)
  buckets[key].representative := best by scalar cost
  buckets[key].hidden += [recipeOf(e)] -- recoverable, never deleted
refineBuckets(X'): -- after a new checked counterexample
  for b in buckets: split b by obsSignature(X', .) over representative ++ hidden;

```

```
resume deferred members that become the best in their new bucket
```

Permanent quotienting requires a checked equivalence strong enough for every future use, or an exact truth table over a known finite complete domain with compositional semantics; if memory pressure forces deletion the loss is recorded in the ledger<sup>[P2,P3,P8,P9]</sup>. Definitional-equality comparisons never mutate another branch’s unknowns, and unification successes are never unioned globally as state-independent equalities<sup>[P3,P5]</sup>. Equality saturation is restricted to a reified, well-typed first-order or monomorphic fragment in which each edge carries a proof and a common type in a common scope; heterogeneous equalities and transports through dependent parents are explicit; the original candidate is retained for execution-policy comparison and the optimized candidate’s property is rechecked rather than borrowed<sup>[P2,P3,P8,P9]</sup>. Proof irrelevance merges proofs of one proposition when the axiom policy agrees; it never merges dictionaries or data records<sup>[P2,P3]</sup>.

**Library learning.** A successful closed subterm (a fold step, a dictionary transformation, a small dependent adapter) is abstracted over its dependency telescope in order, stored as a helper schema ( $\Delta \vdash e : \sigma$ , contract lemmas, cost, provenance), and reused under  $\mathcal{G}$  by finding  $\rho : \Delta \rightarrow \mathcal{G}$ , instantiating  $e$ ,  $\sigma$ , and every lemma through the same  $\rho$ , and native-checking the result<sup>[P1,P6]</sup>. Every learned helper is checked as an ordinary declaration; in evaluation, fragments learned from a test problem never enter other runs unless the protocol permits online learning<sup>[P1]</sup>.

## 19.9 Resource accounting

The ledger of Decision 2.8 lives outside every branch snapshot and is written before a transaction can restore state<sup>[P2,P4,P5,P7,P9]</sup>.

```
structure Ledger where
  -- logical work, monotone: never decremented by rollback
  ruleApplications, unificationSteps, normalizationSteps, proofTasks,
  solverCalls, candidateEvaluations, duplicateCandidates, cancellationsCharged : Nat
  synthesisWork, proofWork : Nat -- counted separately (P9)
  perLane : Lane -> WorkVector
  -- operational controls, independent of the logical counters
  deadline : WallClock; memoryCeiling : Bytes; processLimits : Limits
  -- evidence
  incompleteness : Array IncompletenessEvent -- caps, drops, unsupported, unproved prunes,
  -- finite carrier frontier, retrieval cutoff
  scopes : Array (Lane x GrammarFrontier x Option Checked)
```

Three independent controls apply: the logical work budget (per counter), the wall-clock deadline, and process-level limits on memory, output size, and external process lifetime; heartbeats alone do not account for compilation, IO, or external solvers<sup>[P3,P9]</sup>. Per-rule limits produce a **yielded** or **unsupported** frame, never a refutation; raising a proof budget re-enqueues verification tasks without regenerating terms, and raising a cost bound reuses validated indexes and templates without reusing stale branch-local metavariables<sup>[P3]</sup>. Hard memory exhaustion is an infrastructure outcome (**infrastructure** in Section 3.3), not a completed negative search<sup>[P3]</sup>.

**Interruption propagation.** Outcomes are typed: **failed** is local and triggers rollback; **interrupted** with a cancellation, out-of-memory, or global-budget reason unwinds to the controller, which retires or resets the worker, closes owned solver inputs, terminates the owned process tree, and waits for cleanup before reusing a channel; a delayed solver reply carries request and frame identities so it cannot satisfy a newer obligation<sup>[P3,P7,P9]</sup>. A cancellation is never interpreted as exhaustion<sup>[P4]</sup>.

**Modes.** Deterministic mode fixes provider ordering, tie-breaking, generation seeds, and the logical schedule, so that a transcript replays exactly; wall-clock deadlines may still vary and the receipt says so. Performance mode returns the first verified result from parallel workers and records which lane and branch produced it; final ordering can still be made deterministic by cost and a stable candidate key<sup>[P3,P4,P8]</sup>. Parallel workers share only immutable environment snapshots and validated closed cache entries. A worker receives a closed request slice (a whole lane, a candidate verification, or a connected component of the dependency graph) and returns a closed proposal or a replay plan that the owner rechecks and installs transactionally. A component may run in parallel only with an *independence certificate*: disjoint owned metavariable sets, disjoint synthetic obligations, and an unchanged shared rigid context; when either task introduces a new dependency the components are merged or the conflicting step is serialized<sup>[P1,P4,P5,P9]</sup>. Cold branches are evicted to compact action traces and rebuilt from the nearest checkpoint; the working set of materialized snapshots is bounded and the reconstruction cost is measured against the memory saved<sup>[P1,P5,P6]</sup>.

## 19.10 What the search guarantees

Soundness of accepted results (Theorem 13.1) does not depend on anything in this section. The statements below concern discovery and termination; each is stated with the hypotheses its source proposal attaches, and the unified engine commits to exactly one of them.

**Definition 19.3** (Finite effective search envelope<sup>[P4]</sup>). An envelope consists of a finite provider inventory; a finite term, type, literal, motive, measure, and proof grammar at each declared tier; a finite bound on rule depth or expanded states; and a terminating, resource-limited implementation of every primitive action. A timeout is an unknown result, not a decision.

**Proposition 19.4** (Termination of a bounded run<sup>[P4]</sup>). *A search confined to a finite effective envelope terminates, provided the scheduler does not reschedule the same unfinished action indefinitely without consuming a finite request resource.*

The ledger provides the second hypothesis: every dispatch charges a monotone counter (Section 19.9). A queue cap preserves termination but invalidates an exhaustion or minimality claim, which is why drops are recorded<sup>[P4]</sup>.

**Proposition 19.5** (Relative bounded-search coverage<sup>[P3]</sup>). *Fix an environment, a finite branching rule grammar, a finite cost bound, strictly positive cost for every generative step, terminating administrative steps, and a terminating applicability and checking procedure for each bounded alternative. If the scheduler eventually visits every alternative within the bound and removes branches only with sound no-completion certificates, then every successful leaf in that bounded search graph is eventually visited.*

**Theorem 19.6** (Completeness of a finite certified layer<sup>[P8]</sup>). *Let a layer be a finite effectively enumerable collection of scoped construction plans, a terminating reconstruction procedure, and a total verification procedure that exactly decides the requested property for every reconstructed candidate, with pruning that preserves every satisfying plan. Exhaustive exploration returns a satisfying candidate whenever one exists in the layer, and if it completes without one, no candidate in the layer satisfies the property.*

Only this statement licenses `grammarExhausted` as a negative artifact, and only for the exact layer it names (Decision 2.7); P6’s finite-fragment completeness proposition is the same statement with focusing listed as an existence-preserving transformation<sup>[P6,P8]</sup>.

**Theorem 19.7** (Conditional relative completeness by dovetailing<sup>[P1,P4,P5,P7,P9]</sup>). *Fix a frozen query, an admitted environment, and effective grammars of programs and contract proofs with*

*finitely many plans per tier. Assume (i) a pair  $(e, p)$  satisfying the query has a finite derivation in these grammars; (ii) each step of that derivation is eventually executed with sufficient primitive resource limits; (iii) the schedule fairly increases construction bounds and verification allowances together; (iv) the reference lane prunes only with a checked impossibility certificate or a success-preserving replacement by a retained admitted recipe of strictly smaller rank; and (v) no permanent resource cap discards the successful derivation. Then an unbounded run eventually offers a certified result, assuming the acceptance gate terminates on that pair.*

*Proof.* The derivation has finite syntax sizes, rule count, provider selections, and a finite resource need for each successful primitive computation, so some tier  $(i, s, r, v)$  contains all of them. The diagonal schedule opens that tier after finitely many quanta because every tier receives the reserved share, and the FIFO stream visits the derivation within the tier after finitely many picks. By (iv) no derivation of a valid pair has been removed, and each replacement chain is finite by strict rank decrease, so a successful representative remains. The verification queue reaches allowance  $v$  for that candidate by (iii); the checks complete by (ii), and the gate accepts.  $\square$

Hypothesis (iii) is the substantive one: enumerating programs fairly while giving each a fixed proof timeout does not satisfy it, and neither does a fixed unification timeout on a type instantiation<sup>[P4]</sup>. P5 states the same theorem for “replayable” plans, P7 adds the ranked-replacement clause (iv), P9 states it as “fair relative discovery” with dovetailed validation, and P1 as “conditional eventual discovery”; the wording above takes P4’s hypotheses with P7’s pruning clause because it is the most explicit about proof budgets.

*Remark 19.8* (Fairness is not completeness<sup>[P6]</sup>). The scheduler guarantees that each live lane and each admitted tier receives infinitely many quanta in an unbounded run. This ensures that a reachable finite successful path is not starved, given sufficient budgets. It does not ensure that the path is generated by an incomplete application matcher, that Lean’s unifier proposes the needed higher-order instantiation, or that the path survives heuristic pruning. A fair search that retains only the best eight components per key is incomplete if a ninth is needed; a proof-producing prune can justify removing a branch, a beam width cannot, and the two belong in different parts of the ledger.

**Decision 19.9** (The single commitment). The engine commits to Theorem 19.7 for the reference lane’s declared grammar together with the verification queue, and to Termination for every audit run; it commits to nothing stronger. All other lanes are heuristic accelerators whose drops, caps, retrieval cutoffs, and unproved prunes are recorded as incompleteness events. A negative outcome is `impossible` only with a Lean proof, `fragmentUnderivable` only with a fragment certificate and its stated logic, `grammarExhausted` only under the finite-certified-layer theorem for the exact layer, and otherwise `budgetExhausted` with a resumable state (Section 3.3). The completeness information is attached to the exact route that produced the negative result, and a capable positive lane gains no right to report non-inhabitation because a different lane is complete for a smaller language<sup>[P2,P4,P6,P8,P9]</sup>.

The finite experiments in the proposals bear on this section only narrowly: the finite CEGIS runs of P2, P3, P8, and P9 demonstrate the observational-bucket loss that Section 19.8 prevents, P5’s prototype exhibited the projection-before-constructor priority failure that the pressure ordering addresses, and no proposal measured the reserved shares, the halving constant  $k$ , or the derived deadline; those remain tunable engineering defaults reported in every receipt<sup>[P2,P3,P5,P8,P9]</sup>.

## 20 Algorithms: proof services and the acceptance gate

This section gives the procedures behind Sections 11.3, 7.7, and 13: the proof-service adapter and its worker loop, the constructive propositional lane with its non-derivability certificate, the

acceptance gate as one procedure with named failure results, the freeze step and the transaction adapter’s typed outcomes, the production of negative results, an adversarial checklist, and two end-to-end traces. Where the nine proposals give different versions of one procedure, the most fully specified one is adopted and the alternatives are noted. The worker loop is taken from Forge’s `lean/Forge/Frontend.lean` [4], which is the only compiled implementation of this contract among the sources; the pseudocode below follows its control flow line by line and marks every Leant2 addition.

## 20.1 The proof-service adapter

### 20.1.1 Reply type

The reply type merges P3’s `ProofReply`, P1’s `checkFrozen`, P6’s plug-in outcomes, and P7’s `ProofServiceResult`<sup>[P1,P3,P6,P7]</sup>. P4’s three-valued adapter (`Proved` | `Refuted` | `Unknown`) is the same type without the reduced-goal and patch fields.

```
inductive ProofReply where
| proof      (pf : Expr) (patch : AssignmentPatch) -- patch is empty in frozen mode
| refutation (pf : Expr) (patch : AssignmentPatch) -- pf : goal -> False
| reducedGoal (goal' : MVarId) (reconstruct : Expr) -- reconstruct : goal' -> goal
| unsupported (feature : String)
| unknown    (reason : String) (consumed : Budget) (attempts : Array Attempt)

structure Attempt where -- Forge's replay record, one entry per worker
  worker : String; accepted : Bool; reason : String; ms : Nat

structure AssignmentPatch where assigned : Array (MVarId x Expr) -- each in the permitted set
```

A bare Boolean, a “no goals” message, or a remote checker’s `okay = true` is never a reply; `unknown` carries the budget consumed so that the scheduler can re-queue at a larger tier, and a `refutation` rejects exactly the closed candidate in exactly this context (Section 5.3)<sup>[P1,P2,P3]</sup>.

### 20.1.2 The worker loop

Forge’s `runWorker` saves the full tactic state, runs one worker under `tryCatchRuntimeEx` so that heartbeat and deadline exceptions are caught like any other failure, and on apparent success audits the proof term before keeping the state. Leant2 adds three things: a *mode*, a *permitted assignment set*, and a *budget* argument; in joint mode the assignments the worker made to permitted holes are captured and returned as a patch, and the state is restored either way so that the engine applies the patch through its own transaction<sup>[P2,P7]</sup>.

```
runWorker (goal : MVarId) (worker : Worker) (mode : Joint | Frozen)
  (permitted : Set MVarId) (policy : AxiomPolicy) (budget : Budget)
  : TacticM (Except String (Expr x AssignmentPatch)) := do
  let saved      <- saveState -- Forge: full tactic state
  let goalType   <- instantiateMVars (<- goal.getType) -- the ORIGINAL goal
  let goalCtx    := (<- goal.getDecl).lctx
  let before    := currentAssignments (mvars existing now) -- Leant2: for the patch
  let outcome <- tryCatchRuntimeEx (withBudget budget do -- limits are failures too
    evalTactic worker.script
    unless (<- getUnsolvedGoals).isEmpty do throw "worker left goals open"
    let pf <- instantiateMVars (mkMVar goal)
    if pf.hasSorry then throw "REJECTED: admitted, not proved"
    if pf.hasExprMVar then throw "REJECTED: proof still contains metavariables"
    let escaped := (collectFVars pf).filter (fun x => not (goalCtx.contains x))
    unless escaped.isEmpty do throw "REJECTED: uses variables outside the goal's context"
    let ok <- withLCtx goalCtx (<- goal.getDecl).localInstances do
```

```

      try Meta.check pf; isDefEq (<- inferType pf) goalType
      catch _ => pure false
    unless ok do throw "REJECTED: term does not have the goal's type"
    let bad <- disallowedAxioms policy pf      -- collectAxioms over pf.getUsedConstants
    unless bad.isEmpty do throw s!"REJECTED: axioms outside the policy: {bad}"
    -- Leant2 additions: what did the worker assign besides the goal?
    let delta := (currentAssignments before) \ {goal}
    match mode with
    | Frozen => unless delta.isEmpty do throw "REJECTED: frozen candidate was assigned"
    | Joint  => unless delta.subsetOf permitted do
      throw s!"REJECTED: assigned outside the permitted set: {delta \ permitted}"
    pure (pf, AssignmentPatch.ofDelta delta)
  (fun ex => pure (Except.error (<- ex.toMessageData.toString)))
saved.restore (restoreInfo := true)  -- ALWAYS: assignments, lctx, goals, messages
return outcome

```

Three details are load-bearing. First, the restore happens on success as well as on failure in Leant2 (Forge keeps the state on success because the tactic *is* the caller); the accepted proof and patch are data that the engine installs through the atomic attempt of Section 5, so that the patch is subject to the commit criterion, is visible to wake-ups, and is rolled back with its branch<sup>[P2,P6]</sup>. Second, `disallowedAxioms` iterates `collectAxioms` over every constant the term uses and subtracts the profile’s allowlist (Section 3.4); Forge’s `forge.allowChoice` option is the standard/strict switch, and stock `grind` needs `Classical.choice`, so the strict profile drops `grind` from the portfolio rather than accepting its proofs<sup>[4]</sup>. Third, the type check runs under the goal’s own local context and instances, so nothing the worker introduced is in scope; this is what catches Forge’s “escape” test worker, whose proof is a phantom hypothesis of the goal’s own type.

The frozen mode closes the candidate before the goal is formed: every program hole that is still open is replaced by a rigid local parameter and the obligation becomes universally quantified over those parameters, so that “there should be no computational holes” is enforced structurally rather than by convention<sup>[P2,P6]</sup>.

```

freezeGoal (Phi : Expr) (t : Expr) : MetaM (MVarId x Array FVarId) := do
  let holes := (collectMVars t).filter isProgramHole
  withRigidParams holes fun params => do      -- one fvar per open hole
    let t' := t.replaceMVars holes params
    let g  <- mkFreshExprMVar (Phi.betaRev #[t']) -- Phi applied to the frozen term
    pure (g.mvarId!, params)

```

### 20.1.3 Portfolio order and per-tier budgets

The adapter runs a fixed ordered list of workers, as Forge’s `workers` does (`decide`, `omega`, `forge_cone?`, `grind`, with the oracle omitted in restricted mode), extended by the tiers the proposals agree on<sup>[P1,P3,P5,P8]</sup>. The order is fixed per query so that the replay record is deterministic; the budgets escalate per obligation rather than per worker.

```

tier 0  conversion / rfl; local assumption; constructor of the goal head
        (budget: one unification each; never re-queued)
tier 1  decide (kernel reduction only, never native_decide); curated simp with
        the query’s explicit lemma set; omega for linear Nat/Int goals
tier 2  Forge certificate checkers: cone, Farkas, affine witness, polynomial
        recurrence, conserved invariant -- proof term is ‘sound cert (by decide)’
tier 3  grind (omitted under the strict profile: needs Classical.choice)
tier 4  Aesop with the query-specific rule set; induction ONLY through the
        recursion schema of Section alg:recursion, never as a generic worker
tier 5  optional pinned Mathlib procedures (separate package)
escalation: an obligation that returns ‘unknown’ at tier k with allowance b is

```

```
re-queued at tier k+1 with allowance 2b, up to the query's proof budget;
each attempt is charged to the ledger and never refunded
```

What is delegated is exactly what each tool decides: **decide** closes ground decidable propositions by kernel reduction; **omega** closes linear arithmetic over **Nat** and **Int** with the usual truncated-subtraction case split; **simp** performs controlled unfolding with a lemma set the query names (uncontrolled unfolding of a recursive contract is worse than the synthesis problem); **grind** supplies congruence closure, its arithmetic module, and e-matching over the project's annotated lemmas; Aesop supplies rule-based structural search; and Forge's checkers cover the arithmetic obligation shapes that have a proposal-check-soundness triple, so that the untrusted proposer runs outside Lean and the proof term is an application of the checker's soundness theorem to a certificate that **decide** validates<sup>[P1,P4,P8]</sup>[4]. P3 and P8 additionally put indexed theorem retrieval in tier 4; P2 suggests reusing Aesop's rule infrastructure directly. The unified design keeps retrieval in the construction lanes (Section 15) because a retrieved lemma is a provider, not a proof step.

#### 20.1.4 A joint-mode patch is a branch choice

The engine never lets a worker commit an assignment. It replays the returned patch as a rule application inside the atomic attempt, recorded in the plan as a construction step so that replay, receipts, and rollback treat it like any other choice<sup>[P2,P6,P7]</sup>.

```
applyProofPatch (branch) (obl : Obligation) (reply : ProofReply.proof pf patch) :=
  attempt branch (do
    for (m, v) in patch.assigned do
      require m in obl.permitted
      require admissible v obl.grammar obl.executionPolicy -- P2 firewall: a
        -- proof may not fill a program hole with the spec oracle or a choice term
      m.assign v; wake (dependentsOf m)
    obl.goal.assign pf
    branch.plan.push (ProofAssigned obl.id patch reply.worker))
```

#### 20.1.5 Proof-debt scheduling

Obligations that cannot be attempted yet are debt, not failures. Each carries a cost, a readiness state, and the set of metavariables it waits on; the scheduler gives proof work a turn after a bounded amount of program growth, wakes index and arithmetic obligations immediately when a relevant hole is assigned, and delays heavy tiers until the computational holes an obligation mentions are assigned or exposed as witness choices<sup>[P1,P5]</sup>.

```
structure Obligation where
  goal : MVarId; tier : Nat; allowance : Budget; waitsOn : Set MVarId
  readiness : Ready | Blocked | Deferred; cost : Nat -- size of goal after instantiate

onAssign (m : MVarId) := for o in waitingOn m do
  if (instantiateMVars o.goal.type).hasProgramHole then o.readiness := Deferred
  else o.readiness := Ready; requeue o -- arithmetic/index goals wake now

schedulerTurn := if growthSinceLastProof >= G then
  pick the Ready obligation with least (tier, cost); run adapter at its tier
  match reply with
  | proof/refutation => install through the attempt; discharge or reject branch
  | reducedGoal g' r => replace o.goal by g'; remember r for reconstruction
  | unknown _ c _ => charge c; escalate o (tier+1, 2*allowance) or park it
    in the fair layer if the proof budget is exhausted
```

This is a ranking policy, not a pruning theorem: branches whose debt is expensive stay available in the fair cost layer, and a production heuristic that abandons them must say so in the incompleteness ledger<sup>[P1]</sup>.

## 20.2 The constructive propositional lane

### 20.2.1 Reification

The lane reifies the propositional fragment of a goal and its context. Connectives become formula nodes; anything else becomes an atom keyed by its exact expression after `instantiateMVars`; an expression containing a metavariable refuses reification. Following Forge’s encoder, each atom records at translation time whether it is safe for a negative verdict: an atom that mentions a concrete constant with its own proofs, or an erased instance dictionary, poisons a negative verdict but not a positive one<sup>[P1,P3,P5]</sup>[4].

```
reify (e : Expr) : ReifyM Formula := do
  match (<- whnfR e) with
  | A -> B where B does not depend on the binder => imp (reify A) (reify B)
  | And A B | Prod A B      => and (reify A) (reify B)
  | Or A B | Sum A B        => or (reify A) (reify B)
  | Iff A B                  => and (imp (reify A) (reify B)) (imp (reify B) (reify A))
  | Not A                    => imp (reify A) bot
  | False | Empty           => bot
  | True | Unit | PUnit     => top
  | e' =>
    -- atom: opaque proposition or type
    if e'.hasMVar then throw unsupported
    let a <- atomFor e' -- keyed by the expression, compared with isDefEq
    a.refutationSafe := not (mentionsConcreteConstantWithProofs e' || erasedDictionary e')
    pure (var a)
reifySequent (Gamma) (T) := (Gamma.filterMap (fun h => try reify h.type), reify T)
```

Types in `Sort` and propositions are reified alike, because the lane produces terms, not truth values; the interpretation table `atomFor` is what turns a derivation back into Lean.

### 20.2.2 A terminating focused calculus

The calculus is Dyckhoff’s contraction-free LJ<sub>T</sub> (G4ip), which Djinn implements and which P5 and P9 name as the basis of the lane<sup>[P3,P5,P9]</sup>. Its left implication rule is split by the shape of the antecedent, which is what removes contraction and makes the search terminate under a well-founded weight on sequents. Every rule carries its Lean term constructor.

```
prove (Gamma |- G) : Option Expr := -- terminating; no loop check needed
  -- right rules (invertible first)
  G = top          => True.intro
  G = imp A B      => fun (a : A) => prove (Gamma, a:A |- B)
  G = and A B      => And.intro (prove Gamma|-A) (prove Gamma|-B)
  -- invertible left rules on some hypothesis h
  h : bot          => False.elim h
  h : and A B      => let a := h.1; let b := h.2; prove (Gamma\h, a, b |- G)
  h : or A B       => Or.elim h (fun a => prove (Gamma\h, a |- G))
                      (fun b => prove (Gamma\h, b |- G))
  h : imp (and C D) B => prove (Gamma\h, (fun c d => h (And.intro c d)) |- G)
  h : imp (or C D) B  => prove (Gamma\h, (fun c => h (Or.inl c)),
                      (fun d => h (Or.inr d)) |- G)
  h : imp (var p) B, p:P in Gamma => prove (Gamma\h, (h p) |- G)
  -- non-invertible rules: try each, backtrack (finite because weight decreases)
  G = var p, p in Gamma      => p
  G = or A B                 => Or.inl (prove Gamma|-A) | Or.inr (prove Gamma|-B)
```

```

h : imp (imp C D) B =>          -- the rule that replaces contraction
  let d2b : D -> B := fun d => h (fun _ => d)
  let cd  : C -> D := prove (Gamma\h, d2b |- imp C D)      -- premise 1
  let b   : B      := h cd
  prove (Gamma\h, b |- G)                                     -- premise 2
otherwise => none

```

The derivation is translated to a term over the atom table and handed to the proof-service audit like any other worker output: the term is type-checked against the original goal, its free variables must be hypotheses of the goal, and its axiom closure must be empty (the lane uses no axioms). A derivation that survives is a **proof** reply; the lane is scheduled at tier 1 for goals whose reification succeeds.

### 20.2.3 The non-derivability certificate

When `prove` returns `none` the calculus has decided non-derivability, but the enumeration tag is not a checkable object. The lane therefore runs a bounded countermodel proposer and returns `fragmentUnderivable` only with a certificate that a separate checker validates by recomputing forcing from scratch, as Forge’s extension round does<sup>[P1,P4,P6,P8]</sup>[\[4\]](#).

```

structure KripkeCountermodel where
  worlds : Nat                -- 1 <= worlds <= 32
  root : Fin worlds
  relation : Array (Array Bool) -- reflexive, antisymmetric, transitive; root least
  valuation : Atom -> Array Bool -- persistent: v[i] && relation[i][j] -> v[j]

checkCountermodel (Gamma |- G) (M) : Bool :=
  structural checks on relation and valuation (above), else reject
  let rec force (w) (f) : Bool :=
    | var a  => M.valuation a w
    | bot    => false          | top => true
    | and a b => force w a && force w b
    | or a b  => force w a || force w b
    | imp a b => forall v, M.relation w v -> (force v a -> force v b) -- all successors
    (forall h in Gamma, force M.root h) && not (force M.root G)

```

**Definition 20.1** (Meaning of a fragment certificate). An accepted certificate for the reified sequent  $\Gamma_0 \vdash G_0$  establishes exactly: there is no derivation of  $G_0$  from  $\Gamma_0$  in the named intuitionistic propositional calculus over the named atoms. It does not establish that Lean proves  $\neg T$ , that no Lean term of  $T$  exists, that classical reasoning cannot prove  $T$ , or anything about hypotheses that were not reified. The result is `fragmentUnderivable(IPC, cert)` and never `impossible` (Decision 2.7)<sup>[P1,P2,P5,P6,P8]</sup>.

If the world bound is exhausted before a countermodel is found, the reply is `unknown` with the enumeration tag in the ledger; the finite model property of IPC does not make a fixed world cap a decision procedure<sup>[4]</sup>. The lane’s enforcement tests are classical tautologies (excluded middle, double-negation elimination, Peirce’s law, propositional linearity): each has a countermodel, each is provable classically, so an interface that promoted any of these certificates to a Lean refutation would be visibly unsound. An atom flagged unsafe for refutation at reification time downgrades the certificate to `unknown` with the obstructing atoms named.

### 20.2.4 The classical lane

The classical lane adds `Classical.em`, `Classical.byContradiction`, and `Decidable.em` as explicit providers. It is enabled only by the standard and project-relative profiles, and it is

scheduled after the constructive lane has returned a reply for the same goal, so that constructive candidates are found and displayed first and classical ones are labeled by their axiom inventory (Section 7.7)<sup>[P5,P6]</sup>. For a propositional goal with a certificate the lane first uses Glivenko’s theorem: it runs `prove` on  $\neg\neg G$  and, on success with term  $p$ , returns `Classical.byContradiction (fun h => p h)`; goals with quantifiers skip this step and fall through to the ordinary providers<sup>[4]</sup>. The resulting proof’s axiom closure contains `Classical.choice`, `propext`, and `Quot.sound`, which the receipt reports.

## 20.3 The freeze step and the transaction adapter

### 20.3.1 What is frozen

The gate must receive the original elaborated query from an owner that search cannot mutate; otherwise a search can solve a specialization and claim the original, for example by assigning an ambient output-type hole to the input type and turning an impossible generic map into the identity<sup>[P6]</sup>. The freeze happens once, after elaboration and before any search.

```
freeze (raw : ElaboratedQuery) : FrozenQuery := do
  require raw.unknownIdentifiers.isEmpty          -- no auto-bound implicits
  let ambient  := metavariables present before elaboration (caller-owned)
  let created  := metavariables elaboration created (query-owned)
  let permitted := raw.authorizedHoles             -- explicit, possibly empty
  for m in ambient \ permitted do abstractAsRigidParameter m -- read-only holes
  for m in created do resolve m or classify m as an authorized hole or refuse
  require universe parameters are parameters (no level mvars remain)
  let Q := { E, U, Gamma, T, Phi, Obs, G, Pol, Bud } -- Definition def:query
  Q.manifest := { parameters, authorizedHoles := permitted, epoch := E.epoch }
  Q.digest := canonicalEncoding Q                  -- identity, not evidence
  seal Q                                           -- immutable; the gate reads only sealed queries
```

The manifest records the distinction between parameters and synthesis metavariables. For the tactic entry point, the result of a query is a patch  $\{\text{goal} := t\} \cup \{\text{assignments to permitted ambient holes}\}$ ; the caller applies it transactionally after the gate has accepted, and nothing else in the caller’s state is touched<sup>[P6,P7,P8]</sup>.

### 20.3.2 Typed outcomes of the transaction adapter

Each proposal’s prototype catches exceptions broadly; the production adapter returns typed outcomes and never swallows cancellation<sup>[P6,P9]</sup>.

```
inductive Outcome where
  | progress (children : List MVarId) (changed : Set MVarId)
  | solved   (term : Expr) (manifest : Dependencies)
  | blocked  (watched : Set MVarId) (reason : String)
  | rejected (kind : RejectKind) (cert : Option Certificate)
  | notApplicable | unsupported (feature : String)
  | yielded   (cursor : Continuation) (charged : Work)
  | aborted   (reason : Interrupt)      -- cancellation, OOM, deadline: unwinds to the controller

attempt (snapshot covers: MetaM state, elaborator state incl. postponed synthetic
        mvars, local instances, messages, scratch environment) :=
  let s <- saveAll
  try run rule; on Outcome.progress/solved keep; otherwise restoreAll s
  catch e => if isInterrupt e then restoreAll s; throw e -- never "not applicable"
            else restoreAll s; pure (rejected (exception e) none)
  ledger.charge work -- outside the snapshot; never refunded (Decision dec:ledger)
```

## 20.4 The acceptance gate as one procedure

The procedure merges P4's `finalize(Q, program, proof)`, P6's `accept`, P8's `accept`, P9's `Authorize(0, P)`, and the five-step transactions of P3, P5, and P7; the two Forge pitfalls are handled in step 4<sup>[all]</sup>. The step numbering matches Section 13. Each `fail` names the result returned; nothing is published on any failure, and the candidate remains available as `typedOnly` only when its own closure passed steps 1–7 independently of the proof.

```
gate (Q : FrozenQuery) (b : CandidateBundle) : GateM Verdict := do
  -- 1. origin
  unless b.epoch = Q.manifest.epoch do fail staleEpoch
  let (T, Phi) := (Q.T, Q.Phi) -- never b.inferredType
  -- 2. close (in a fresh native state; nothing of the search is in scope)
  let t <- instantiateMVars b.program; let p <- instantiateMVars b.proof
  for e in [t, p] ++ b.helpers.map (.value) do
    if e.hasMVar || e.hasLevelMVar then fail (unresolvedHole e)
    if e.hasLooseBVars then fail malformedTerm
    if (collectFVars e).any (fun x => not (Q.Gamma.contains x)) then fail (escapedLocal e)
  let tC := mkLambdaFVars Q.Gamma t (preserveLets := true) -- close over the telescope
  let TC := mkForallFVars Q.Gamma T
  let PC := mkForallFVars Q.Gamma (Phi.betaRev #[t]) -- Phi applied to THIS t
  let pC := mkLambdaFVars Q.Gamma p
  abstract exactly Q.U as level parameters; fail universeSpecialized if fewer remain
  -- 3. bundle
  let helpers <- topologicalOrder b.helpers -- fail cyclicHelpers on a cycle
  for h in helpers do
    if h.kind = axiom || h.value.isNone then fail (missingBody h.name)
    if h.value.mentions b.programName && not h.isCheckedRecursion then fail selfReference
  let defn := { name := b.programName, levelParams := Q.U, type := TC, value := tC }
  let thm := { name := b.theoremName, levelParams := Q.U, type := PC,
    value := pC.replace (t |-> Expr.const b.programName Q.U) }
  -- 4. kernel-check, both pitfalls handled
  let opts := Q.options.set debug.skipKernelTC false -- pinned OFF, not unset
  let mut env := Q.E.scratchCopy
  for d in helpers ++ [defn, thm] do
    match Kernel.Environment.addDecl env opts d with
    | .ok env' => env := env' -- Except IS the signal
    | .error e => fail (kernelRejected d.name e)
  -- 5. exact-statement matching, read back from the checked environment
  let ci := env.find! b.theoremName
  unless ci.type == PC do fail (statementMismatch ci.type PC) -- structural, not defeq
  unless (env.find! b.programName).type == TC do fail statementMismatch
  unless ci.value.mentionsConst b.programName do fail proofAboutAnotherTerm
  -- 6. audits: two closures, computed separately
  let axP <- collectAxioms env b.theoremName
  let axT <- collectAxioms env b.programName
  if sorryAx in axP ++ axT then fail admitted
  unless (axP ++ axT).subsetOf Q.Pol.allowedAxioms do fail (axiomViolation ((axP++axT) \
    allowed))
  let deps := transitiveConstants env b.programName -- runtime closure
  for c in deps do
    if isPartial c || isUnsafe c || hasImplementedBy c || hasExtern c then
      unless Q.Pol.runtimeAllows c do fail (runtimeViolation c)
    if isNoncomputable c && Q.Pol.requiresExecutable then fail (runtimeViolation c)
    if c in Q.G.withheldProviders then fail (withheldProviderUsed c)
  -- 7. compile
  let compiled := if Q.Pol.requiresExecutable then compileDecl env b.programName
    else notRequested
  if compiled = failed e && Q.Pol.requiresExecutable then fail (compileFailed e)
  -- 8. source replay (presentation evidence; never replaces step 4)
  let src := delab env Q.presentationContext (helpers ++ [defn, thm])
  let re <- elaborateFresh Q.E src (autoImplicit := false)
```

```

let replay := if re.value == tC then exact
              else if <- isDefEq re.value tC then variant -- reported as such
              else failedPresentation
if replay = failedPresentation && Q.Pol.strict then fail replayMismatch
-- 9. publish atomically
publishAll (helpers ++ [defn, thm]) in Q.E in one command transaction
return verified (Accepted.mk defn) (Accepted.mk thm)
          (receipt Q b axP axT deps compiled replay)

```

Two differences between the sources are resolved here. P3 and P5 audit before the kernel check and P4, P6, P8, and P9 after; the gate audits after, because `collectAxioms` must run over the constants as the kernel accepted them, including generated helpers. If the `CoreM addDecl` path is used instead of the synchronous kernel entry, the gate awaits the deferred kernel task and requires its `.ok`; under `Elab.async` the presence of the constant in the environment is not evidence[4]. P7 freezes the declaration’s type separately from its body and P8 reads the statement back from the installed constant; the gate does both, since the read-back is what catches a valid term paired with a proof about another term<sup>[P7,P8]</sup>.

**Invariant 20.2** (Closed over the telescope, checked at the frozen statement). The program is checked at  $\forall \Gamma. T$  and the proof at  $\forall \Gamma. \Phi(t)$  with  $\Phi$  taken from the frozen query and applied to the exact accepted  $t$ , even when  $\Phi$  ignores its argument; the theorem refers to the definition by its generated name. Reopening  $\Gamma$  yields the two judgments of Definition 3.2; this is the premise of Theorem 13.1<sup>[P2,P5,P8]</sup>.

### 20.4.1 Receipt fields

```

structure Receipt where
  query      : Digest x FrozenQuery.manifest -- request identity, epoch, toolchain, imports
  accepted   : { program, theorem, helpers : Array Name } -- exact names and closed types
  contract   : Expr -- PC as checked, not as displayed
  axiomsProof, axiomsProgram : List Name -- separate closures, actual use
  runtimeDeps : List (Name x RuntimeMark) -- implemented_by, extern, partial,
    noncomputable
  flags      : { kernelChecked, compiled, evaluatedOnExamples : Bool }
  sourceReplay : exact | variant | failedPresentation
  policy      : Profile x allowedAxioms x runtimePolicy x providerPolicy x tiersUsed
  search      : bounds, incompleteness ledger, lane and branch that produced it
  counts      : { rejected, notApplicable, infrastructure : Nat }
  proofProvenance : Array Attempt -- Forge’s replay record per obligation
  external     : Option RemoteCheck -- operational evidence only, never kernel

```

The receipt’s accepted-object identifier is tied to the environment generation; a hash detects mismatch but cannot mint an accepted object, and a receipt about one candidate never transfers to a cosmetically similar one<sup>[P3,P7,P8,P9]</sup>.

## 20.5 Negative results

The four negative statements of P8, each produced and certified differently<sup>[P4,P5,P6,P8,P9]</sup>:

1. **budgetExhausted**: the scheduler’s budget ended with live continuations. Certified by nothing; the result carries the resumable state identifier and the ledger. Never cached as anything but “not yet”.
2. **grammarExhausted** with unknown obligations: every plan in a named finite grammar was constructed and some proof obligations returned **unknown** at the maximal tier. Certified by the enumeration ledger (grammar identity, bound, count of plans, per-plan verdict); it is an incompleteness record, not a certificate.

3. **grammarExhausted** with all candidates refuted: as above, but each plan has a checked **refutedCandidate** (a proof of  $\neg\Phi(t)$  or a replayed counterexample) and the enumeration has a checked coverage theorem for that grammar. A grammar-relative impossibility; still not impossible.
4. **impossible**: a Lean term  $q : (t : T) \rightarrow \Phi(t) \rightarrow \text{False}$ , or  $q : T \rightarrow \text{False}$  for a type-only query, that passed the gate's steps 2–6 as a proof at the frozen statement. Only this is an impossibility claim for the exact Lean request.

The negative lane attempts route 4 directly, using the same proof-service adapter on the negated goal, while ordinary synthesis continues; P7's three safe routes are an explicit contradiction from a hypothetical inhabitant, a checked coverage theorem over a genuinely finite domain, and a proved reflection theorem whose conclusion is exactly the noninhabitation claim<sup>[P5,P7]</sup>.

```

refuteGlobally (Q) : ProofReply :=
  -- goal: (t : T) -> Phi t -> False
  -- template 1: polymorphic instantiation (P2, P4, P5, P6, P7, P8 all check one)
  if T = forall (A B : Type), A -> B      then fun f => Empty.elim (f Unit Empty ())
  if T = forall (A : Type), A              then fun f => Empty.elim (f Empty)
  generally: for each universally quantified type binder, try Unit and Empty
    at each position, then apply the hypothetical inhabitant to () and eliminate
  -- template 2: empty inductive or impossible index
  if T's head is an inductive with no constructor, or every constructor's index
    equations fail by no-confusion: fun x => nomatch x    (kernel-checked cases)
  -- template 3: inconsistent contract
  if Phi t reduces to a hypothesis h : a = a -> False or to h : False:
    fun t h => h rfl / fun t h => h
  -- otherwise: hand the negated goal to the portfolio at tier 1..4
  every result passes the audit of runWorker and then gate steps 2-6

```

When only a fragment certificate is available, the lane reports **fragmentUnderivable** beside any **budgetExhausted** and continues; a fragment certificate or a budget never enters a cache as impossibility, and the negative ledger records the logic, grammar, assumptions, abstraction direction, and exhausted bounds<sup>[P9]</sup>. The prototypes' bounded-failure controls (**noFalse**, an impossible natural-number subtype) are tests of a bounded tactic and are reported as **budgetExhausted**; their **noPolyCast** and **uniformEmpty** theorems are route 4<sup>[P4,P5,P7]</sup>.

## 20.6 Adversarial acceptance checklist

Each item names the shortcut it targets and the verdict the gate or adapter must return. A negative control passes only under the expected category; missing output alone is not evidence of correct rejection<sup>[P8]</sup>.

1. **Forge's four test workers** [4]. **admit**: adapter rejects (**hasSorry**). **choice**: rejected under strict, accepted and labeled under standard (**collectAxioms**). **malformed** (goal assigned **True.intro** without a type check): rejected by **Meta.check/isDefEq** against the original goal, although **getUnsolvedGoals** is empty. **escape** (phantom local): rejected by the free-variable check under the goal's own context.
2. **P4's negative controls**. **True.intro** : **False** and **rfl** : **id 0 = 1**: **kernelRejected**. **axiom forged** : **False** used by a proof: remote checker returns **okay = true**; gate returns **axiomViolation [forged]** (or **missingBody** if the axiom arrived in the bundle). **sorry** : **False**: **admitted**.
3. **P8's twelve cases**. A valid term with a proof about another term: **proofAboutAnotherTerm**. Two **Inhabited Nat** dictionaries: distinct candidates, distinct receipts. **Vec alpha n** for **Vec alpha m**: **statementMismatch** unless a checked transport is in the bundle. An older hole assigned a younger branch variable: **escapedLocal**, sibling state restored. Passes tests at zero, fails at one: **bucket refined**, never **verified**. Universal verifier

timeout: `unknown`, never refuted. Invalid recursive decrease: `selfReference`, no fuel inserted. Kernel-accepted but uncompileable definition: `verified` with `compiled = false`, or `compileFailed` when executable output is required. Hidden reference function in the output: `withheldProviderUsed`. Imported theorem with an unapproved axiom: `axiomViolation` although the source shows no placeholder. Printer drops an implicit argument: `sourceReplay = variant` or `failedPresentation`. Worker dies during a check: `infrastructure`; the partial reply is retired by its frame identifier.

4. **P7's additions.** Independent universe parameters specialized to equal ones: `universeSpecialized`. Strict-implicit binders and dependent fields sharing a witness: closure preserves binder info, else `statementMismatch`. A proposition whose proof cannot be eliminated into data: construction rule inapplicable, not a rejection. Callback that assigns an ambient metavariable: joint-mode `REJECTED`: `assigned outside the permitted set`. Stale cache after undo: replayed against the current epoch or `staleEpoch`. Exported source that selects a different instance: `failedPresentation`.
5. **P5's invariant table.** Helper depending on `sorryAx` behind a short theorem: `admitted`. Completed task from a replaced environment: `staleEpoch`. Repeated restore/retry: ledger charged each time. Missing providers, proof timeout, fragment underivability, and checked non-inhabitation: four distinct constructors.
6. **P6 and P2.** Falsely strengthened original goal: `statementMismatch`. Spurious solver model: not a counterexample until replayed. Incomplete finite enumeration: no `grammarExhausted`. Different rendered elaboration: `variant`. Missing helper bodies: `missingBody`.

## 20.7 Worked traces

### 20.7.1 An accepted candidate

Query:  $T = (n : \mathbb{N}) \rightarrow \{m : \mathbb{N} // m = n + 1\}$ , no contract ( $\Phi = \text{True}$ ), standard profile, executable output requested; this is the subtype-successor example that P1's and P4's prototypes solve by joint mode<sup>[P1,P4]</sup>.

```
search  intro n; head Subtype.mk on the goal: children ?m : Nat, ?p : ?m = n + 1
        obligation ?p is Ready (waits on nothing); scheduler turn -> adapter, Joint,
        permitted = {?m}
adapter tier 0 rfl: unify ?m = n + 1 assigns ?m := n + 1; pf := Eq.refl (n + 1)
        audit: no sorry, no mvars, fvars {n} in goal ctx, check ok, axioms {},
        delta = {?m} subset of permitted -> proof pf {?m := n+1}; state restored
engine   applyProofPatch: admissible (n + 1 is grammar/runtime admissible); assign ?m,
        assign ?p; plan += ProofAssigned(?p, {?m := n+1}, "rfl")
        no obligations remain -> candidate t := fun n => Subtype.mk (n + 1) (Eq.refl _)
frozen   verify Phi = True at t: tier 0 constructor -> True.intro
gate     1 epoch ok 2 no mvars/fvars; Gamma empty, tC := t, TC := T, PC := True
        3 no helpers; thm.value := True.intro
        4 addDecl defn, thm with skipKernelTC := false -> .ok, .ok
        5 read-back types == TC, PC; thm mentions the definition name (vacuously ok)
        6 axioms {} / {}; runtime deps {Subtype.mk, HAdd.hAdd, instHAdd, Nat.add}
        computable, none withheld 7 compile -> ok
        8 delab "fun n => <n + 1, rfl>" re-elaborates to tC (exact)
        9 publish; receipt: kernelChecked, compiled, axioms [] / [], replay exact
```

### 20.7.2 A rejected candidate

Query:  $T = \mathbb{N} \rightarrow \mathbb{N}$  with  $\Phi(f) = \forall n, f\ n = n + 1$ , standard profile. A proposal plugin returns the bundle  $\{f := \text{fun } n \Rightarrow n + 1, \text{proof } p := \text{fun } n \Rightarrow \text{forged } n, \text{helper axiom } \text{forged} :$

forall n, (fun n => n + 1) n = n + 1}; the remote checker used in the prototypes reports `okay = true` for such source, as P4’s control shows<sup>[P4]</sup>.

```
gate      1 epoch ok    2 closes; no mvars, no escaped locals
           3 helpers: ‘forged’ has kind axiom and no value -> fail (missingBody forged)
-- variant: the axiom already exists in the imported environment (project file)
           3-4 pass; kernel accepts thm (it is well typed relative to E)
           5 read-back type == PC                                (statement is right)
           6 collectAxioms thm = {forged}; allowed = {propext, Quot.sound, Classical.choice}
              -> fail (axiomViolation [forged]); program’s own closure is clean
result    typedOnly (f) with pending obligation Phi f; ledger: rejected += 1,
           reason axiomViolation; receipt not issued; nothing published
```

The same candidate proved by Forge’s “escape” worker never reaches the gate: the adapter’s free-variable check rejects the proof inside `runWorker`, restores the state, and records `Attempt("escape", false, "uses 1 variable(s) outside the goal’s context")` in the replay record before the next worker runs<sup>[4]</sup>.

## Part III

# Evidence, evaluation, and roadmap

## 21 Evidence available today

Every one of the nine packages compiled its Lean sources remotely through the AXLE service on Lean 4.34.0, with `import Lean only`, `ignore_imports=false`, and `theorems_only=false`, invoked from Wolfram Language. No package had a local Lean toolchain; no package ran an independent checker; every receipt is a transcribed summary of the service response, not a raw capture. Whole-file service times of roughly 1.2 to 2.1 seconds are elaboration latencies and say nothing about synthesis cost. Table 1 summarizes the prototypes.

Collectively the prototypes demonstrate:

- the continuation-backtracking core of Decision 2.2, with witness-first regressions such as  $\{n // n = 1\}$ ,  $\{f // f \text{ 11 29} = 29\}$ , and  $\{c : \text{Choice} // \text{Good } c\}$  that no first-answer-then-prove pipeline passes;
- dependent application, dependent pairs, indexed vector head and tail by `cases`, transport along a hypothesis, higher-rank arguments, independent universe parameters, strict-implicit binders;
- dictionary identity: selecting the second of two same-typed dictionaries because the contract says so;
- finite counterexample-guided synthesis inside Lean with universal correctness by `decide`, and a proof-carrying certifier;
- hand-written proof-carrying motives and their universal correctness theorems;
- the kernel/compiler boundary, observed four times;
- that the checking service’s `okay` flag is insufficient: a custom axiom and a `sorry` both returned `okay=true` and were caught only by the failed-declaration report and the axiom audit.

They do not demonstrate: automatic recursion, motive, carrier, or invariant discovery; a provider index or any library composition beyond explicitly named providers; scheduling, fairness, or resource accounting beyond depth fuel; caching; serialization of plans; the acceptance gate as a separate component; any run of the Leant baseline corpus; any comparison with Leant or Djex; the original integer-indexing and two-universe fixtures. Every proposal says so.

| P | Tactic / files | Synthesized (auto)                        | Hand-supplied and notable                                                                                                       |
|---|----------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 1 | leant2_core    | 21 examples, 4 controls; 24/25 axiom-free | <code>certifiedMap</code> , <code>affineFold</code> ; Python bucket and 81-sketch CEGIS                                         |
| 2 | leant2_core    | 12 declarations; 10 axiom-free            | branch bodies in <code>vectorMap</code> skeleton; <code>ReverseCertificate</code> ; 471-term Python CEGIS                       |
| 3 | micro_synth    | 15 declarations; all axiom-free           | 2 elimination skeletons; 3,873-term Python CEGIS with length abstraction; handwritten certificates                              |
| 4 | leant2_core    | 14 definitions; 19/20 axiom-free          | <code>vecMap</code> template; 27 affine contracts generated and checked with Mathlib; 4 negative-control files                  |
| 5 | leant2_mini    | 11 definitions; 13/14 axiom-free          | <code>vectorReplicate</code> branches; <code>certifiedAppend</code> ; <code>prefixCannotComplete</code> ; 1,875-term Python lab |
| 6 | leant2_pilot   | 14 declarations; all axiom-free           | <code>Vec.map</code> , <code>Vec.zip</code> , <code>reverseAux</code> ; <code>safePruneAt</code> ; recursor codegen probe       |
| 7 | solve          | 13 declarations; 19/20 axiom-free         | <code>vecMap</code> skeleton; affine CEGIS in Lean with proof-carrying certifier                                                |
| 8 | leant2_core    | 11 definitions; 12/13 axiom-free          | Boolean CEGIS in Lean (208 trees, all 16 tables); <code>CertifiedFold</code> with <code>indexFold</code>                        |
| 9 | leant2_demo    | 24 audits; 23 axiom-free                  | <code>listMap</code> with supplied spine; Boolean CEGIS in Lean (9,168 trees)                                                   |

Table 1: Prototype evidence in the nine packages. “Axiom-free” counts printed empty inventories; the remainder use `propext`, and in two certificate files also `Quot.sound`. No inventory contains `sorryAx`.

## 22 Evaluation plan

### 22.1 Tiers

1. **Baseline tier.** The corpus of Definition 1.4, judged by Definition 1.5. This is the release gate for the first usable Leant2 (G3 below). It is run through the compatibility REPL of Section 14.2 in deterministic mode, and its results are recorded as a new golden set that lists outcomes, accepted candidates, and axiom labels.
2. **Extended tier.** Leant’s Church behavior ledger (the 94 historically accepted cells are regression targets), `test-recursive`, `test-behavioral`, and `test-context`, each re-expressed as batch requests with the same session declarations and with independent Lean replay of every accepted candidate.
3. **New corpus.** A versioned corpus of exact requests, not type strings: type, context, universes, contract, imports, admitted implementation components and proof-only components, permitted axioms, profile, and any supplied sketch. Strata: polymorphic and higher-order plumbing; dependent pairs and records; indexed inductives and transport; local dictionary identity; library composition; structural recursion; accumulator algorithms; well-founded algorithms; behavioral contracts (finite examples and universal laws separately); impossible and adversarial requests in a separate denominator. Held-out families, not held-out rows. Three assistance levels: no sketch, supplied recursion skeleton, supplied invariant sketch<sup>[P4,P6,P9]</sup>.

### 22.2 Leakage and controls

The reference implementation, its aliases, its equation lemmas, and any oracle are excluded from the provider inventory by transitive dependency audit; the search worker and the reference oracle run in separate processes; library reuse, composition, and new recursion are reported separately. A permanent adversarial acceptance suite includes: a proof about a different term;

two `Inhabited Nat` dictionaries with different payloads; `Vec a n` offered for `Vec a m`; scope escape; a candidate that passes every current example and fails the next; a verifier timeout; an invalid decrease; a kernel-accepted but uncomparable term; a hidden reference implementation; an unapproved axiom; a printer that changes an implicit argument; a worker death mid-request; a stale cache after `:reset`; a callback assigning an ambient metavariable; a falsely strengthened goal<sup>[P2,P5,P7,P8]</sup>.

### 22.3 Metrics and ablations

Metrics are reported as distributions with censored timeouts: fraction of requests with a verified implementation under budget; time to first typed, tested, and verified candidate; charged work by kind; kernel and replay cost; peak memory; candidate and proof size; axiom and runtime-dependency profile; counterexample replays; reactivated bucket members; and the failure category distribution. Ablations disable one mechanism at a time: continuation backtracking (replaced by first-answer commitment), known-major projections, proof-first ordering, result-head indexing, context slices, contract propagation (replaced by post-filtering), proof-carrying motives, carrier discovery, certified pruning, reversible buckets, the reference lane, and the adaptive lane weights (replaced by fixed Leant-like windows)<sup>[P1,P2,P4,P5]</sup>.

A comparison with Leant and Djex is meaningful only on the shared fragment with pinned repositories, toolchains, and limits; the baseline corpus is that fragment, and the comparison is per-query outcome and time, not an aggregate percentage.

## 23 Roadmap

Gates are evidence, not dates. Each gate names its exit criteria; nothing in a later gate is a prerequisite of the baseline.

### G0 Trusted path.

Origin record, epochs, the native adapter with rollback and interruption tests, the acceptance gate with Forge’s audit, receipts, source replay. Exit: malformed, admitted, scope-escaped, wrong-target, stale-epoch, and forbidden-axiom proposals are refused; a false contract is refused; a closed accepted result replays in a fresh module without synthesis tactics.

### G1 Native core.

Obligation graph, cursor frames, atomic attempts, introductions, spines with prefixes, known-major projections, constructors, exact locals, dictionary modes, the constructive propositional lane with its certificate, the classical lane, transport. Exit: the witness-first regressions; independent universes; strict implicits; two dictionaries with different payloads; `synth-basic`, `synth-manual`, `synth-inductives`, `synth-prove`, and `synth-instance-implicit` pass the baseline criterion.

### G2 Composition.

Provider index with tiers, component recipes and the join, demand graph and context slices, the adaptive scheduler with the reserved reference lane, caches with epoch invalidation, the type-invention ladder. Exit: `synth-library`, all `synth-provider-*`, `synth-djinn-providers`, `synth-quantified-provider`, and all rank-N and Church fixtures pass; no starvation under the fairness test; cache invalidation after `:reset`.

### G3 Baseline release.

Contract propagation, typed observations, joint and frozen proof modes, the proof portfolio including Forge’s checkers, certified CEGIS, reversible buckets, the compatibility REPL. Exit: **the full baseline of Definition 1.5 passes in deterministic mode**, including `synth-behavior`; the adversarial acceptance suite passes; per-query time is reported against Leant’s under equal wall-clock budgets.

**G4 Dependent recursion.**

Structural schemas from recursor metadata, capabilities, dependency-closed and behavioral motive generalization, proof-carrying motives, lowering through the equation compiler with the correspondence route. Exit: map, append, length, zip, index-safe access, and filter with relational contracts, with no handwritten skeleton; the two-products invariant on every recursive result.

**G5 Certified behavior and carriers.**

Contract decomposer with reconstruction terms, certified domain plugins on the `CertificateSpec` shape, function-valued carrier discovery, accumulator invariant synthesis, measure synthesis, well-founded schemas. Exit: defaulted indexing discovered without a sketch (the predecessors’ open fixture); accumulator reversal with a synthesized invariant; a bare solver verdict fails certification; the extended tier reaches its historically accepted cells.

**G6 Scale.**

Persistent indexes, plan serialization and worker restart, parallel lanes with independence certificates, mutual and nested recursion, effect adapters, optional SMT and learned ranking, project-scale evaluation on the new corpus.

## 24 Risks

- **State coupling.** The cost of `saveState/restore` per attempt, heartbeat accounting across branches, and memory of retained frontiers are unmeasured. Mitigation: cursor frames, the two memory profiles, profiling at G1 before the scheduler is tuned<sup>[P5,P9]</sup>.
- **Rank-N coverage.** Leant inherits Djex’s rank-N and Church machinery; Leant2 replaces it with native introduction plus the type-invention ladder. The 69-query Church fixture is the test, and it sits in G2, before the baseline release.
- **Recursion discovery.** Motive generalization and carrier invention are described as bounded ladders without cost data. They are deferred to G4 and G5 so that the baseline does not depend on them.
- **Proof bottlenecks.** Universal contracts need proofs the portfolio may not find. Mitigation: proof-carrying motives, Forge’s checkers for arithmetic, the unresolved queue with growing budgets, and honest `typedOnly` and `testedOnly` outcomes.
- **Evidence provenance.** All current evidence is remote and transcribed. The first local action of G0 is to replay the nine prototype files under a local Lean 4.34.0 toolchain and record raw receipts.
- **Scope creep.** The proposals together describe more than any team can build; the gates above are ordered so that a usable, baseline-passing Leant2 exists at G3 with recursion, carriers, and scale still open.

## A Provenance ledger

Each row names an idea adopted in this document, the proposals that develop it most fully, and the alternatives that were not adopted. “All” means every proposal contains the idea in some form.

| Idea                                                                 | Adopted from        | Alternatives and notes                                                                                                                                        |
|----------------------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Exact <code>Expr</code> as semantic authority; three representations | all; P3, P5, P8, P9 | The earlier rewrite study’s neutral IR was rejected by all nine. P1 keeps only replay plans serializable; P6 names the pair “search plan / constraint state”. |

| Idea                                                                        | Adopted from           | Alternatives and notes                                                                                                |
|-----------------------------------------------------------------------------|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Whole-continuation backtracking                                             | all                    | P2 adds the commit criterion; P3 names the continuation ownership invariant; P1 contrasts with Aesop safe rules.      |
| Separate program and proof judgments                                        | P2, P3, P5, P8         | P4 uses a subtype in experiments and separates in the interface; P9 packs with <b>PSigma</b> when needed.             |
| Pointwise packaging                                                         | P1, P3, P9             | Registered as a checked transformation, not syntactic.                                                                |
| Four constraint kinds                                                       | P1, P7                 | P1 names hard contract, example, preference; P7 adds construction restriction and operational objective.              |
| Result algebra                                                              | P4, P6, P8, P9         | Merged from taxonomies of 5 (P1) to 9 (P4, P9) constructors; <b>fragmentUnderivable</b> from P5; three flags from P9. |
| Trust profiles                                                              | P1, P3, P6, P9         | Names vary; all separate proof axioms from runtime dependencies.                                                      |
| Origin record and epochs                                                    | P9; P1, P6, P7         | “Environment generation” (P1, P3), “epoch” (P6, P7).                                                                  |
| Obligation graph with roles, coupled clusters                               | P3, P6, P7, P8         | P7 makes constructor application a hyperedge; P8 states four invariants.                                              |
| Branch ownership                                                            | P6, P7, P8             | P1 calls it the ownership barrier.                                                                                    |
| Three blocked states                                                        | P2, P6                 |                                                                                                                       |
| Joint versus frozen proof modes                                             | P1, P2, P7             | P2’s proof-service firewall; P4’s candidate identity on reopening.                                                    |
| Lane portfolio with reserved reference lane                                 | P1, P2, P7, P9         | P5 fixes 1 in 8 quanta; P9 the diagonal tuple schedule and FIFO stream.                                               |
| Pressure and priority formulas                                              | P5, P6; P1             | Weights are placeholders to be tuned at G1.                                                                           |
| Cost vector and Pareto frontier                                             | P2, P3, P6, P8         |                                                                                                                       |
| Cursor frames                                                               | P6, P8                 |                                                                                                                       |
| Bidirectional focusing, rule labels                                         | P3, P6, P9             |                                                                                                                       |
| Spine prefixes                                                              | P2, P3, P6, P8, P9     |                                                                                                                       |
| Known-major projections                                                     | P1; P5                 | P5’s projection-first ordering after a heartbeat failure.                                                             |
| Type-invention ladder                                                       | P1, P2, P6, P9         |                                                                                                                       |
| Dictionary identity, two instance modes                                     | P1, P3, P5, P6, P8     |                                                                                                                       |
| Constructive lane with Kripke certificate                                   | P1, P4, P6, P8; Forge  | Forge’s extension round built the checker.                                                                            |
| Dependent elimination with provenance of removed branches                   | P3, P4, P6, P8, P9     | P4 records kernel / exact-fragment / heuristic removal.                                                               |
| Equality repair and transport                                               | P3, P4, P8             |                                                                                                                       |
| Provider index, tiers                                                       | P4, P5, P8, P9         | P1’s synthesis lemmas; P9’s 50-provider cap remark.                                                                   |
| Component recipes and the join                                              | P6, P7                 | P7’s context slices.                                                                                                  |
| Providers with contract lemmas                                              | P3, P5, P8, P9         |                                                                                                                       |
| Carrier grammar and demand rules                                            | P8; P1, P4, P5, P7, P9 | P5 adds behavioral generalization.                                                                                    |
| Recursion routes and schemas                                                | P2, P3, P4, P5, P8, P9 | P4’s recursion descriptor; P9’s <b>RecSketch</b> .                                                                    |
| Recursive-call capabilities                                                 | all                    | P2’s failed run as motivation.                                                                                        |
| Motive by dependency-closed generalization; failure-directed generalization | P2, P6                 |                                                                                                                       |

| Idea                                                              | Adopted from               | Alternatives and notes                                                                                    |
|-------------------------------------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------|
| Proof-carrying motives                                            | all                        | “Certificate algebras” (P5), “refinement motives” (P1), “packed motives” (P3), “certified motive” (P8).   |
| Accumulator invariants as sub-synthesis                           | P2, P3, P4, P6, P8, P9     |                                                                                                           |
| Measure grammar                                                   | P1, P3, P5, P8             |                                                                                                           |
| Two products per recursion plan                                   | P4, P6, P8, P9             | P6’s lowering algorithm from the plan.                                                                    |
| Contract decomposer with reconstruction terms                     | P2, P6, P8, P9             | P2’s necessary / sufficient / equivalent tags.                                                            |
| Typed observations, residual evaluation, hole consistency table   | P2, P3, P6, P7             |                                                                                                           |
| Proof-service portfolio and adapter                               | P1, P3, P5, P6, P9; Forge  | Forge’s frontend is the reference implementation of the adapter.                                          |
| Certified CEGIS with unknown branch                               | P1, P4, P5, P6, P7, P8, P9 | P8’s finite bound; P7’s proof-carrying certifier.                                                         |
| Reversible observational buckets                                  | P1, P2, P3, P5, P9         | Three independent ablations show permanent merging loses solutions.                                       |
| Sound sketch pruning and domain plugin protocol                   | P3, P5, P6, P8, P9; Forge  | P4’s exact affine separator family; P3’s <code>DomainReply</code> .                                       |
| SMT with three uses; typed sketches                               | P2, P5, P6                 | P5’s branch-expanded versus native-first encodings.                                                       |
| Cache keys and qualified negative entries                         | P1, P2, P4, P5, P7, P9     |                                                                                                           |
| Acceptance gate pipeline                                          | all                        | Order and steps merged; P8’s exact-statement matching and source re-elaboration; P2’s declaration bundle. |
| <code>skipKernelTC</code> and <code>async addDecl</code> pitfalls | Forge                      | Not present in any of the nine proposals.                                                                 |
| Receipts                                                          | P3, P7, P8                 |                                                                                                           |
| Isolation profiles                                                | P3, P6, P9                 |                                                                                                           |
| Frontends and worker/controller                                   | P1, P2, P6, P7, P9         | P9 puts recursive bundles at command level.                                                               |
| Compatibility REPL                                                | this document              | Required by Decisions <a href="#">1.1</a> and <a href="#">1.2</a> .                                       |
| Evaluation corpus, leakage rules, adversarial suite               | P2, P4, P5, P6, P7, P8, P9 |                                                                                                           |
| Gated roadmap                                                     | all                        | Reordered so that the baseline release precedes recursion discovery.                                      |

Ideas present in the proposals and *not* adopted: a monolithic `Subtype` answer type (P4’s experiments); persistent native metavariable states across sessions (rejected by P6); permanent observational quotienting outside known finite complete domains (rejected by all); a dedicated learned ranker in the first release (deferred by all); a Haskell-hosted solver process as part of the semantic engine (rejected by all).

## B The baseline corpus

Counts per transcript in `C:\Leant\test` at revision `6bf05ad`. “Queries” is the number of `:synth` lines; “where” the number with a behavioral assertion; “cand.” the number of queries whose golden shows at least one candidate; “uninh.” the “provably uninhabited” verdicts; “ctrl” the negative controls (bounded miss, none passed, preflight error).

| Transcript                             | queries | where | cand. | uninh. | ctrl | Features exercised                                                                                                               |
|----------------------------------------|---------|-------|-------|--------|------|----------------------------------------------------------------------------------------------------------------------------------|
| synth-basic                            | 8       | 0     | 7     | 1      | 0    | products, sums, S/K combinators, Peirce (classical), propositional                                                               |
| synth-behavior                         | 13      | 12    | 9     | 0      | 4    | assertions; <b>where False</b> ; ill-typed and unknown-name controls; verification-window controls                               |
| synth-both-frontier                    | 1       | 0     | 1     | 0      | 0    | engine interleaving (ignored)                                                                                                    |
| synth-church-layered-providers         | 6       | 0     | 6     | 0      | 0    | Church encodings via session providers                                                                                           |
| synth-church-providers                 | 6       | 0     | 6     | 0      | 0    | Church encodings via providers                                                                                                   |
| synth-church-rankn                     | 69      | 0     | 69    | 0      | 0    | nested polymorphic callbacks, <b>Type 1</b> families, strict implicits, seeded compositions                                      |
| synth-church                           | 9       | 0     | 9     | 0      | 0    | Church numerals, pairs, lists                                                                                                    |
| synth-djinn-providers                  | 11      | 0     | 11    | 0      | 0    | session axioms as providers                                                                                                      |
| synth-five-binder-rankn                | 6       | 0     | 6     | 0      | 0    | five-binder rank-N                                                                                                               |
| synth-inductives                       | 13      | 0     | 11    | 2      | 0    | <b>Bool, Ordering, Option, Except, Decidable</b> , session inductives and structures                                             |
| synth-instance-implicit                | 4       | 0     | 4     | 0      | 0    | instance-implicit binders                                                                                                        |
| synth-library                          | 7       | 0     | 7     | 0      | 0    | <b>List.map, foldr, foldl, flatten, append, replicate</b>                                                                        |
| synth-manual                           | 31      | 0     | 31    | 0      | 0    | the manual's tour: state passing, CPS, <b>Option</b> bind, <b>Decidable</b> combinators, classical logic, session <b>opaques</b> |
| synth-parametric-rankn                 | 9       | 0     | 9     | 0      | 0    | parametric rank-N                                                                                                                |
| synth-prove                            | 5       | 0     | 4     | 1      | 0    | <b>:synth</b> inside <b>:prove</b> ; <b>forall p, p or not p</b> reported uninhabited (fragment)                                 |
| synth-provider-argument-rankn          | 2       | 0     | 2     | 0      | 0    | rank-N provider arguments                                                                                                        |
| synth-provider-bound-context-head      | 3       | 0     | 3     | 0      | 0    | bound context heads                                                                                                              |
| synth-provider-constraint-closure      | 6       | 0     | 6     | 0      | 0    | instance-constraint closure                                                                                                      |
| synth-provider-contextual-assignment   | 9       | 0     | 6     | 0      | 3    | contextual assignment; three bounded misses under restricted settings                                                            |
| synth-provider-higher-kind-assignment  | 12      | 0     | 12    | 0      | 0    | higher-kinded provider assignment                                                                                                |
| synth-provider-implicit-visible-result | 8       | 0     | 8     | 0      | 0    | implicit arguments with visible results                                                                                          |
| synth-provider-metadata-fitting        | 2       | 0     | 2     | 0      | 0    | provider metadata                                                                                                                |
| synth-provider-refutation-fallback     | 5       | 0     | 4     | 1      | 0    | refutation retained while providers run; empty inductive                                                                         |
| synth-provider-structural-assignment   | 6       | 0     | 6     | 0      | 0    | structural provider assignment                                                                                                   |
| synth-quantified-provider              | 9       | 0     | 9     | 0      | 0    | quantified providers                                                                                                             |
| synth-quartic-rankn                    | 3       | 0     | 3     | 0      | 0    | quartic rank-N                                                                                                                   |
| synth-query-closed-rankn               | 3       | 0     | 3     | 0      | 0    | closed polytypes at local providers                                                                                              |
| synth-quintic-rankn                    | 2       | 0     | 2     | 0      | 0    | quintic rank-N                                                                                                                   |
| synth-recursive-rankn                  | 4       | 0     | 4     | 0      | 0    | recursive inductive families at <b>Type 1</b>                                                                                    |

| Transcript             | queries | where | cand. | uninh. | ctrl | Features exercised |
|------------------------|---------|-------|-------|--------|------|--------------------|
| synth-six-binder-rankn | 6       | 0     | 6     | 0      | 0    | six-binder rank-N  |
| Total                  | 278     | 12    | 266   | 5      | 7    |                    |

The five “provably uninhabited” queries are `forall a b : Type, a -> b`; `forall a b : Type, Option a -> b`; `forall a b : Type, Pair a b -> Empty`; an empty session inductive under a provider whose instance does not apply; and `forall p : Prop, p or not p`. The first four admit a Lean refutation by instantiating with `Unit` and `Empty`, and by `cases` on the empty inductive; the fifth admits only a fragment certificate under the constructive lane, and a labeled classical candidate under the standard profile.

## C Interface sketches

The following declarations fix the shape of the main interfaces. They are design sketches consistent with the prototypes’ tested API surface, not compiled code.

```
-- Roles guide scheduling; they never affect typing.
inductive Role | program | typeArg | universe | instance | indexEq | contract | termination |
  invariant | reconstruction

structure Obligation where
  goal      : MVarId
  role      : Role
  residual  : Option Expr      -- residual contract on this hole
  deps      : Array MVarId     -- metavariables whose assignment may change this obligation

inductive StepResult
| progress (changed : Array MVarId) (new : Array Obligation)
| solved   (term : Expr) (manifest : Dependencies)
| blocked  (on : Array MVarId) (reason : String)
| yielded  (cursor : Cursor) (charged : Work)
| rejected (kind : RejectKind) (cert : Option Checked)
| unsupported (feature : String)
| interrupted (reason : Interrupt)

structure Lane where
  name : Name
  step : Origin -> Task -> Quantum -> WorkerM StepResult

inductive ProofReply
| proof (p : Expr) (patch : Option AssignmentPatch) -- patch only in joint mode
| refutation (q : Expr)
| reduced (goal : MVarId) (reconstruct : Expr)
| unsupported
| unknown (spent : Work)

inductive DomainReply
| unsupported
| rankingHint (score : Int) -- no logical authority
| necessaryConstraint (c : Expr) (necessity : Expr)
| counterexample (input : Expr) (valid : Expr) (violation : Expr)
| noCompletion (cert : Checked) (scope : GrammarId)

-- Only Leant2.Accept can construct this.
structure Accepted where
```

```
private mk ::  
  program   : Expr  
  proof     : Expr  
  receipt   : Receipt
```

The Forge contract adopted for every certified domain checker is:

```
structure CertificateSpec (Problem Certificate : Type) where  
  denotes : Problem -> Prop  
  check   : Problem -> Certificate -> Bool  
  sound   : forall (p : Problem) (c : Certificate), check p c = true -> denotes p
```

## References

- [1] V. Reshetnikov et al. *Leant: a Djex-based synthesis REPL for Lean 4*. Repository snapshot 6bf05ad78c467989e68290f2d08bbcd40802d485, September 14, 2026. Files consulted: README.md, docs/synth-internals.md, docs/behavioral-synthesis.md, docs/SYNTHESIS\_PROPOSAL.md, src/Leant/Synth/Fragment.hs, test/synth-\*.txt, test/synth-\*.golden, test/run-tests.sh, test-church/behavior-ledger.md, test-recursive/README.md, test-behavioral/README.md, test-context/global-methods/README.md.
- [2] V. Reshetnikov et al. *Djex*. Standalone snapshot e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5; submodule revision pinned by Leant e237e8667190aff38faaa590ce5b12afbfffac452. Files consulted: README.md, docs/architecture.md, docs/rank-n.md, docs/source-typed-evidence-graph.md.
- [3] Leant project. *Rewriting Leant and Djex in Lean 4: Feasibility, Architectural Consequences, Tradeoffs, and Recommended End State*. docs/Leant\_Djex\_Lean4\_Rewrite\_Analysis/, August 2026.
- [4] V. Reshetnikov et al. *Forge: a certificate-producing proof-planning layer above Lean's grind*. Repository snapshot 44d191e. Files consulted: README.md, lean/Forge/Frontend.lean, lean/Forge/Checker/, lean/Forge/Design/Contracts.lean, docs/IDEAS-FROM-LEANT-DJEX.md, docs/STATUS.md.
- [5] Proposal 01, *Leant2: A Lean-Native Architecture for Proof-Carrying Program Synthesis*, docs/proposals/01-lean-native-synthesis-architecture/.
- [6] Proposal 02, *Leant2: A Lean-Native Architecture for Type- and Specification-Directed Synthesis*, docs/proposals/02-architecture-checked-experiments/.
- [7] Proposal 03, *Leant2: Lean-Native Synthesis of Programs and Their Contracts*, docs/proposals/03-programs-and-contracts/.
- [8] Proposal 04, *Leant2: Native Dependent Program Synthesis in Lean*, docs/proposals/04-native-dependent-synthesis/.
- [9] Proposal 05, *Leant2: Native Dependent-Type and Behavioral Program Synthesis in Lean*, docs/proposals/05-dependent-behavioral-synthesis/.
- [10] Proposal 06, *Leant2: Lean-Native Program Synthesis with Dependent Types and Behavioral Contracts*, docs/proposals/06-algorithms-checked-pilot/.
- [11] Proposal 07, *Leant2: Lean-Native Program Synthesis from Types and Behavioral Specifications*, docs/proposals/07-architecture-prototypes/.
- [12] Proposal 08, *Leant2: Lean-Native, Contract-Directed Program Synthesis*, docs/proposals/08-architecture-checked-prototypes/.
- [13] Proposal 09, *Leant2: Native Dependent Program Synthesis in Lean (design)*, docs/proposals/09-native-dependent-design/.
- [14] Lean development team. *The Lean Language Reference*, version 4.34.0: The Type System; Universes; Inductive Types; Recursive Definitions; Instance Synthesis; Elaboration and Compilation; Validating a Lean Proof; the grind tactic.
- [15] Lean community. *Metaprogramming in Lean 4*, chapter on MetaM.
- [16] J. Limperg and A. Halkjær From. *Aesop: White-Box Best-First Proof Search for Lean*. CPP 2023.
- [17] C. Norman and J. Avigad. *Canonical for Automated Theorem Proving in Lean*. arXiv:2504.06239, 2025.
- [18] N. Polikarpova, I. Kuraj, A. Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. PLDI 2016.

- [19] Z. Guo, M. James, D. Justo, J. Zhou, Z. Wang, R. Jhala, N. Polikarpova. *Program Synthesis by Type-Guided Abstraction Refinement*. POPL 2020.
- [20] P.-M. Osera and S. Zdancewic. *Type-and-Example-Directed Program Synthesis*. PLDI 2015.
- [21] J. Lubin, N. Collins, C. Omar, R. Chugh. *Program Sketching with Live Bidirectional Evaluation*. ICFP 2020.
- [22] N. Mulleners, J. Jeuring, B. Heeren. *Program Synthesis Using Example Propagation*. PADL 2023.
- [23] T. Knoth, D. Wang, N. Polikarpova, J. Hoffmann. *Resource-Guided Program Synthesis*. PLDI 2019.
- [24] A. Mohamed et al. *Lean-SMT: An SMT tactic for discharging proof goals in Lean*.
- [25] M. Carneiro. *Lean4Lean: Verifying a Typechecker for Lean, in Lean*.
- [26] R. Dyckhoff. *Contraction-Free Sequent Calculi for Intuitionistic Logic*. JSL 57(3), 1992.
- [27] Axiom Math. *AXLE check tool documentation*. <https://axle.axiommath.ai/v1/docs/tools/check/>.